

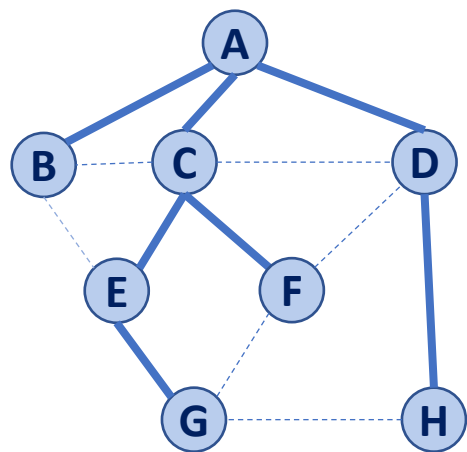
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

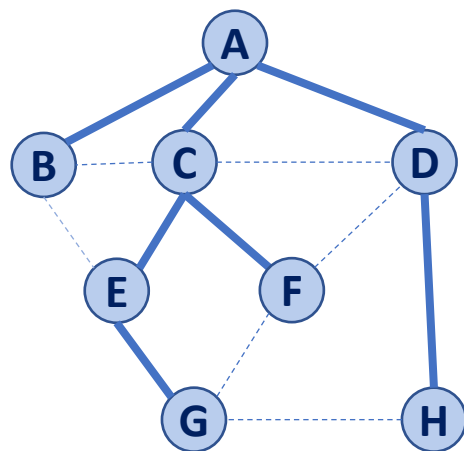
Traversal: BFS



d	p		Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

~~G H F E D B C A~~

Traversal: BFS



Discovery

..... Cross

d	p		Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

→ Init

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```



```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

→ Init

→ ?

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

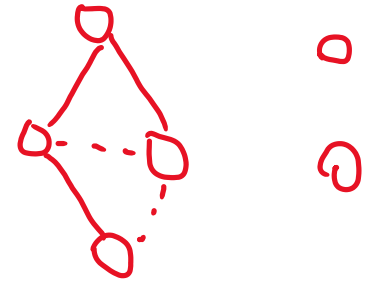
```

1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

?



```

14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)

```

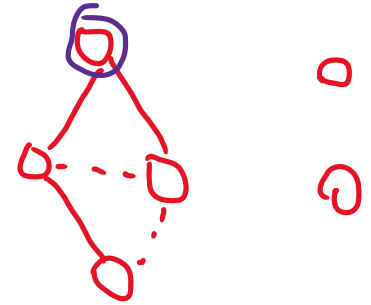
```

1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

?



```

14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)

```

```

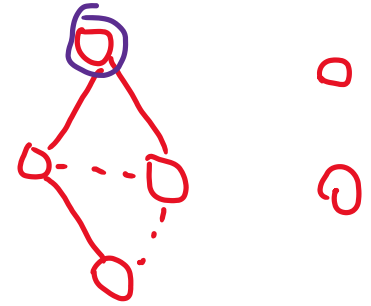
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

Ensure BFS

? runs on every comp?



```

14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)

```

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this?

- *How do we use this to count components?*

Q2: Does our implementation detect a cycle?

- *How do we use this to detect a cycle?*

Q3: What is the running time?

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? *yes :/0 -12*

- ***How do we use this to count components?***

Q2: Does our implementation detect a cycle?

- ***How do we use this to detect a cycle?***

Q3: What is the running time?

```

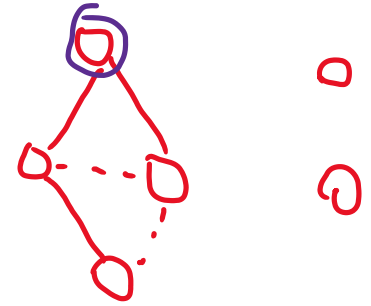
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

Ensure BFS

? runs on every comp?



compCnt++;

```

14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)

```

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? *yes :10 -12*

- ***How do we use this to count components?***

line 12 compCnt++;

Q2: Does our implementation detect a cycle?

- ***How do we use this to detect a cycle?***

Q3: What is the running time?

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? *yes :10 -12*

- ***How do we use this to count components?***

line 12 compCnt++;

Q2: Does our implementation detect a cycle? *yes*

- ***How do we use this to detect a cycle?***

Q3: What is the running time?

```

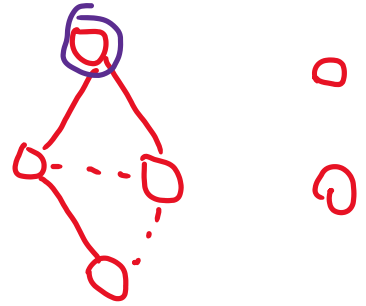
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

Ensure BFS

? runs on every comp?



compCnt++;

cycle = true;

```

14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)

```

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? *yes :10 -12*

- ***How do we use this to count components?***

Line 12 compCnt++;

Q2: Does our implementation detect a cycle? *yes*

- ***How do we use this to detect a cycle?***

Line 27 cycle = true;

Q3: What is the running time?

BFS Analysis

Q1: Does our implementation handle disjoint graphs? If so, what code handles this? *yes :10 -12*

- ***How do we use this to count components?***

Line 12 compCnt++;

Q2: Does our implementation detect a cycle? *yes*

- ***How do we use this to detect a cycle?***

Line 27 cycle = true;

Q3: What is the running time? *$O(n+m)$ Ideal*

```

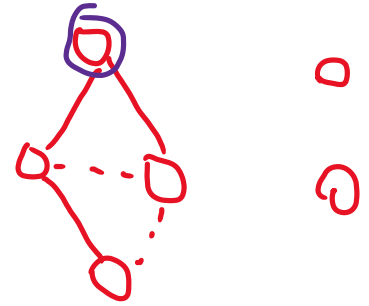
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

Init

Ensure BFS

? runs on every comp?



compCnt++;

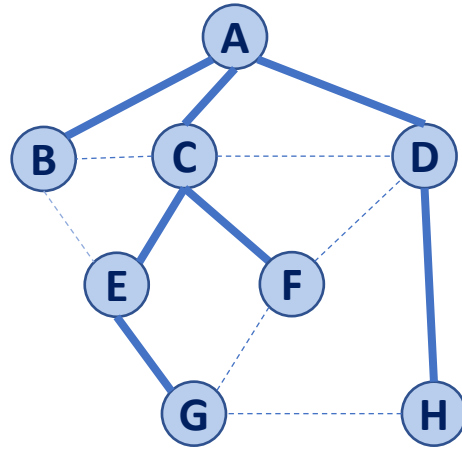
cycle = true;

```

14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)

```

Running time of BFS



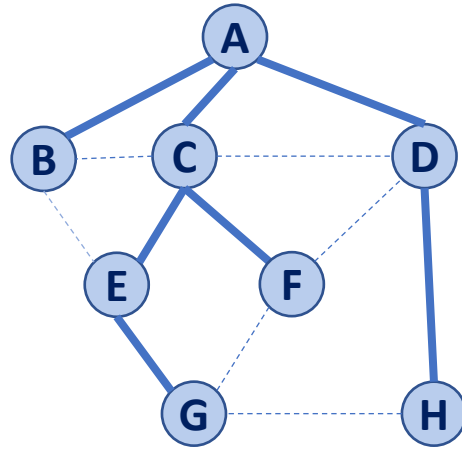
While-loop at **:19?**

For-loop at **:21?**

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



Running time of BFS



While-loop at **:19?**

n total runs

For-loop at **:21?**

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

n elem. ←


```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

n elem.

deg(v)

```

1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)

```

$$\sum_{v \in G} \deg(v) = 2m$$

```

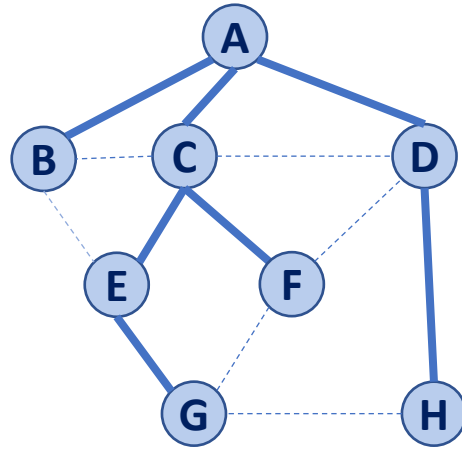
14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)

```

n elem.

deg(v)

Running time of BFS



While-loop at :**19**?

n total runs

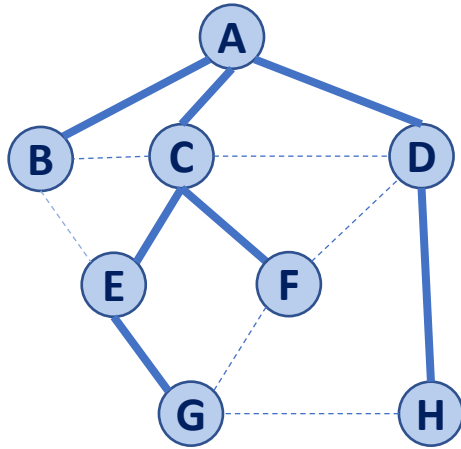
For-loop at :**21**?

2m total runs

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



Running time of BFS



While-loop at :**19**?

n total runs

For-loop at :**21**?

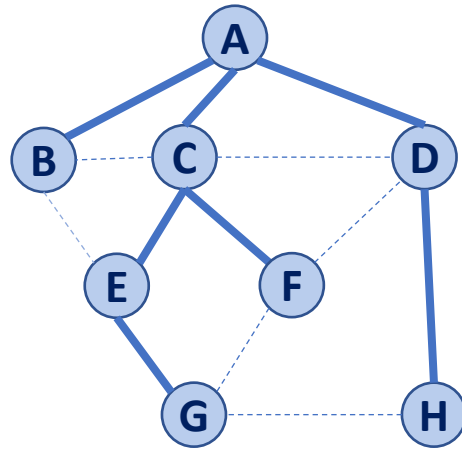
2m total runs

$$n + 2m = O(n + m)$$

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



Running time of BFS



In complete
 $O(n^2)$

While-loop at :19?

n total runs

For-loop at :21?

$2m$ total runs

$n + 2m = O(n + m)$

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

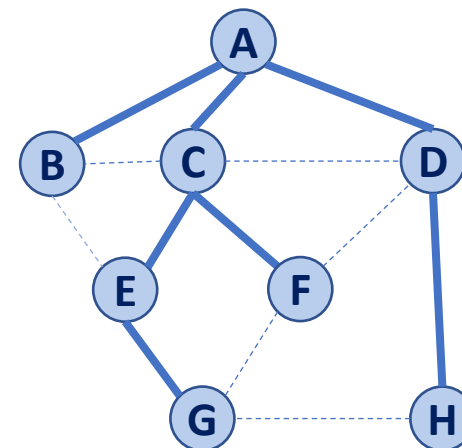
Q1: What is a shortest path from **A** to **H**?

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

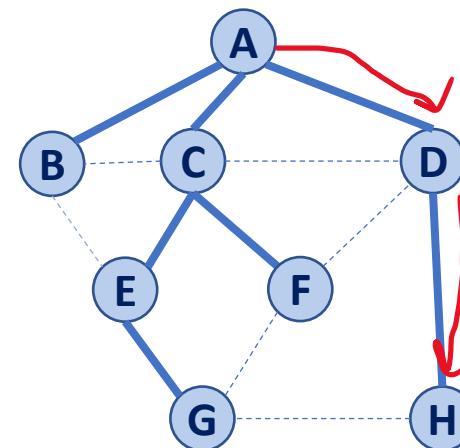
Q1: What is a shortest path from **A** to **H**?

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

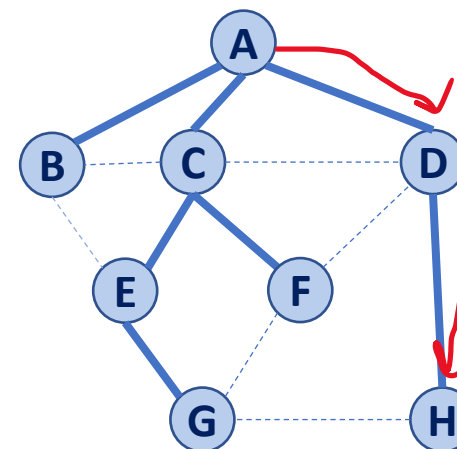
Q1: What is a shortest path from **A** to **H**?

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

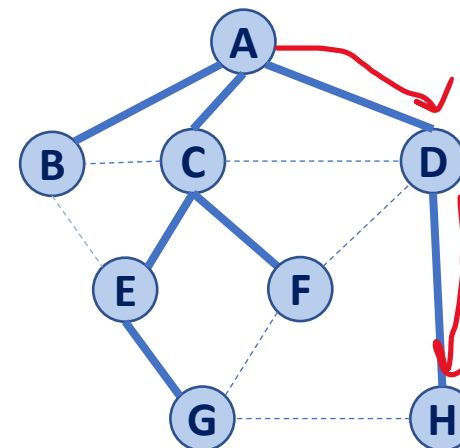
Q1: What is a shortest path from **A** to **H**?

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

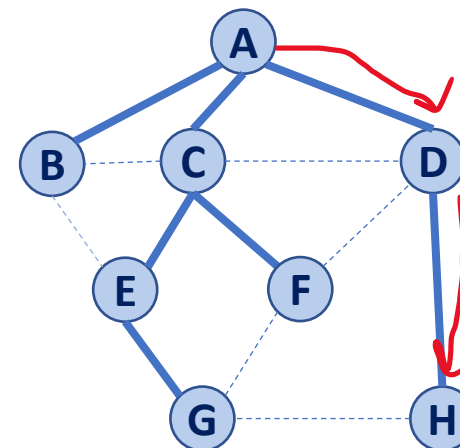
Q1: What is a shortest path from **A** to **H**?

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d=2$

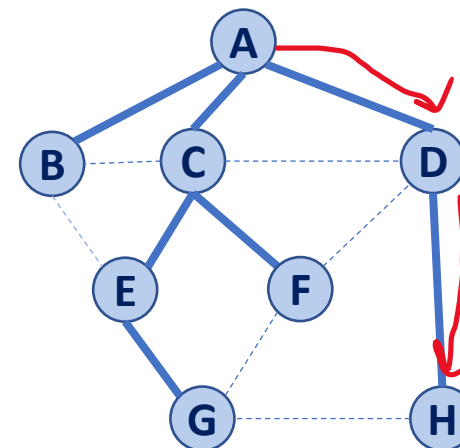
Path found via p from H

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d = 2$

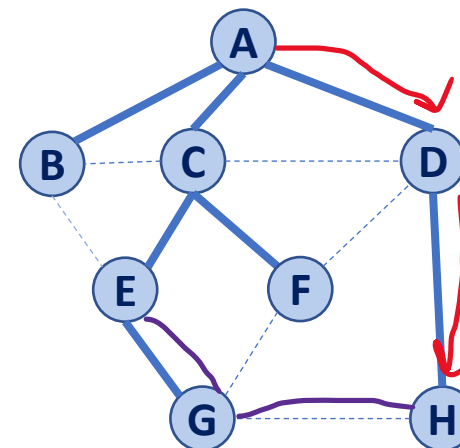
Path found via p from H

Q2: What is a shortest path from **E** to **H**?

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d=2$

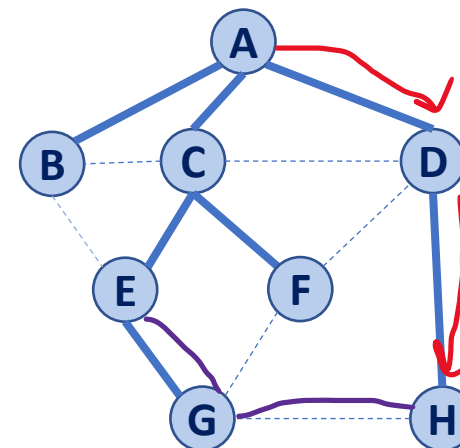
Path found via p from H

Q2: What is a shortest path from **E** to **H**? *We don't know*

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d = 2$

Path found via p from H

Q2: What is a shortest path from **E** to **H**? *We don't know*

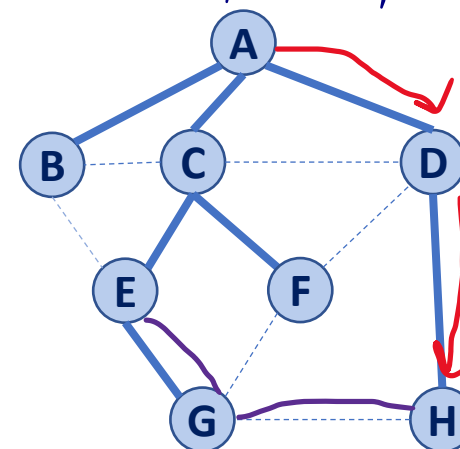
→ BFS gives us a single source

Q3: How does a cross edge relate to **d**?

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

shortest path SSSP



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d = 2$

Path found via p from H

Q2: What is a shortest path from **E** to **H**? *We don't know*

→ BFS gives us a single source

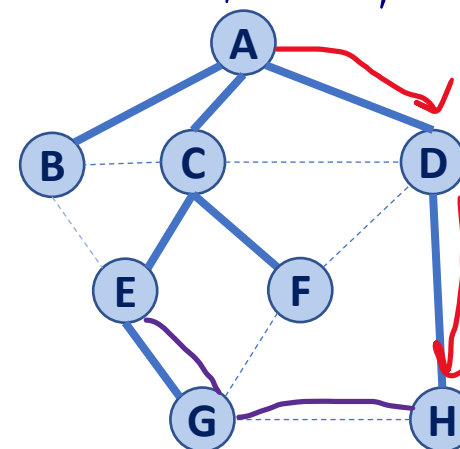
Q3: How does a cross edge relate to **d**?

$$|d(u) - d(v)| \leq 1$$

Q4: What structure is made from discovery edges?

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G

shortest path SSSP



BFS Observations

Q1: What is a shortest path from **A** to **H**? $d = 2$

Path found via p from H

Q2: What is a shortest path from **E** to **H**? *We don't know*

→ BFS gives us a single source

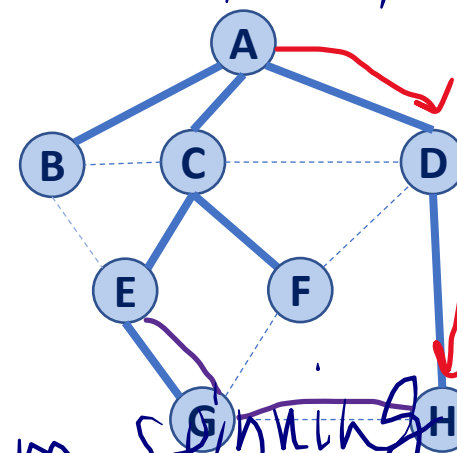
Q3: How does a cross edge relate to **d**?

$$|d(u) - d(v)| \leq 1$$

Q4: What structure is made from discovery edges?

Minimum spanning tree (MST)

d	p	v	Adjacent
0	A	A	C B D
1	A	B	A C E
1	A	C	B A D E F
1	A	D	A C F H
2	C	E	B C G
2	C	F	C D G
3	E	G	E F H
2	D	H	D G



shortest path SSSP

BFS Observations

Obs. 1: Traversals can be used to count components.

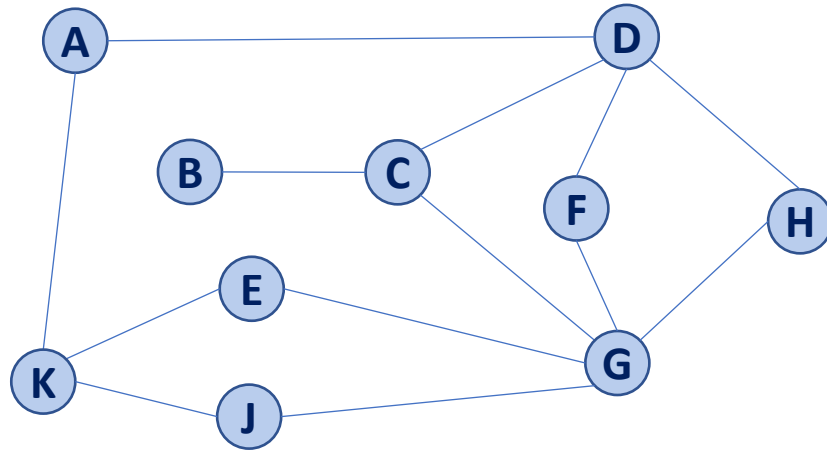
Obs. 2: Traversals can be used to detect cycles.

Obs. 3: In BFS, **d** provides the shortest distance to every vertex.

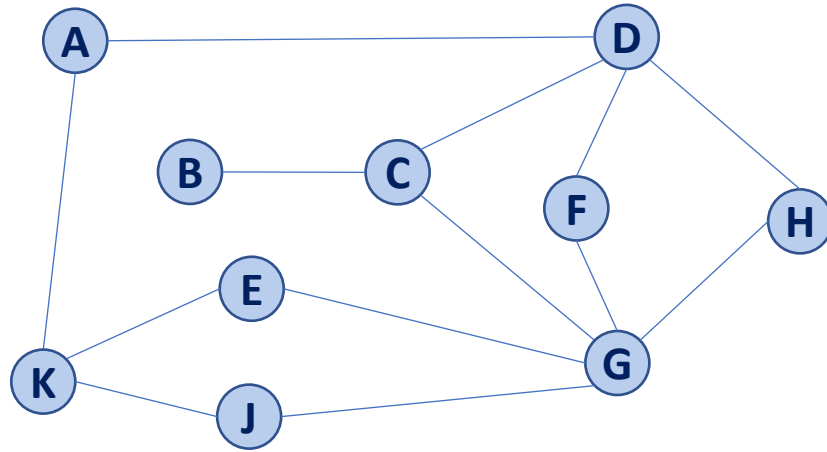
Obs. 4: In BFS, the endpoints of a cross edge never differ in distance, **d**, by more than 1:

$$|d(u) - d(v)| \leq 1$$

Traversal: DFS

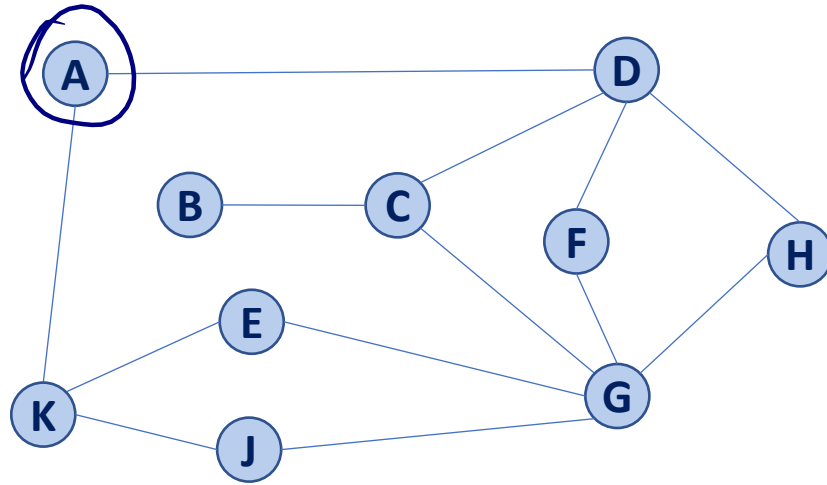


Traversal: DFS



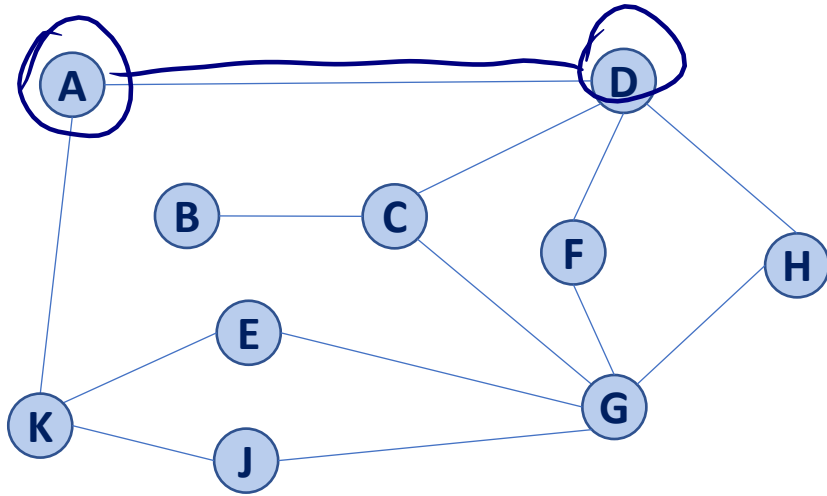
——— Discovery
- - - - - Back

Traversal: DFS



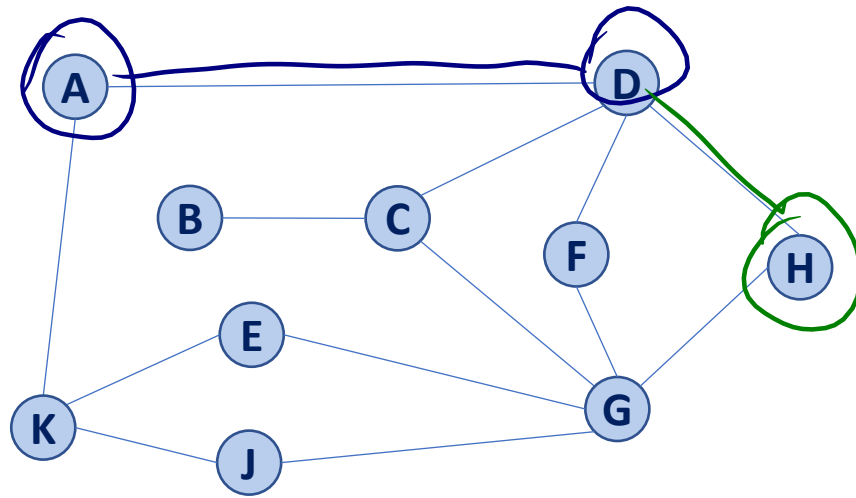
——— Discovery
- - - - - Back

Traversal: DFS



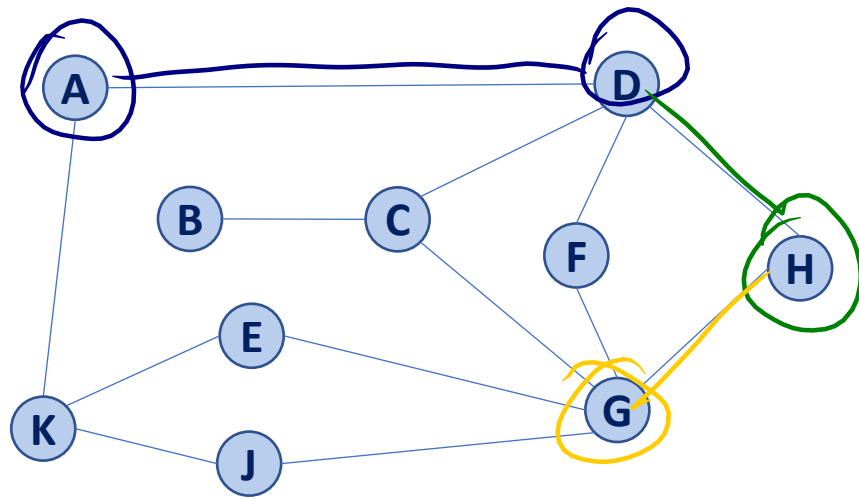
—— Discovering
- - - - Back

Traversal: DFS



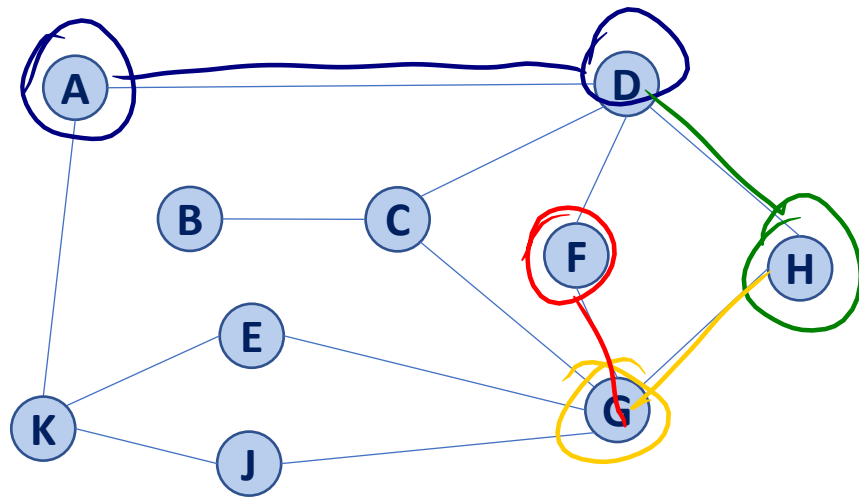
——— Discovery
- - - - - Back

Traversal: DFS



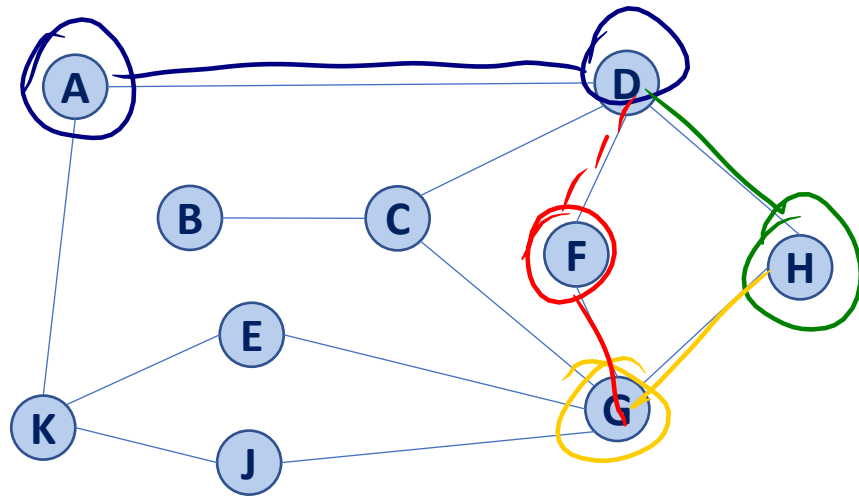
————— Discovery
- - - - - Back

Traversal: DFS



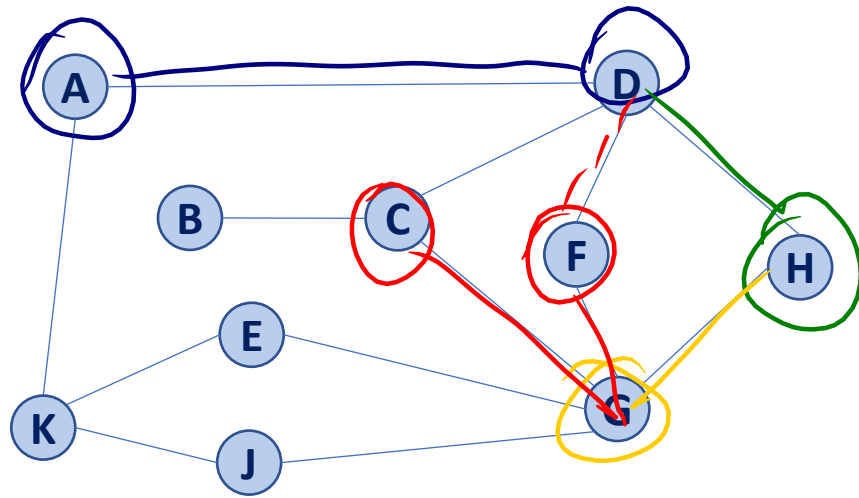
—— Discovering
- - - - Back

Traversal: DFS



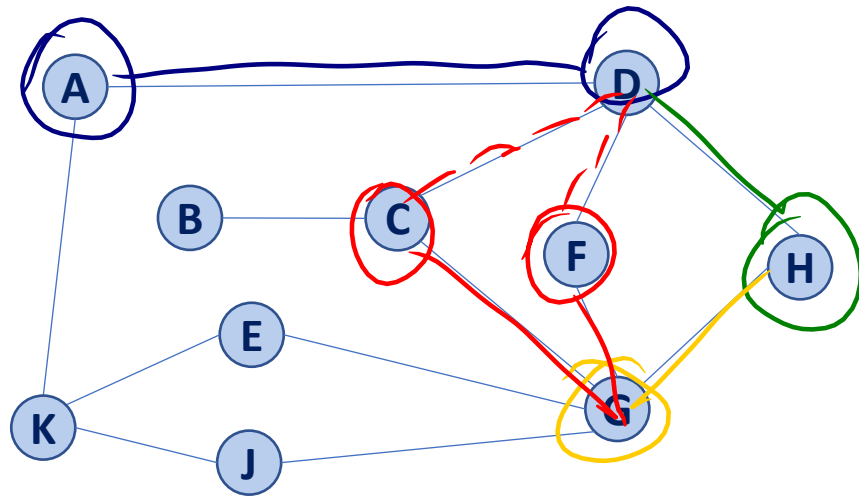
Discovering
Back

Traversal: DFS



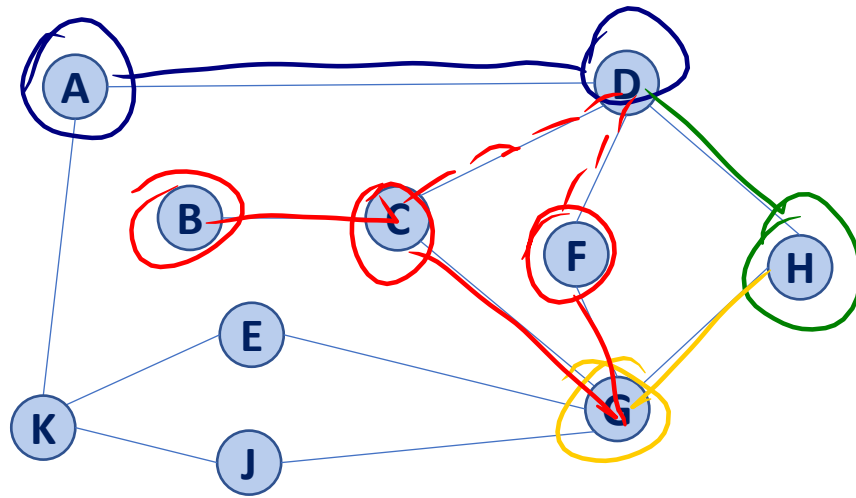
——— Discovery
- - - - - Back

Traversal: DFS



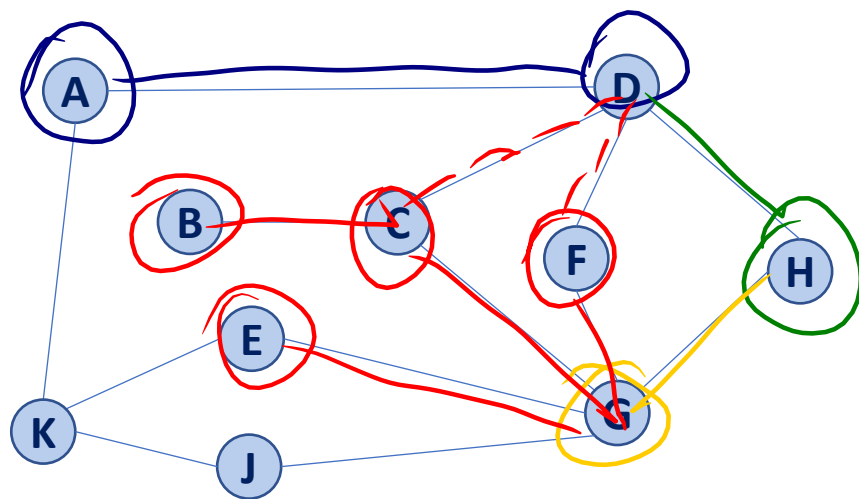
—— Discovery
- - - - Back

Traversal: DFS



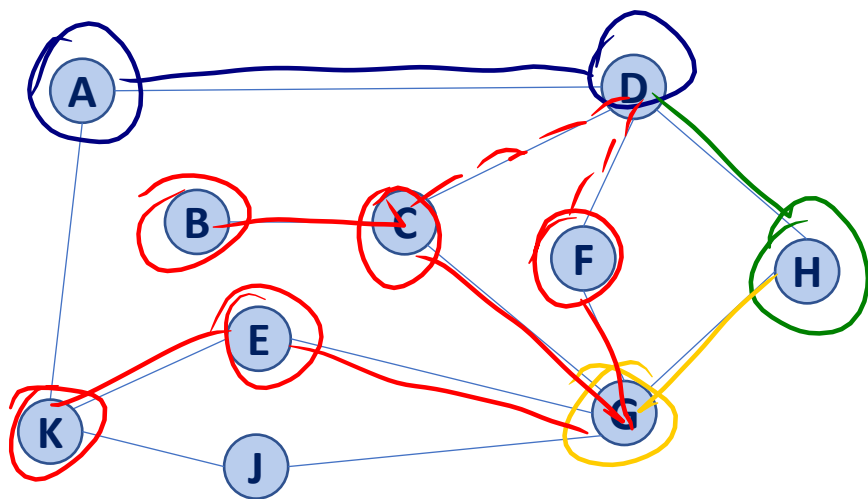
——— Discovery
- - - - - Back

Traversal: DFS



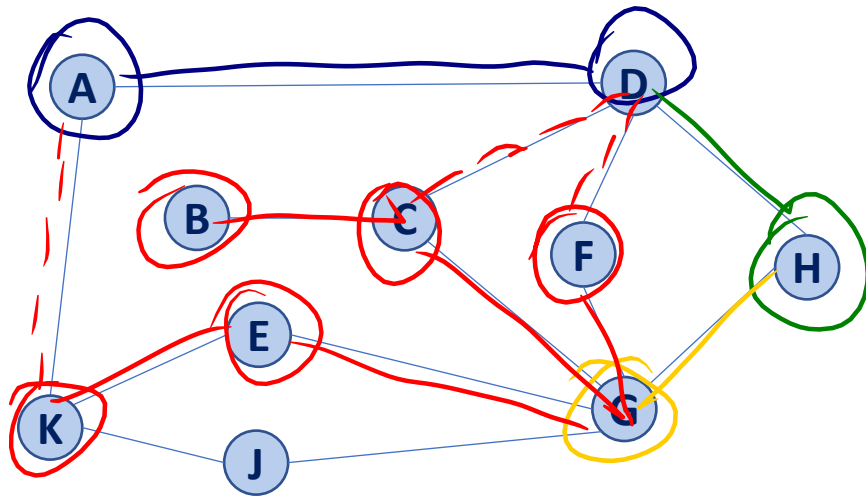
—— Discovering
- - - - - Back

Traversal: DFS



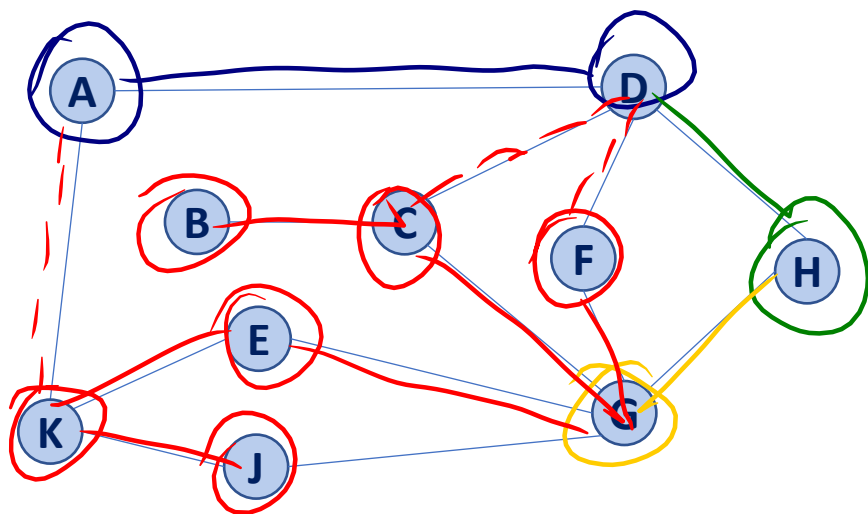
—— Discovery
- - - - Back

Traversal: DFS



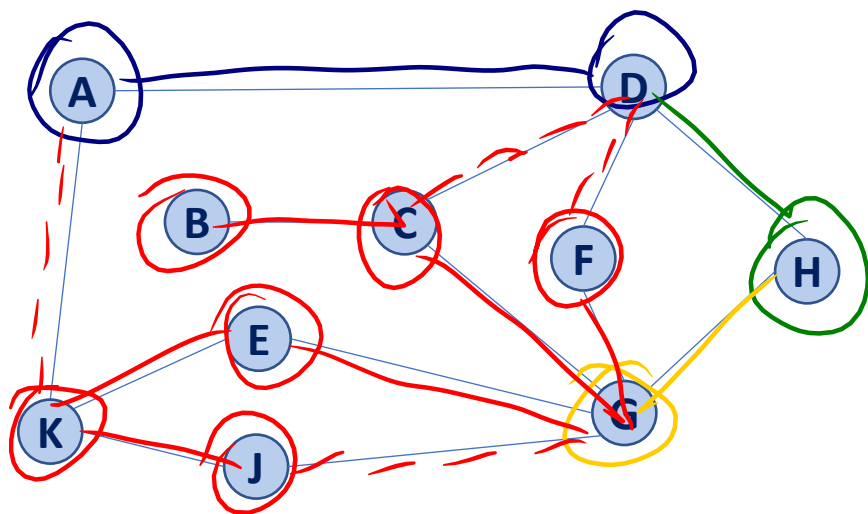
—— Discovery
- - - - Back

Traversal: DFS



————— Discovery
- - - - - Back

Traversal: DFS



—— Discovery
- - - - Back

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty() :
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)) :
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4           G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7       setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9       setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11      if getLabel(v) == UNEXPLORED:  
12          — BFS(G, v)
```

DFS

```
14 BFS(G, v) :  
15     Queue q  
16     setLabel(v, VISITED)  
17     q.enqueue(v)  
18  
19     while !q.empty():  
20         v = q.dequeue()  
21         foreach (Vertex w : G.adjacent(v)):  
22             if getLabel(w) == UNEXPLORED:  
23                 setLabel(v, w, DISCOVERY)  
24                 setLabel(w, VISITED)  
25                 q.enqueue(w)  
26             elseif getLabel(v, w) == UNEXPLORED:  
27                 setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4           G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7       setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9       setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11      if getLabel(v) == UNEXPLORED:  
12          — BFS(G, v)
```

DFS

```
14 BFS(G, v) :  
15     Queue q  
16     setLabel(v, VISITED)  
17     q.enqueue(v)  
18  
19     while !q.empty():  
20         v = q.dequeue()  
21         foreach (Vertex w : G.adjacent(v)):  
22             if getLabel(w) == UNEXPLORED:  
23                 setLabel(v, w, DISCOVERY)  
24                 setLabel(w, VISITED)  
25                 q.enqueue(w)  
26             elseif getLabel(v, w) == UNEXPLORED:  
27                 setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4           G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7       setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9       setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11      if getLabel(v) == UNEXPLORED:  
12          DFS BFS(G, v)
```

DFS

```
14 BFS(G, v) :  
15     Queue q  
16     setLabel(v, VISITED)  
17     q.enqueue(v)  
18  
19     while !q.empty():  
20         v = q.dequeue()  
21         foreach (Vertex w : G.adjacent(v)):  
22             if getLabel(w) == UNEXPLORED:  
23                 setLabel(v, w, DISCOVERY)  
24                 setLabel(w, VISITED)  
25                 q.enqueue(w)  
26             elseif getLabel(v, w) == UNEXPLORED:  
27                 setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4           G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7       setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9       setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11      if getLabel(v) == UNEXPLORED:  
12          DFS BFS(G, v)
```

DFS

```
14 BFS(G, v) :  
15   Queue q  
16   setLabel(v, VISITED)  
17   q.enqueue(v)  
18  
19   while !q.empty() :  
20       v = q.dequeue()  
21       foreach (Vertex w : G.adjacent(v)) :  
22           if getLabel(w) == UNEXPLORED:  
23               setLabel(v, w, DISCOVERY)  
24               setLabel(w, VISITED)  
25           q.enqueue(w)  
26           elseif getLabel(v, w) == UNEXPLORED:  
27               setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4           G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7       setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9       setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11      if getLabel(v) == UNEXPLORED:  
12          DFS BFS(G, v)
```

DFS

DFS(G, w)

```
14 BFS(G, v) :  
15 Queue q  
16   setLabel(v, VISITED)  
17   q.enqueue(v)  
18  
19   while !q.empty() :  
20       v = q.dequeue()  
21       foreach (Vertex w : G.adjacent(v)):  
22           if getLabel(w) == UNEXPLORED:  
23               setLabel(v, w, DISCOVERY)  
24               setLabel(w, VISITED)  
25               DFS q.enqueue(w)  
26           elseif getLabel(v, w) == UNEXPLORED:  
27               setLabel(v, w, CROSS)
```

DFS

```
1 BFS(G) :  
2   Input: Graph, G  
3   Output: A labeling of the edges on  
4         G as discovery and cross edges  
5  
6   foreach (Vertex v : G.vertices()):  
7     setLabel(v, UNEXPLORED)  
8   foreach (Edge e : G.edges()):  
9     setLabel(e, UNEXPLORED)  
10  foreach (Vertex v : G.vertices()):  
11    if getLabel(v) == UNEXPLORED:  
12      DFS  
      BFS(G, v)
```

DFS

DFS(G, w)

```
14 BFS(G, v) :  
15 Queue q  
16   setLabel(v, VISITED)  
17   q.enqueue(v)  
18  
19   while !q.empty() :  
20     v = q.dequeue()  
21     foreach (Vertex w : G.adjacent(v)):  
22       if getLabel(w) == UNEXPLORED:  
23         setLabel(v, w, DISCOVERY)  
24         setLabel(w, VISITED)  
25         q.enqueue(w)  
26       elseif getLabel(v, w) == UNEXPLORED:  
27         setLabel(v, w, CROSS)  
         BACK
```



```
1 DFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and back edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          DFS(G, v)
```

```
14 DFS(G, v) :
15 —Queue q
16   setLabel(v, VISITED)
17 —q.enqueue(v)
18
19 —while !q.empty():
20 —v = q.dequeue()
21   foreach (Vertex w : G.adjacent(v)):
22       if getLabel(w) == UNEXPLORED:
23           setLabel(v, w, DISCOVERY)
24           setLabel(w, VISITED)
25           DFS(G, w)
26       elseif getLabel(v, w) == UNEXPLORED:
27           setLabel(v, w, BACK)
```

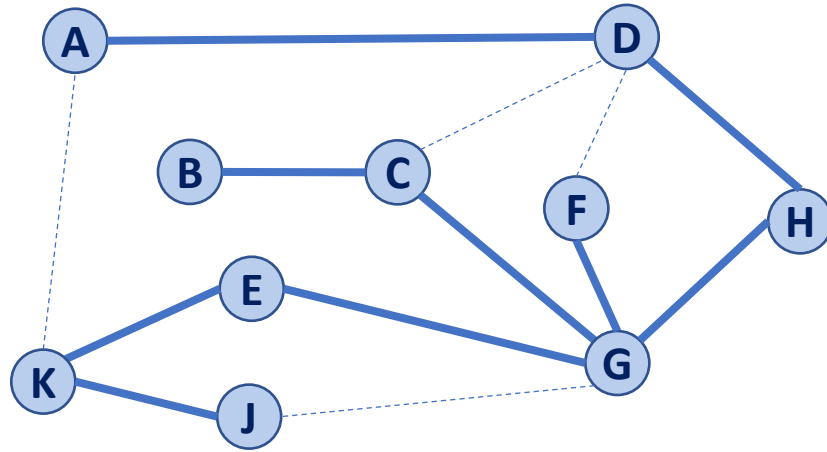
CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Traversal: DFS

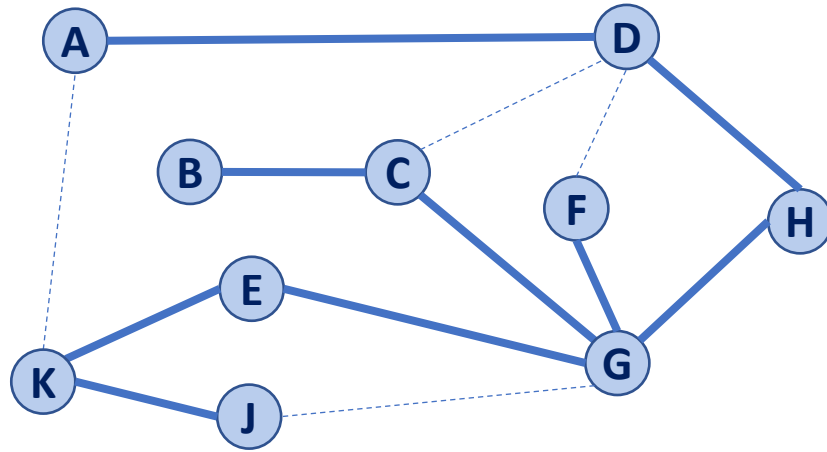


————— Discovery Edge

- - - - - Back Edge

Traversal: DFS

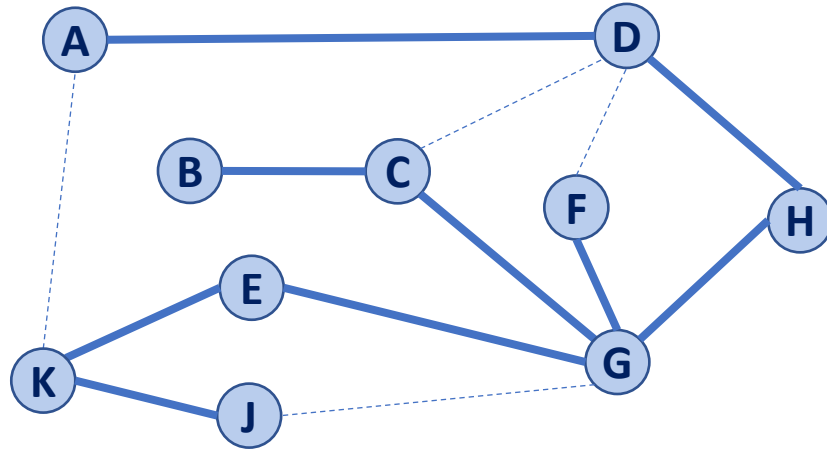
① How vertices are visited



————— Discovery Edge

----- Back Edge

Traversal: DFS

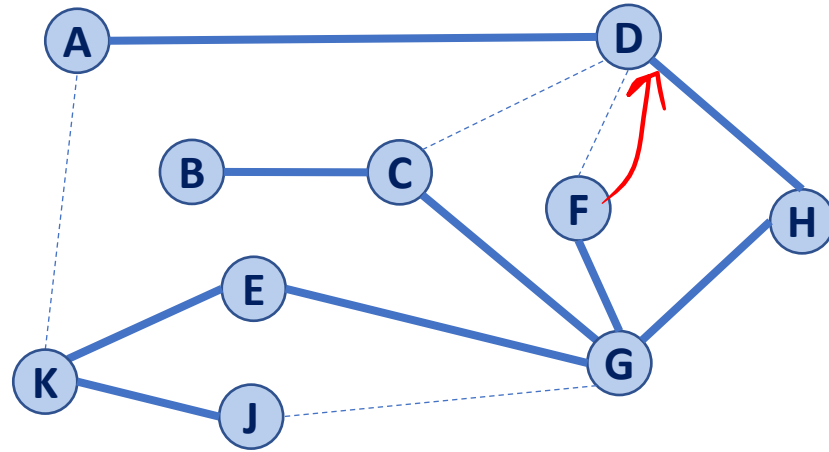


————— Discovery Edge

----- Back Edge

- ① How vertices are visited
- ② The non-discovery edge

Traversal: DFS

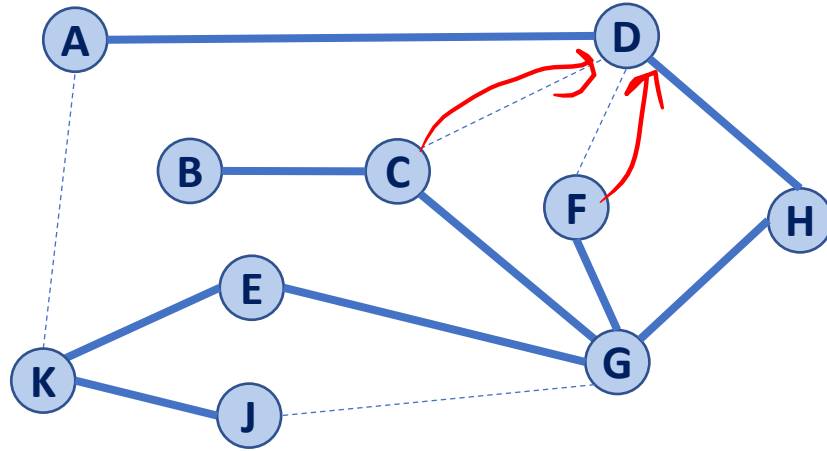


————— Discovery Edge

----- Back Edge

- ① How vertices are visited
- ② The non-discovery edge

Traversal: DFS

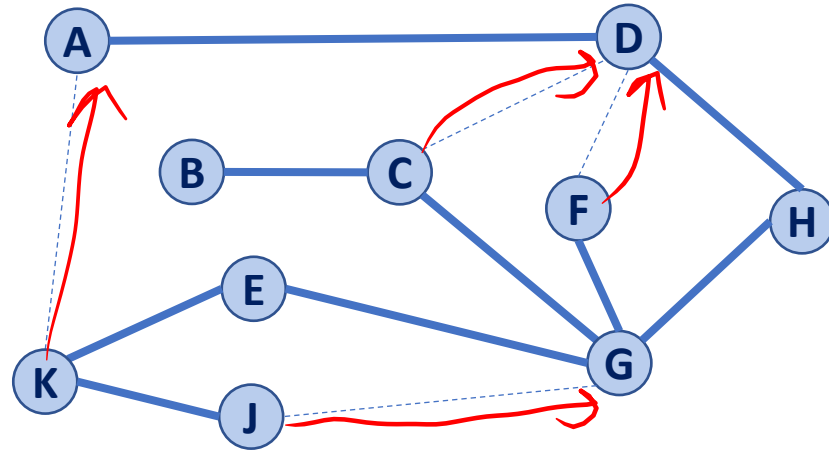


————— Discovery Edge

----- Back Edge

- ① How vertices are visited
- ② The non-discovery edge

Traversal: DFS



- ① How vertices are visited
- ② The non-discovery edge

————— Discovery Edge

----- Back Edge


```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

```
14 BFS(G, v) :
15     Queue q
16     setLabel(v, VISITED)
17     q.enqueue(v)
18
19     while !q.empty():
20         v = q.dequeue()
21         foreach (Vertex w : G.adjacent(v)):
22             if getLabel(w) == UNEXPLORED:
23                 setLabel(v, w, DISCOVERY)
24                 setLabel(w, VISITED)
25                 q.enqueue(w)
26             elseif getLabel(v, w) == UNEXPLORED:
27                 setLabel(v, w, CROSS)
```

```

14 DFS(G, v):
15 Queue q
16   setLabel(v, VISITED)
17 q.enqueue(v)
18
19 while !q.empty():
20 v = q.dequeue()
21   foreach (Vertex w : G.adjacent(v)):
22     if getLabel(w) == UNEXPLORED:
23       setLabel(v, w, DISCOVERY)
24       setLabel(w, VISITED)
25       DFS(G, w)
26   elseif getLabel(v, w) == UNEXPLORED:
27     setLabel(v, w, BACK)

```

```

14 DFS(G, v):
15   Stack s
16   setLabel(v, VISITED)
17   s.push(v)
18
19   while !s.empty():
20     v = s.pop()
21     foreach (Vertex w : G.adjacent(v)):
22       if getLabel(w) == UNEXPLORED:
23         setLabel(v, w, DISCOVERY)
24         setLabel(w, VISITED)
25         s.push(w)
26       elseif getLabel(v, w) == UNEXPLORED:
27         setLabel(v, w, BACK)

```

```

14 DFS(G, v):
15 Queue q
16   setLabel(v, VISITED)
17 q.enqueue(v)
18
19 while !q.empty():
20 v = q.dequeue()
21   foreach (Vertex w : G.adjacent(v)):
22     if getLabel(w) == UNEXPLORED:
23       setLabel(v, w, DISCOVERY)
24       setLabel(w, VISITED)
25       DFS(G, w)
26   elseif getLabel(v, w) == UNEXPLORED:
27     setLabel(v, w, BACK)

```

```

14 DFS(G, v):
15   Stack s
16   setLabel(v, VISITED)
17   s.push(v)
18
19   while !s.empty():
20     v = s.pop()
21     foreach (Vertex w : G.adjacent(v)):
22       if getLabel(w) == UNEXPLORED:
23         setLabel(v, w, DISCOVERY)
24         setLabel(w, VISITED)
25         s.push(w)
26       elseif getLabel(v, w) == UNEXPLORED:
27         setLabel(v, w, BACK)

```

Running time of DFS

Labeling:

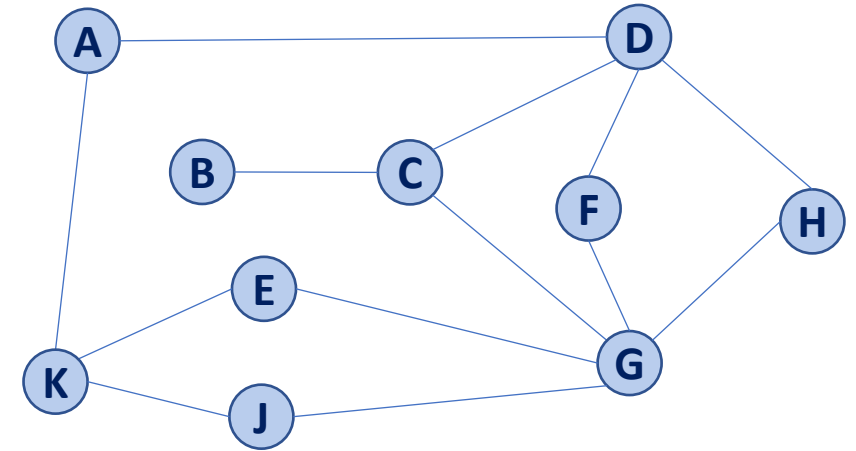
- Vertex:

- Edge:

Queries:

- Vertex:

- Edge:



```
14 DFS(G, v):  
15     setLabel(v, VISITED)  
16     foreach (Vertex w : G.adjacent(v)):  
17         if getLabel(w) == UNEXPLORED:  
18             setLabel(v, w, DISCOVERY)  
19             DFS(G, w)  
20         elseif getLabel(v, w) == UNEXPLORED:  
21             setLabel(v, w, BACK)
```

```
1 BFS(G) :
2   Input: Graph, G
3   Output: A labeling of the edges on
4           G as discovery and cross edges
5
6   foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8   foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10  foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12          BFS(G, v)
```

$\rightarrow O(n)$
 $\rightarrow O(m)$

```
14 BFS(G, v) :
15   Queue q
16   setLabel(v, VISITED)
17   q.enqueue(v)
18
19   while !q.empty():
20       v = q.dequeue()
21       foreach (Vertex w : G.adjacent(v)):
22           if getLabel(w) == UNEXPLORED:
23               setLabel(v, w, DISCOVERY)
24               setLabel(w, VISITED)
25               q.enqueue(w)
26           elseif getLabel(v, w) == UNEXPLORED:
27               setLabel(v, w, CROSS)
```

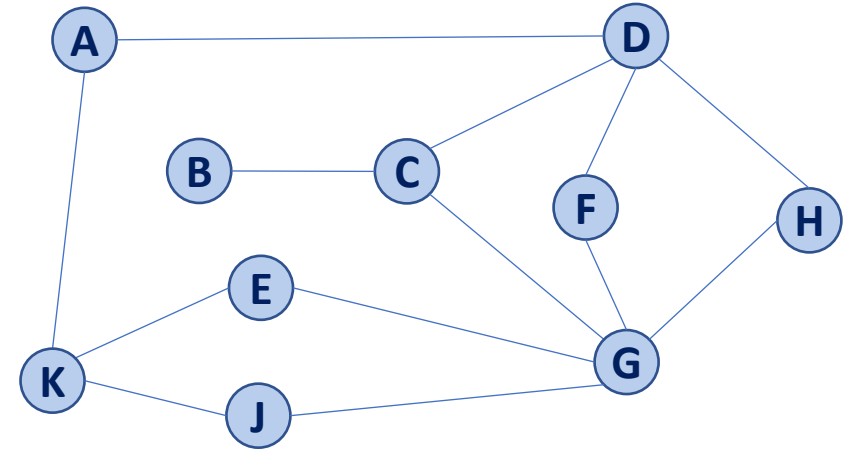
Running time of DFS

Labeling:

- Vertex: $O(n)$
- Edge: $O(m)$

Queries:

- Vertex:
- Edge:



```
14 DFS(G, v):
15     setLabel(v, VISITED)
16     foreach (Vertex w : G.adjacent(v)):
17         if getLabel(w) == UNEXPLORED:
18             setLabel(v, w, DISCOVERY)
19             DFS(G, w)
20         elseif getLabel(v, w) == UNEXPLORED:
21             setLabel(v, w, BACK)
```

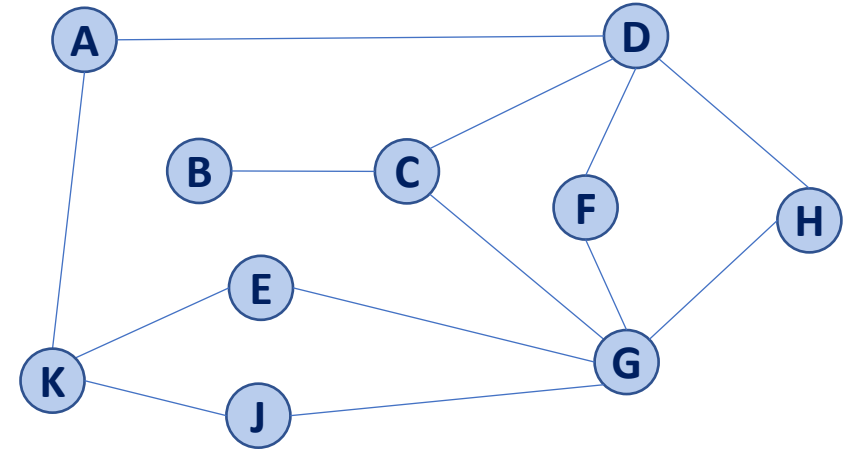
Running time of DFS

Labeling:

- Vertex: $O(n)$
- Edge: $O(m)$

Queries:

- Vertex:
- Edge:



```
14 DFS(G, v):  
15     setLabel(v, VISITED)  
16     foreach (Vertex w : G.adjacent(v)):  
17         if getLabel(w) == UNEXPLORED:  
18             setLabel(v, w, DISCOVERY)  
19             DFS(G, w)  
20         elseif getLabel(v, w) == UNEXPLORED:  
21             setLabel(v, w, BACK)
```

$\rightarrow \text{deg}(v)$

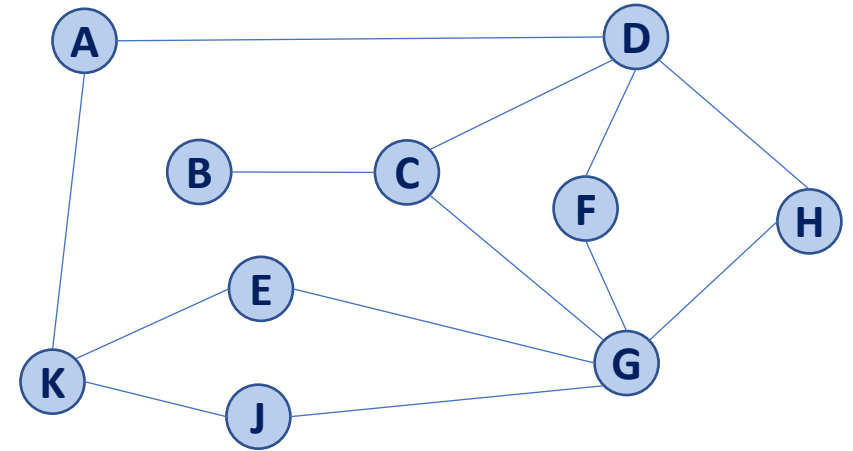
Running time of DFS

Labeling:

- Vertex: $O(n)$
- Edge: $O(m)$

Queries:

- Vertex: $O(m)$
- Edge: $O(m)$



```
14 DFS(G, v):  
15     setLabel(v, VISITED)  
16     foreach (Vertex w : G.adjacent(v)):  
17         if getLabel(w) == UNEXPLORED:  
18             setLabel(v, w, DISCOVERY)  
19             DFS(G, w)  
20         elseif getLabel(v, w) == UNEXPLORED:  
21             setLabel(v, w, BACK)
```

$\rightarrow \text{deg}(v)$

Running time of DFS

Labeling:

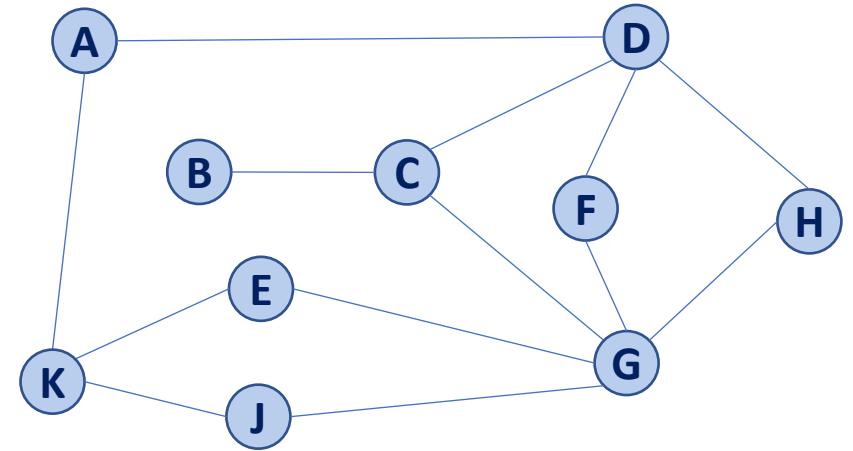
- Vertex: $O(n)$
- Edge: $O(m)$

Queries:

- Vertex: $O(m)$

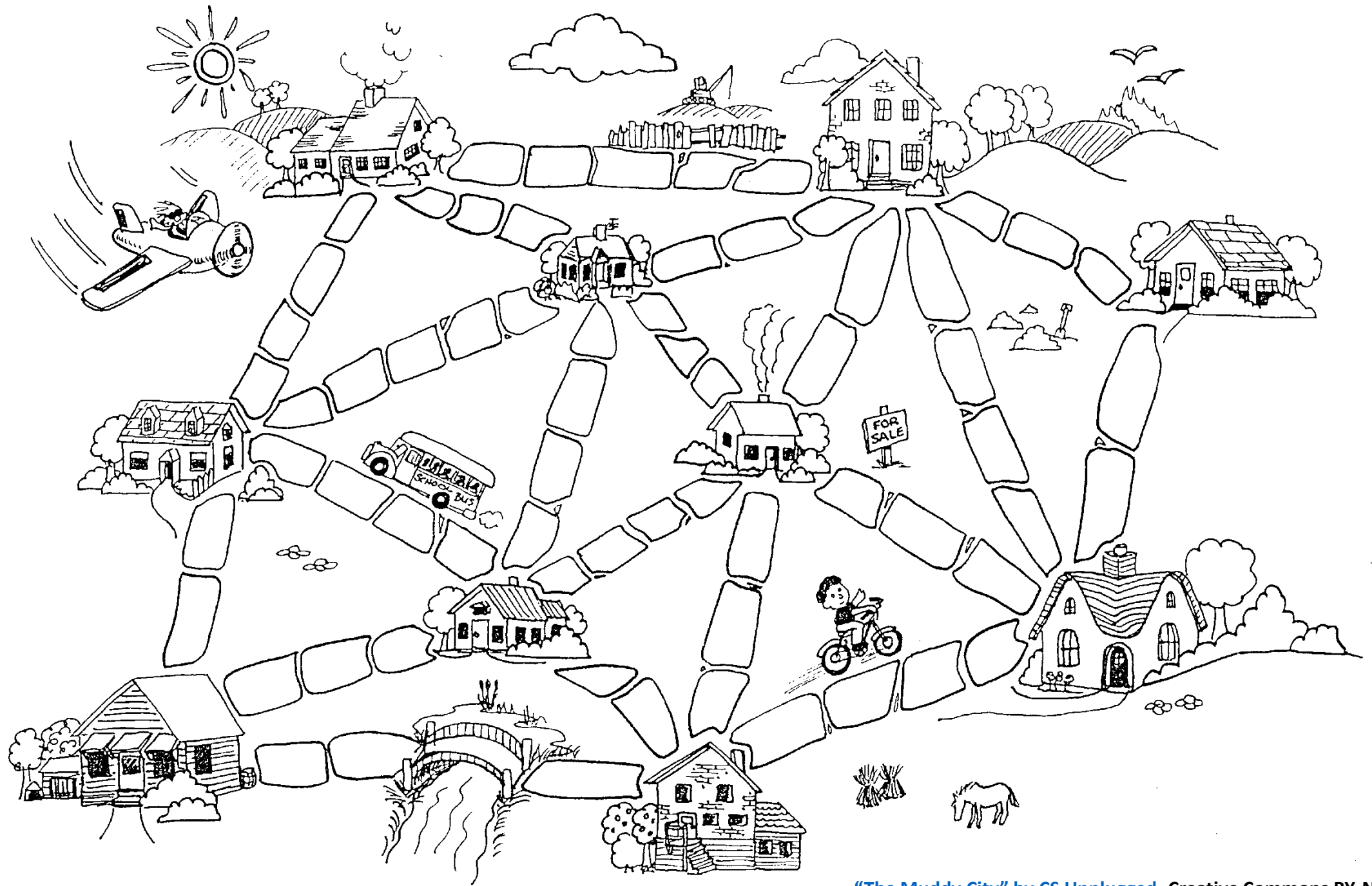
- Edge: $O(m)$

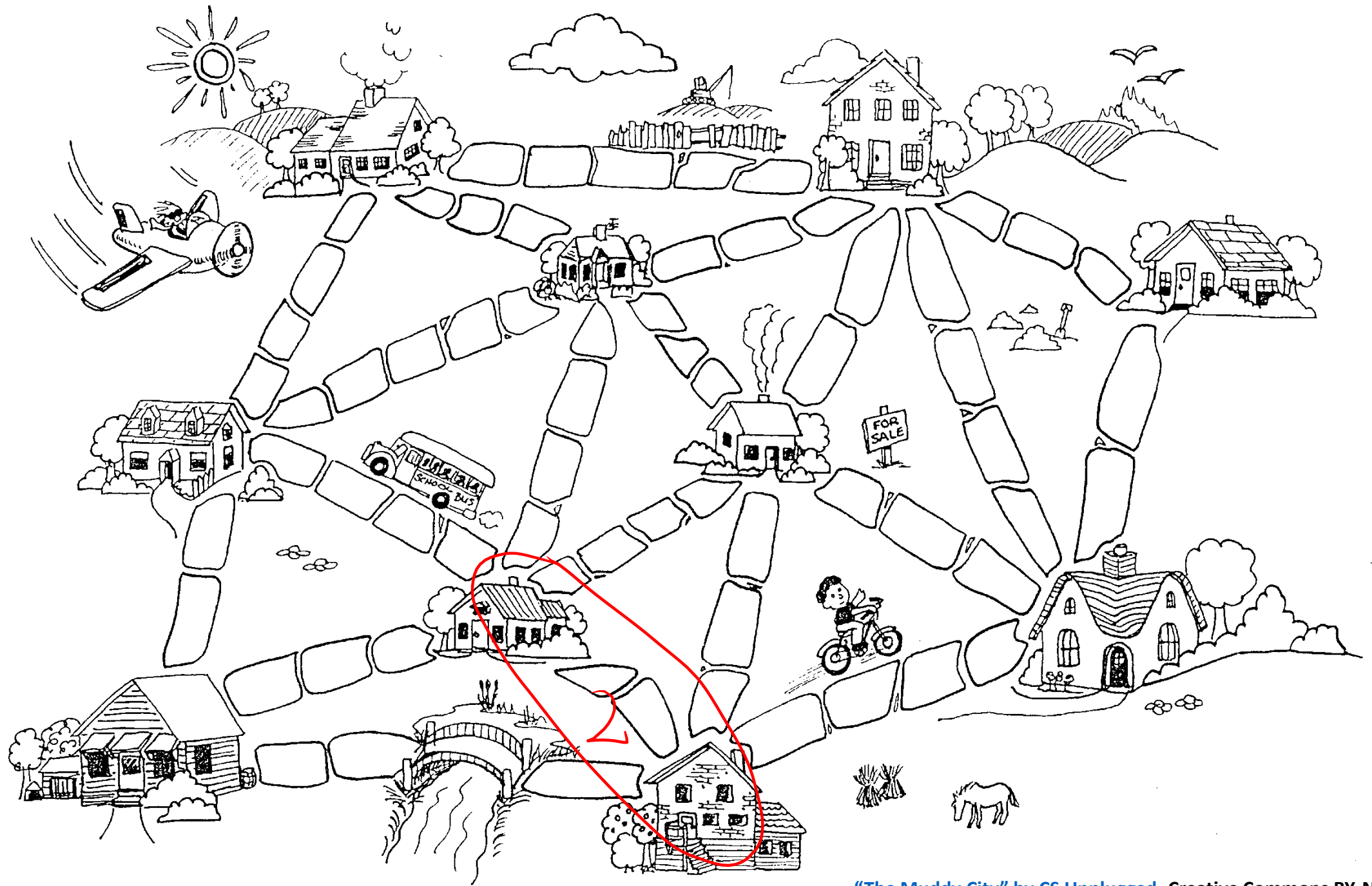
total $O(n+m)$



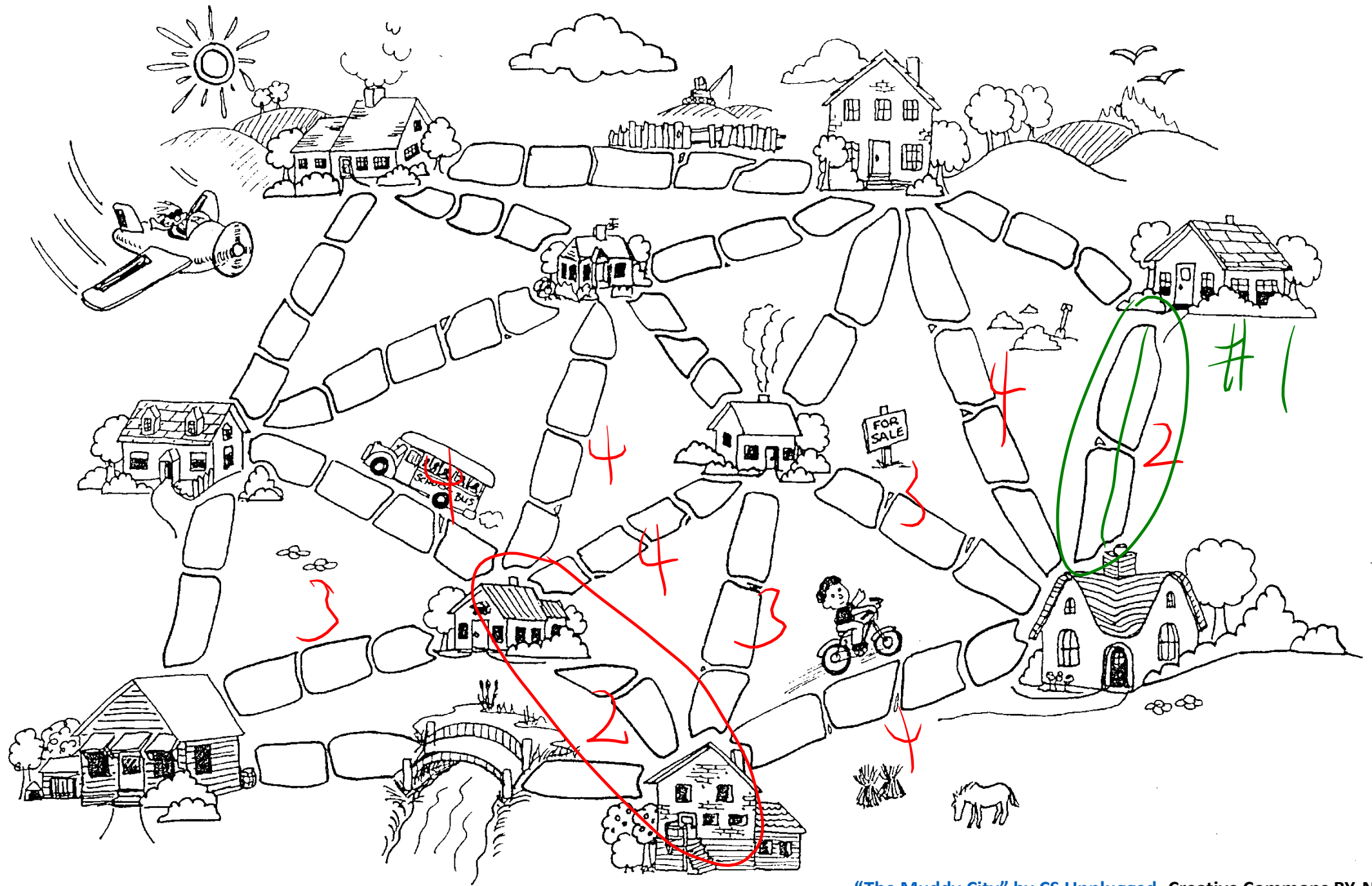
```
14 DFS(G, v):
15     setLabel(v, VISITED)
16     foreach (Vertex w : G.adjacent(v)):
17         if getLabel(w) == UNEXPLORED:
18             setLabel(v, w, DISCOVERY)
19             DFS(G, w)
20         elseif getLabel(v, w) == UNEXPLORED:
21             setLabel(v, w, BACK)
```

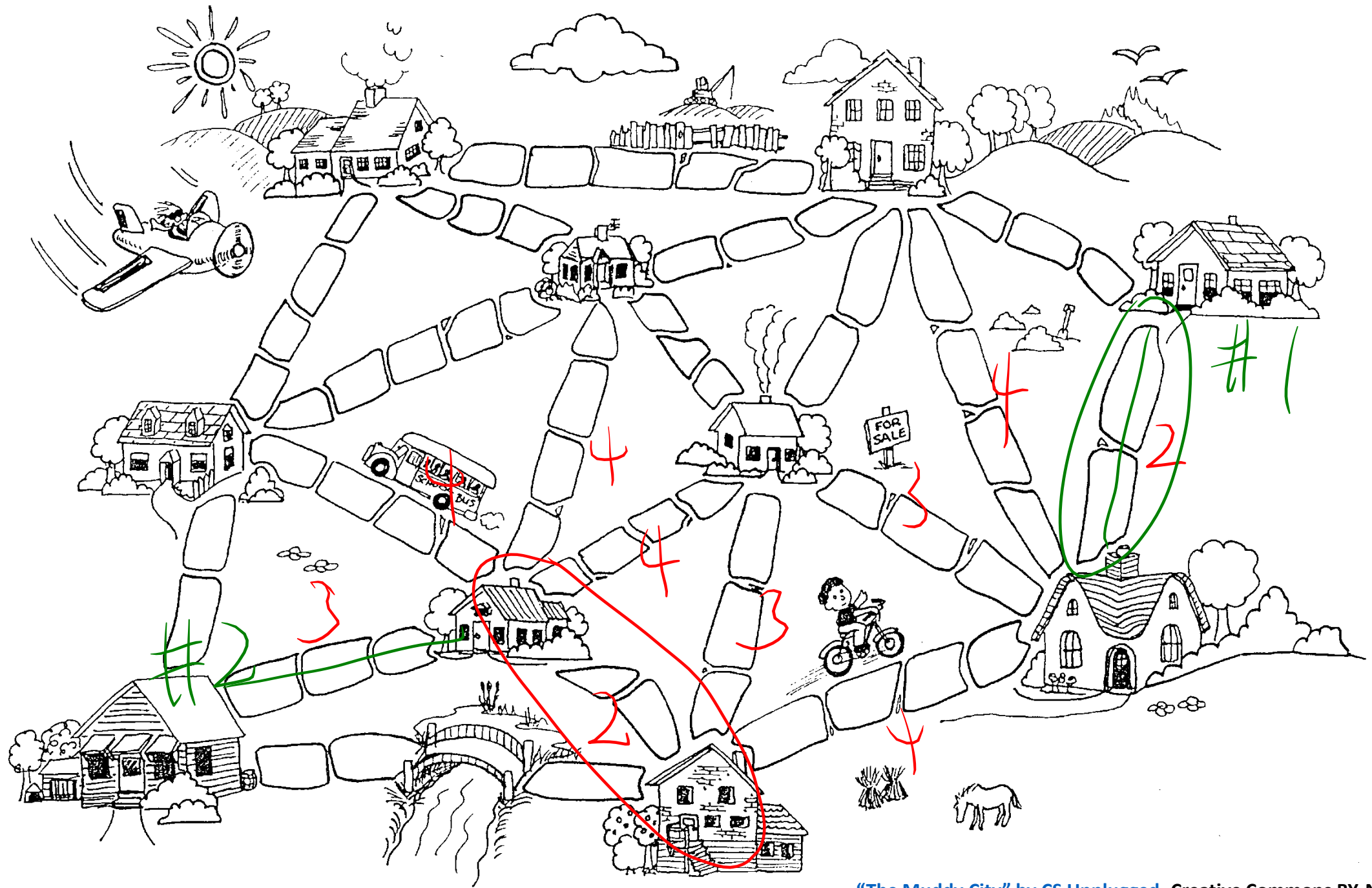
$\rightarrow \text{deg}(v)$

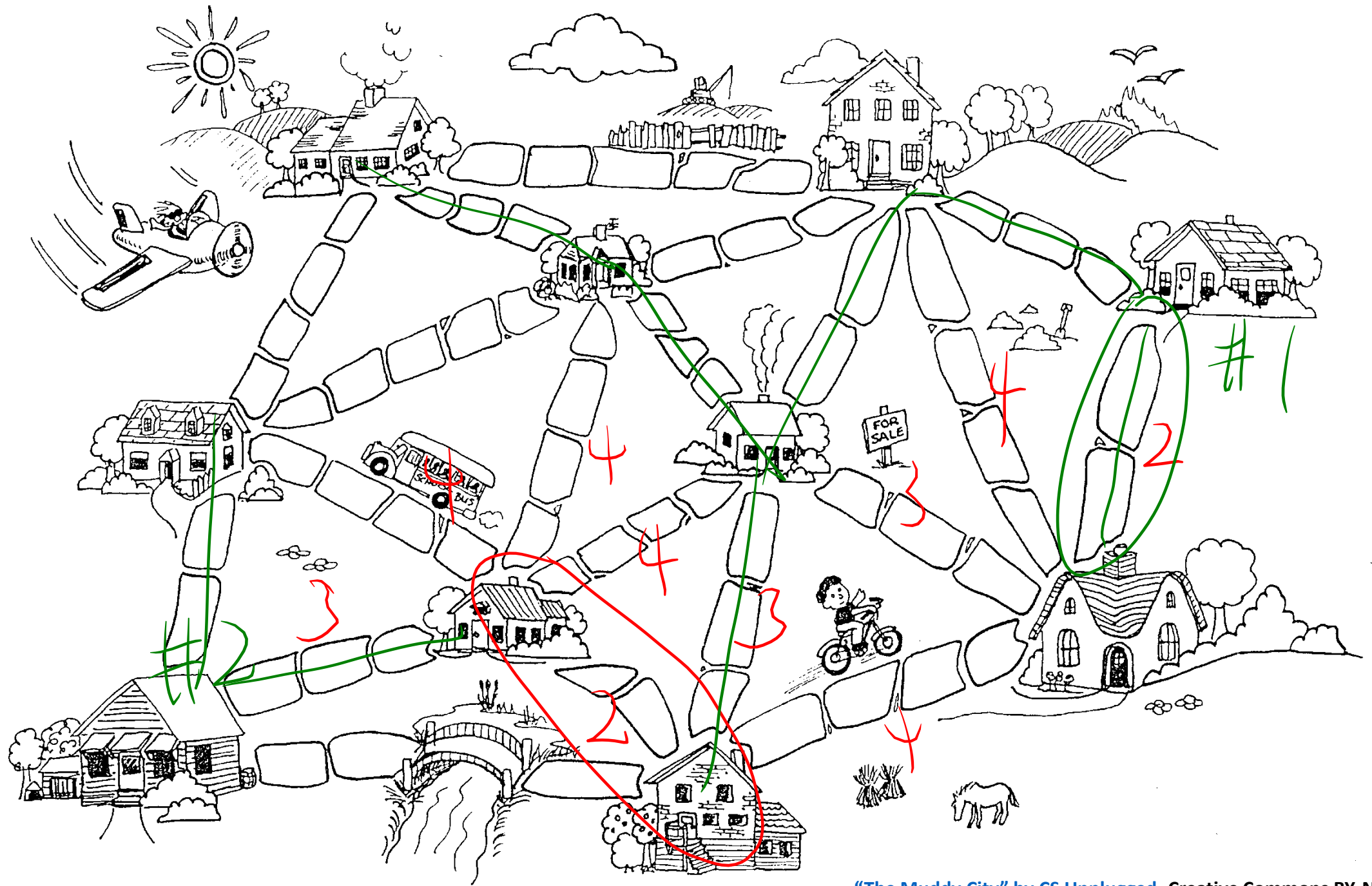










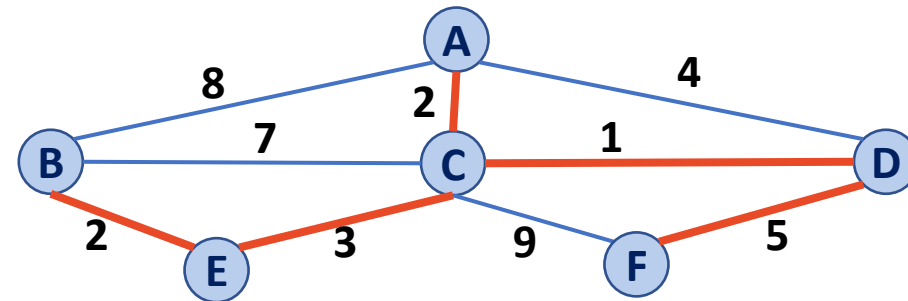


Minimum Spanning Tree Algorithms

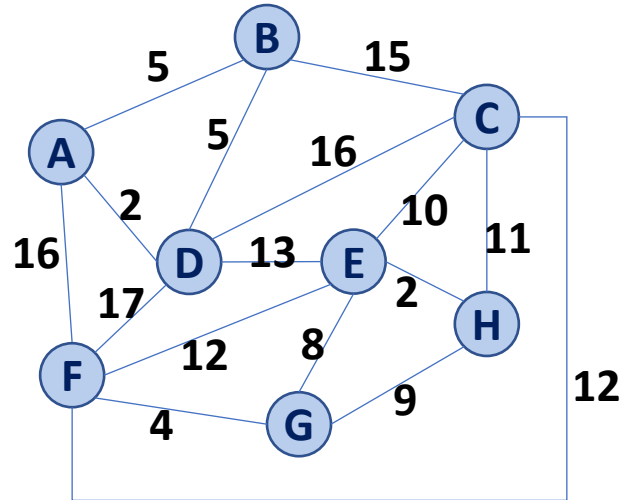
Input: Connected, undirected graph **G** with edge weights (unconstrained, but must be additive)

Output: A graph G' with the following properties:

- G' is a spanning graph of G
- G' is a tree (connected, acyclic)
- G' has a minimal total weight among all spanning trees

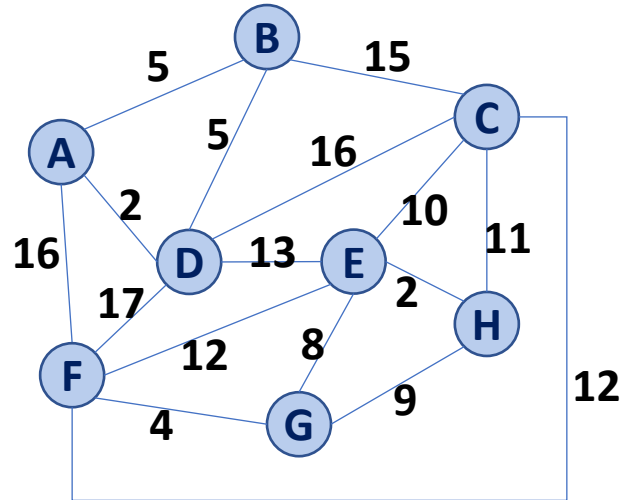


Kruskal's Algorithm



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

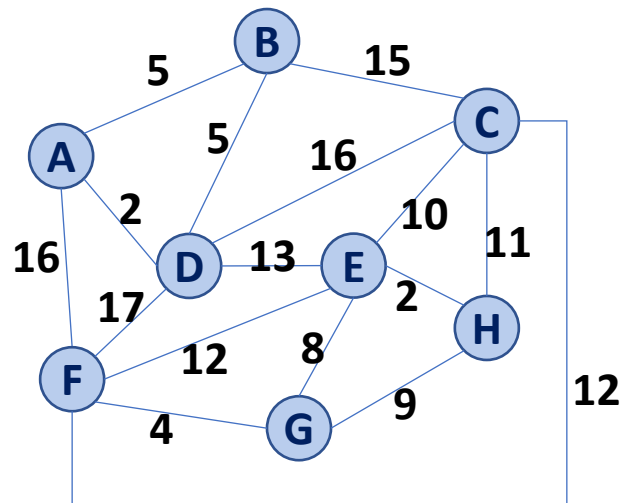
Kruskal's Algorithm



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order

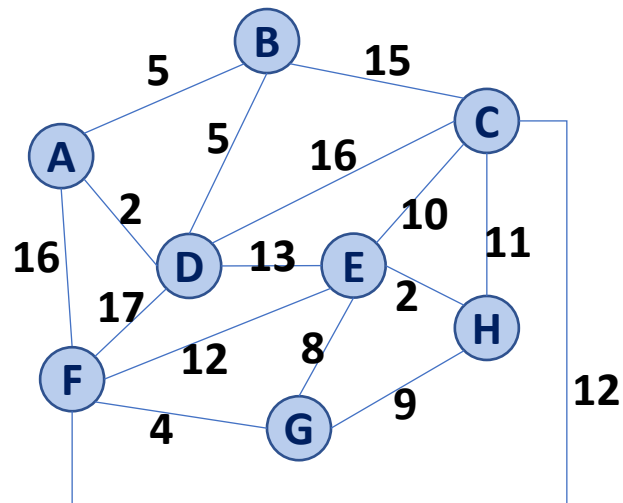
Kruskal's Algorithm



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order
• Heap
• Sorted
array

Kruskal's Algorithm



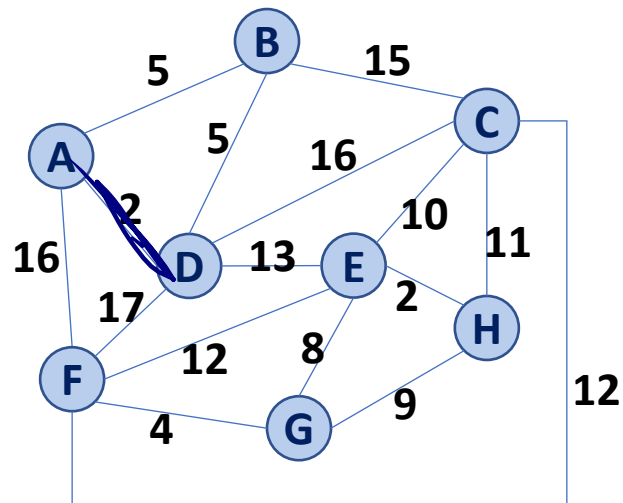
PQ

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order

* Heap
* Sorted
array

Kruskal's Algorithm



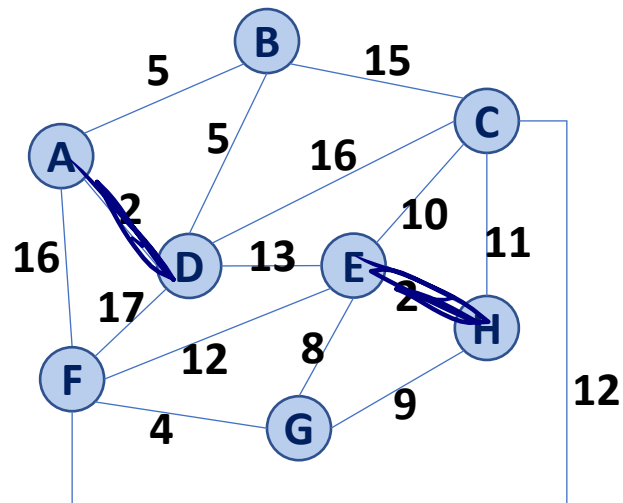
PQ

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order

- Heap
- Sorted array

Kruskal's Algorithm



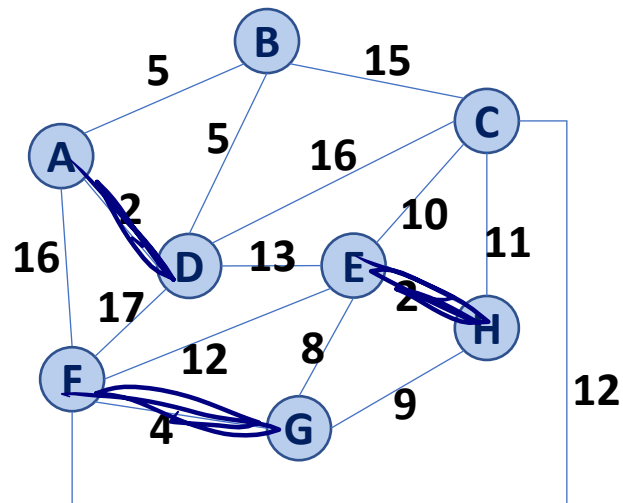
PQ

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order

- * Heap
- * Sorted array

Kruskal's Algorithm



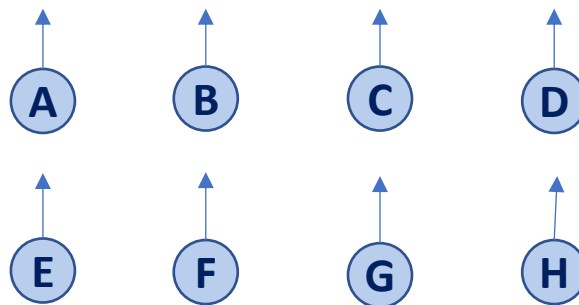
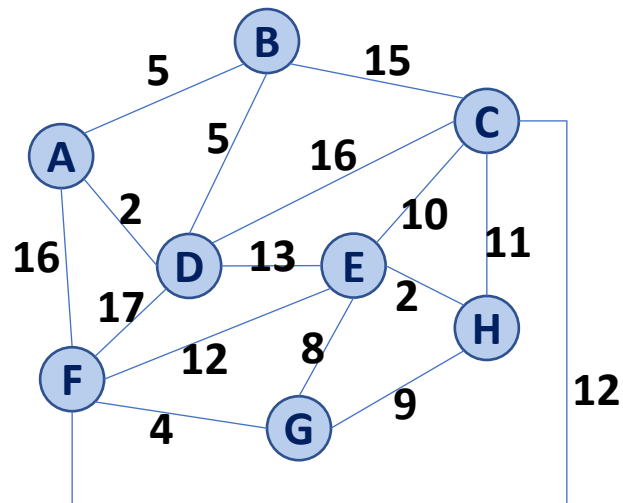
PQ

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

weighted
order

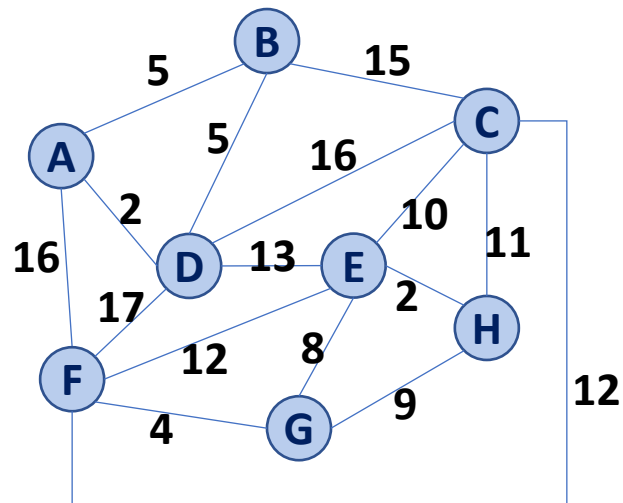
- Heap
- Sorted array

Kruskal's Algorithm

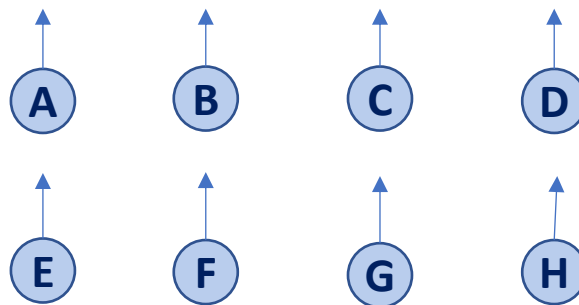


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

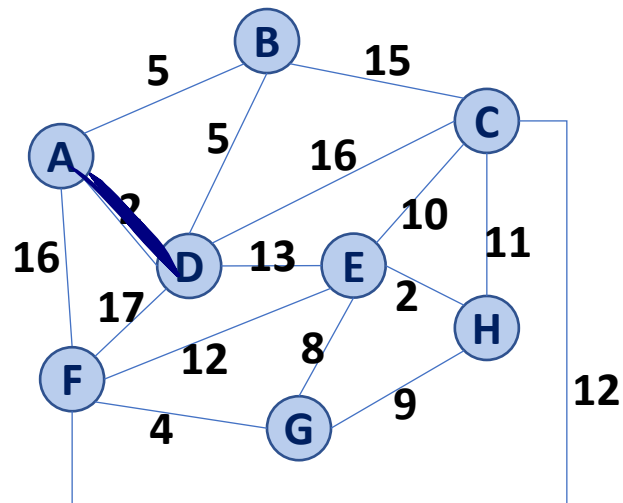


DS

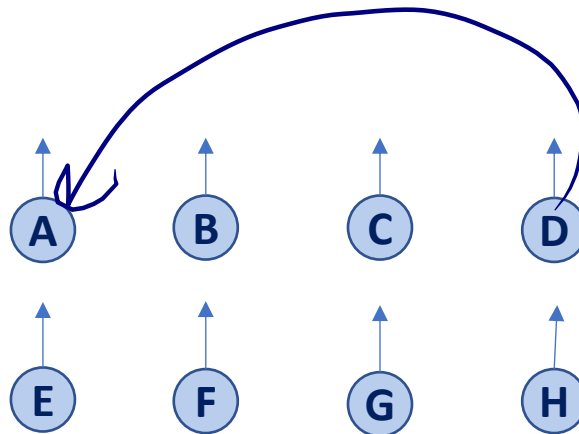


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

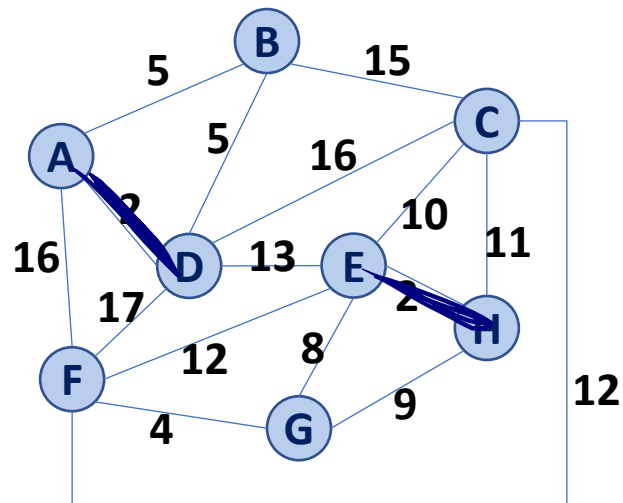


DS

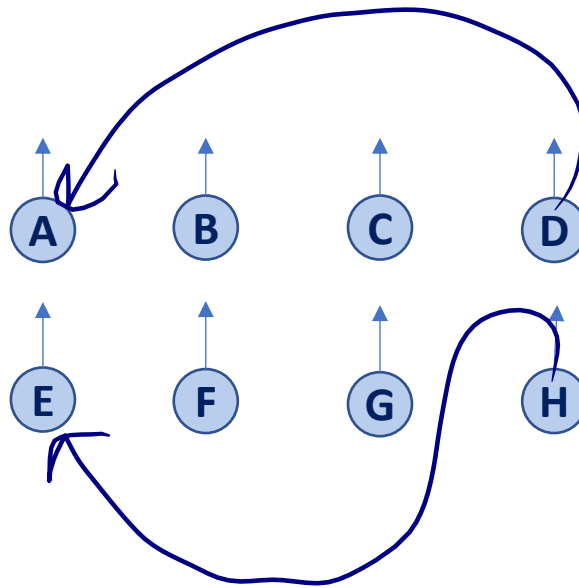


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

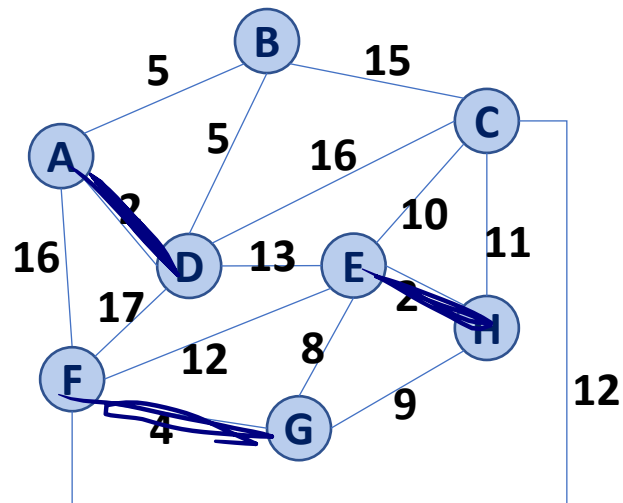


DS

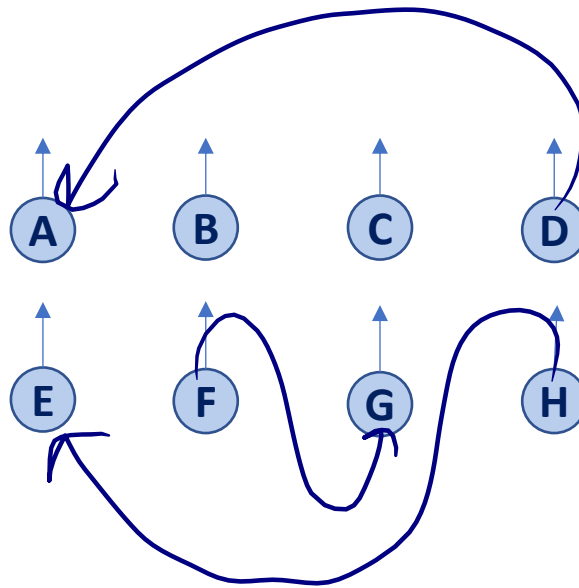


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

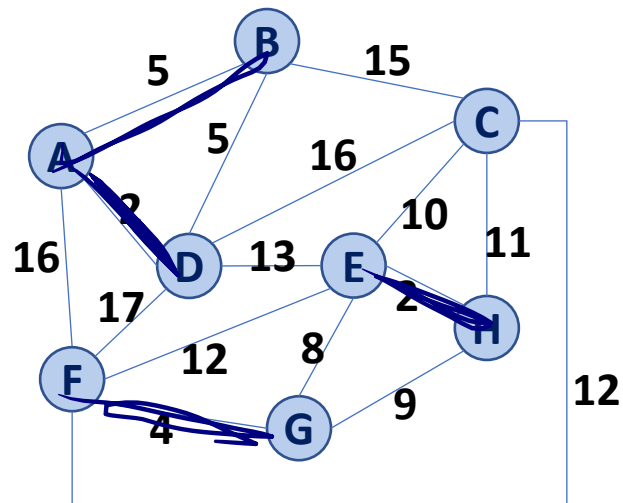


DS

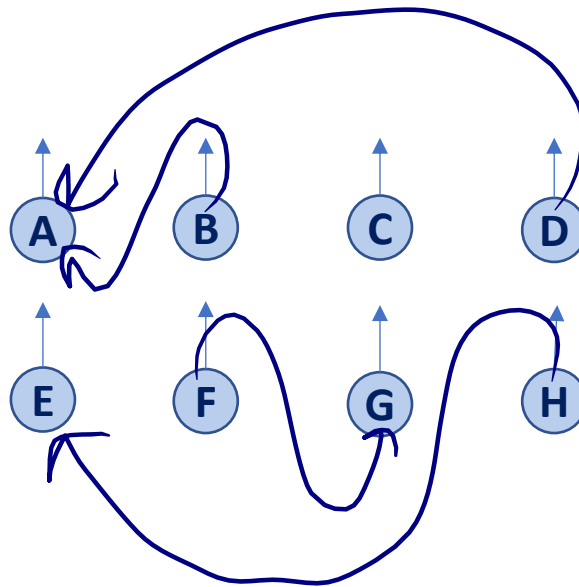


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

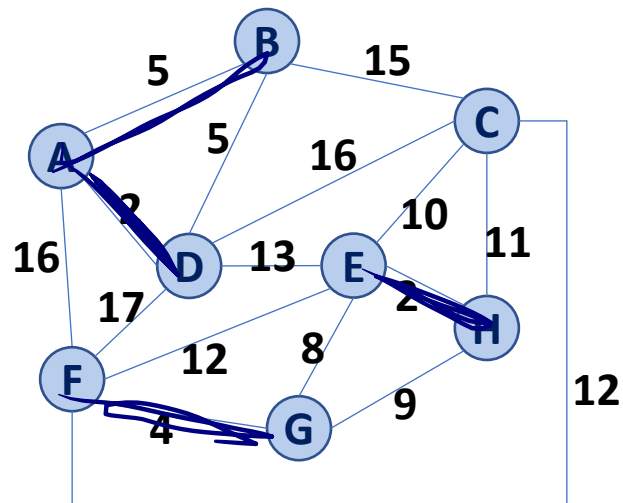


DS

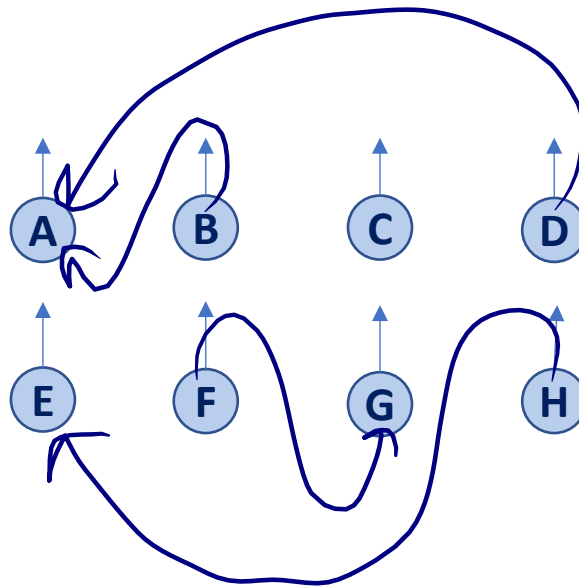


(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm



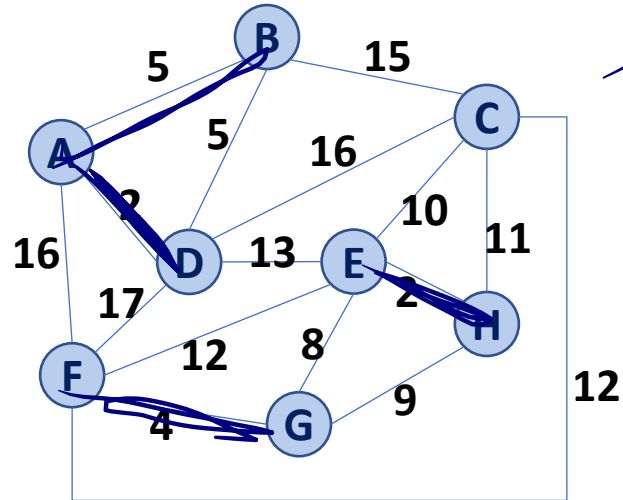
DS



skip

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

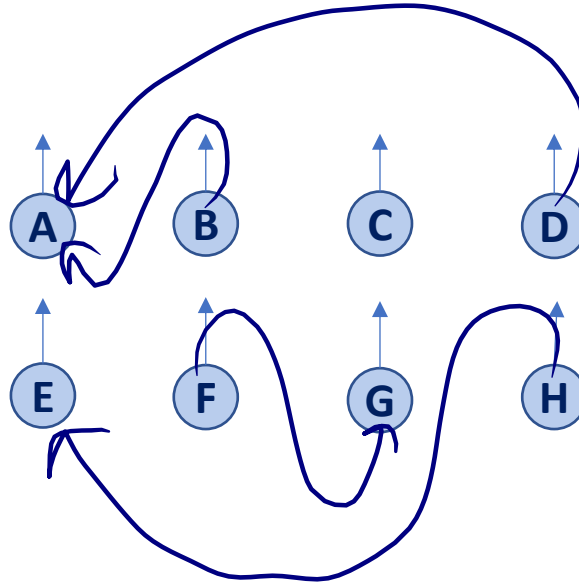
Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

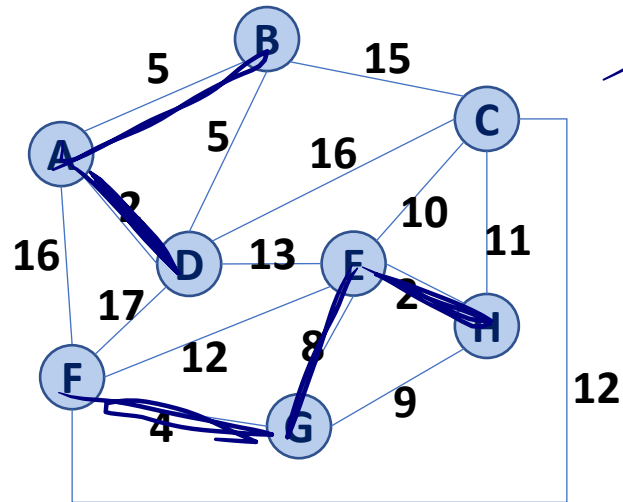
skip

DS



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

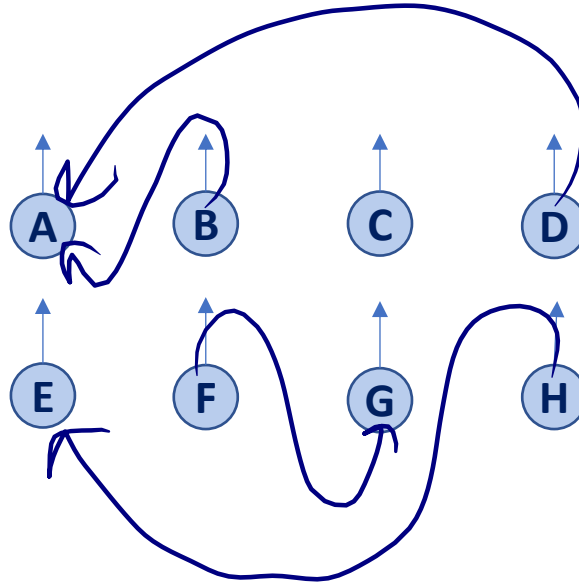
Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

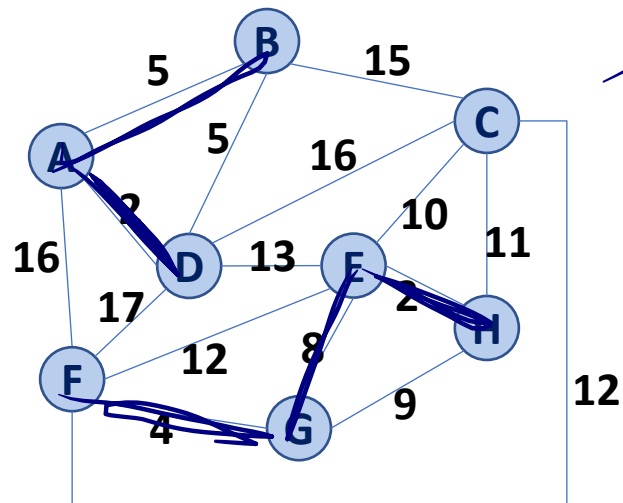
skip

DS



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

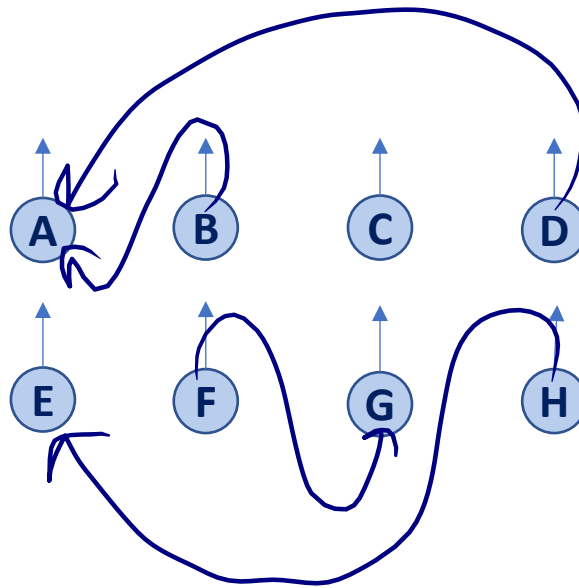
Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

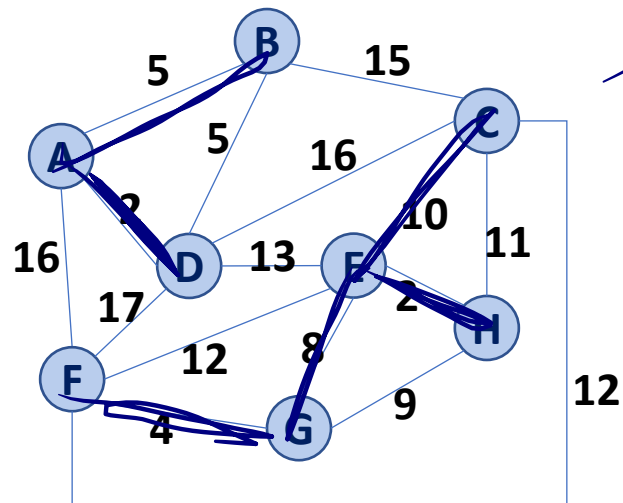
skip
skip

DS



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

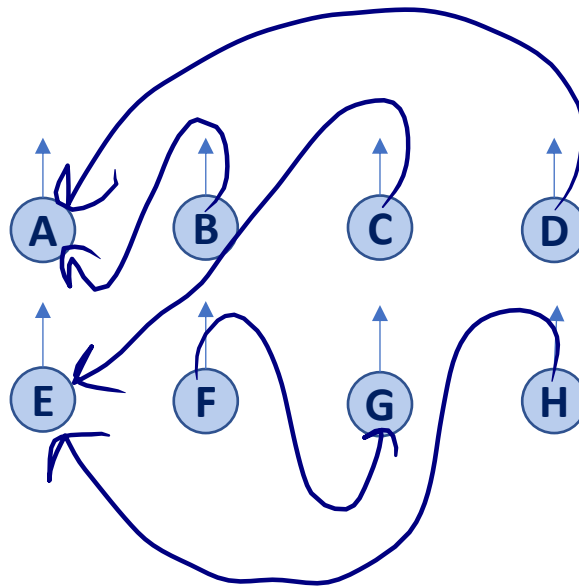
Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

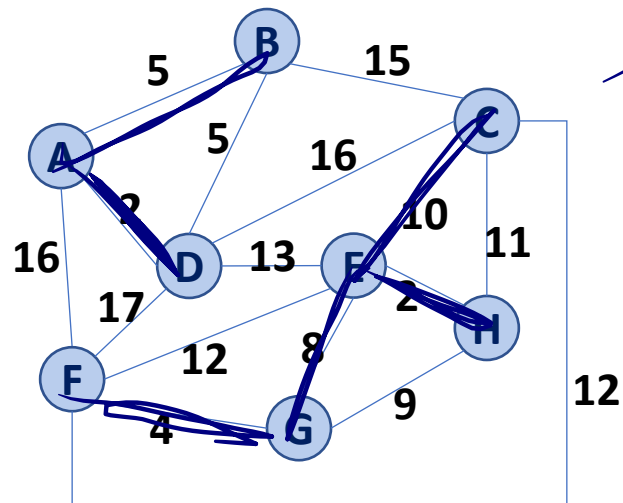
skip
skip

DS



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

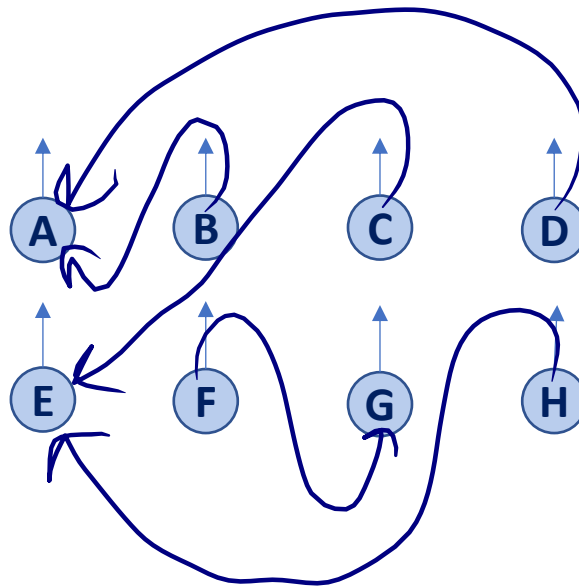
Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

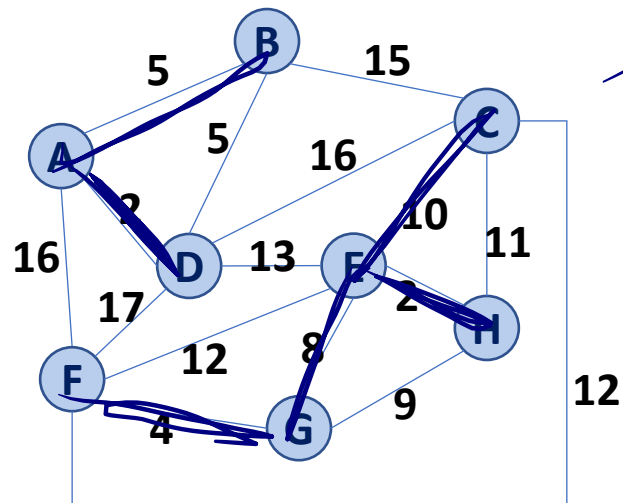
skip
skip
skip

DS



(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm

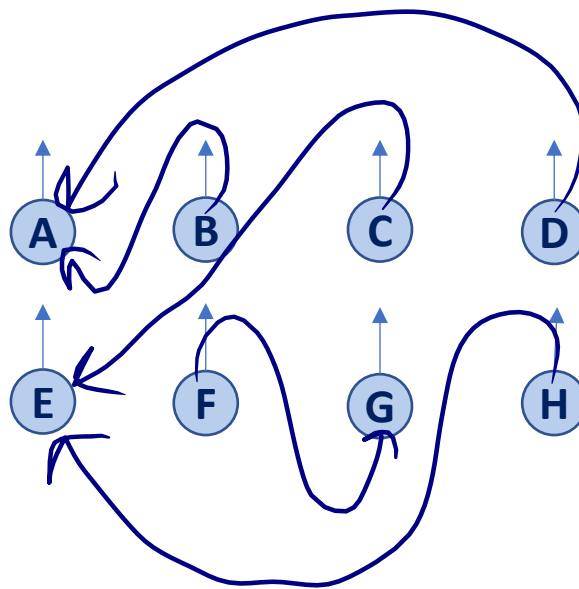


Stop when
of edges in G'
 $= n-1$

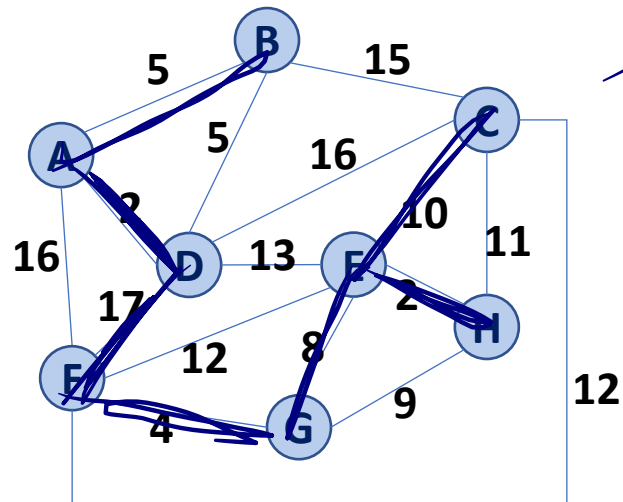
skip
skip
skip
skip
skip

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

DS

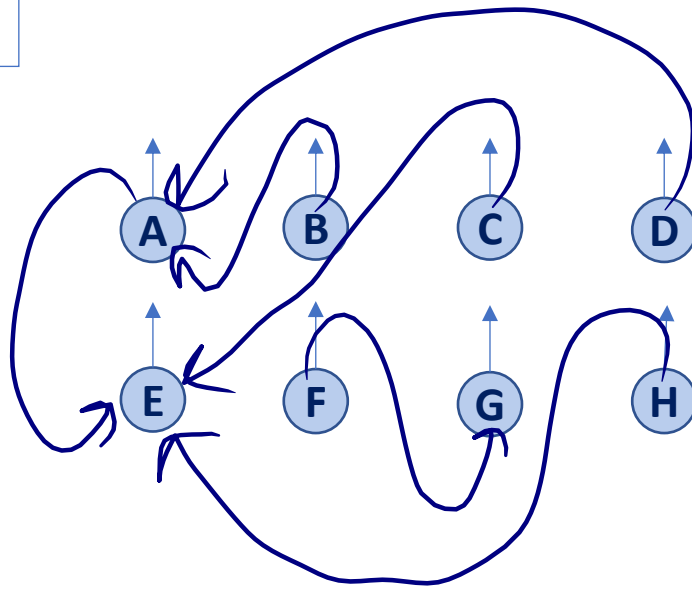


Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

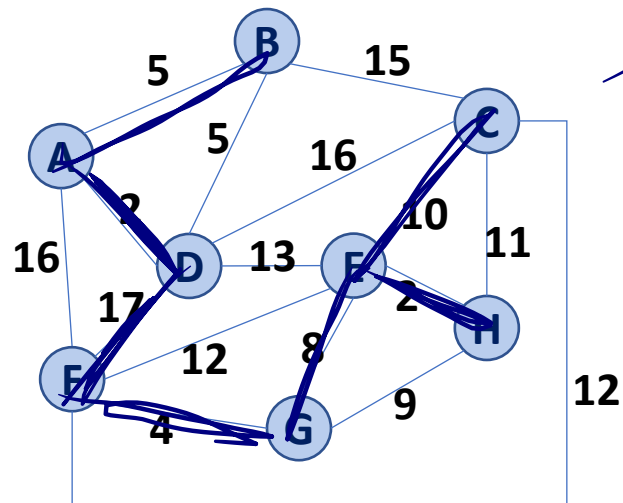
DS



skip
skip
skip
skip
skip

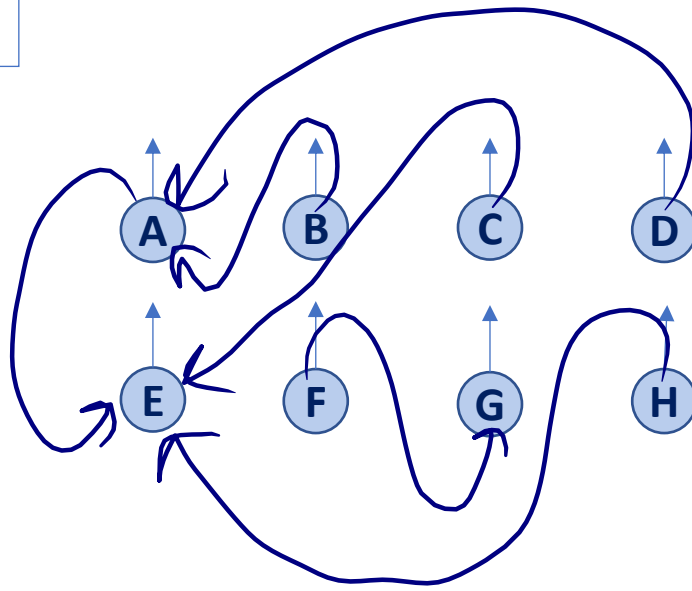
(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Kruskal's Algorithm



Stop when
of edges in G'
 $= n-1$

DS



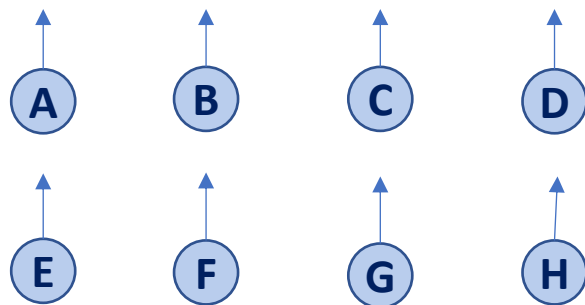
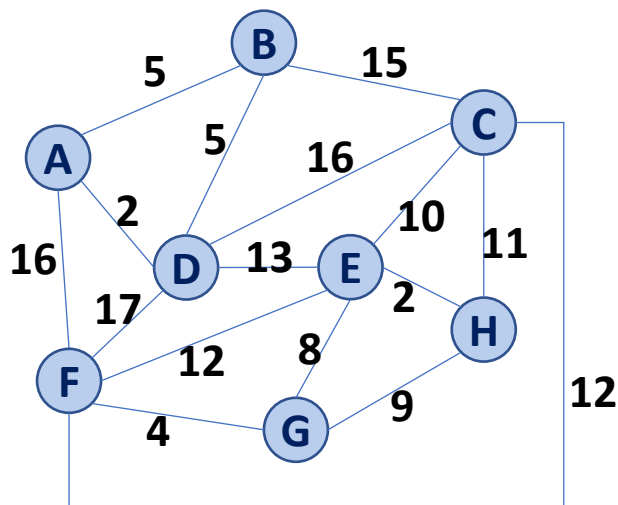
skip
skip
skip
skip
skip

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

Done!

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

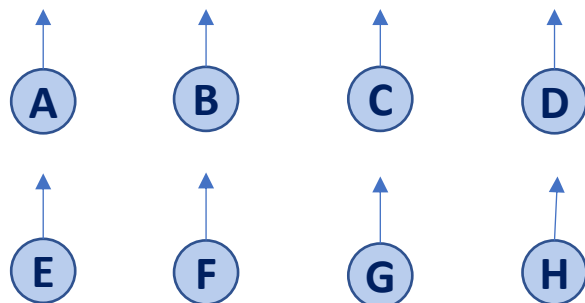
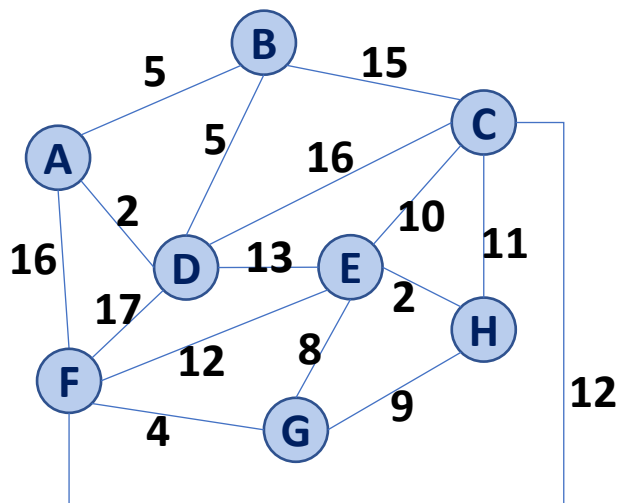


```

1  KruskalMST(G):
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T
    
```

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

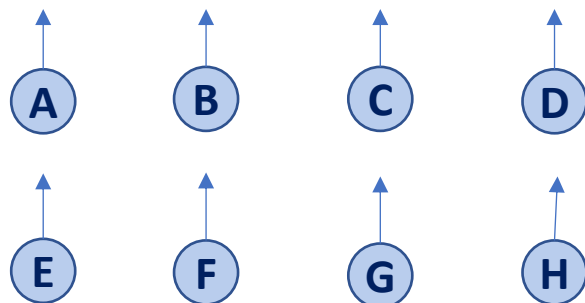
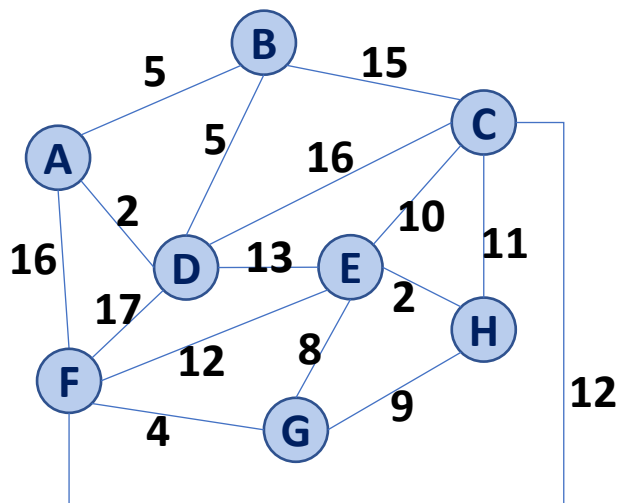


```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T
    
```


Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)

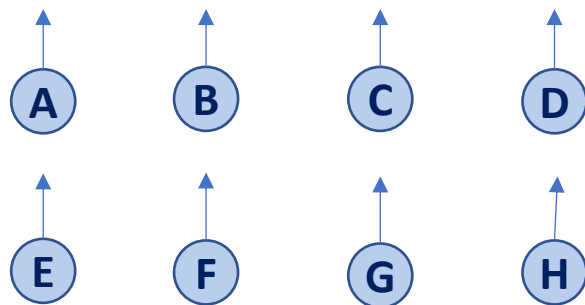
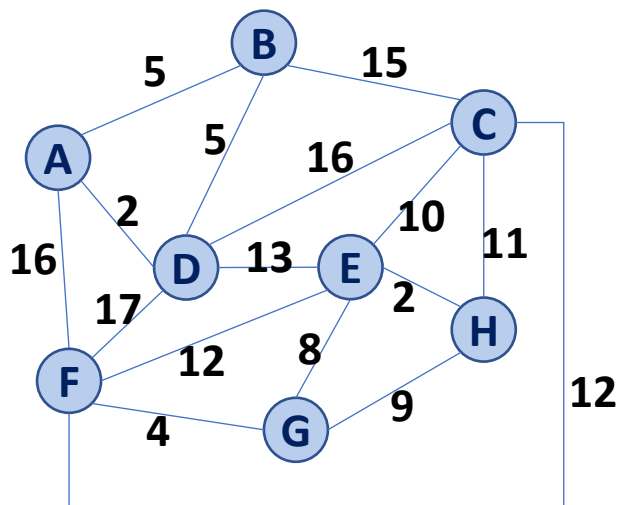


```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T
    
```

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)



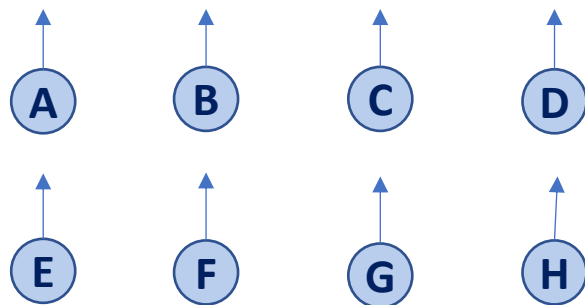
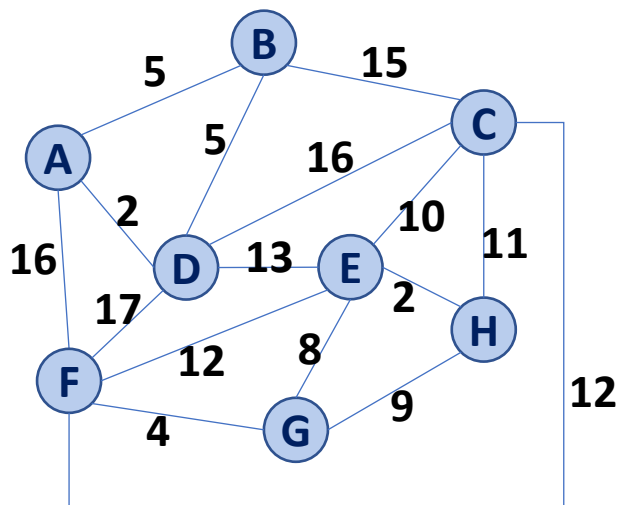
```

1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q      // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                    forest.find(v) )
18
19  return T

```

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)



```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T

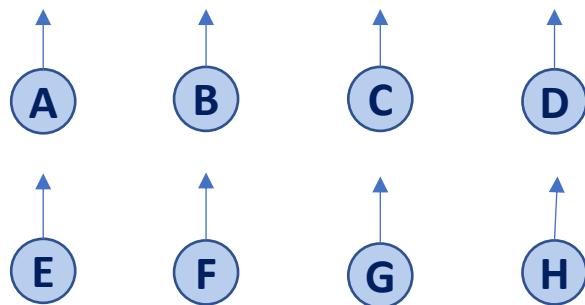
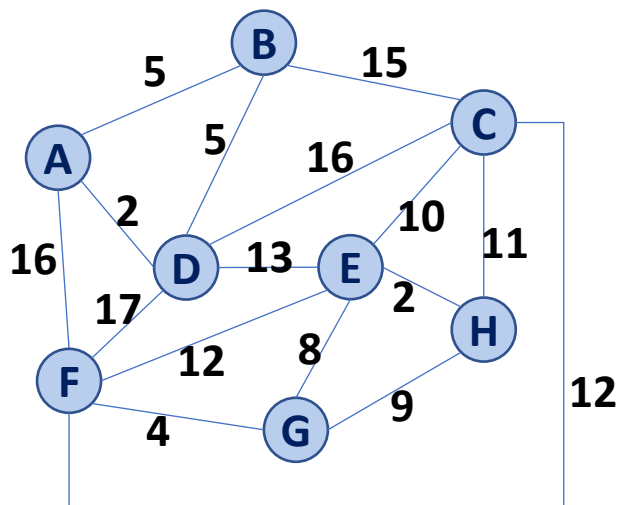
```

Handwritten annotations in blue ink:

- Next to line 3: $O(n)$
- Next to line 10: $O(n)$
- Next to line 12: $O(m)$
- A large blue bracket on the right side of lines 3 through 17, spanning the initialization and main loop.
- A blue arrow points from the $O(m)$ annotation to the condition $|T.edges()| < n-1$ in line 12.

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)



```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T

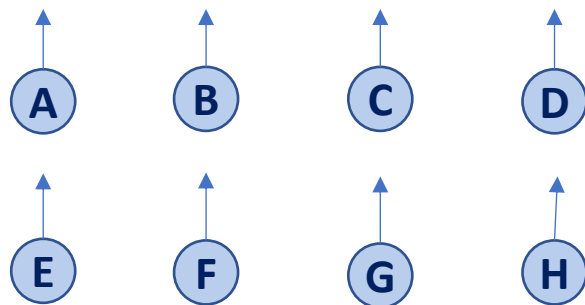
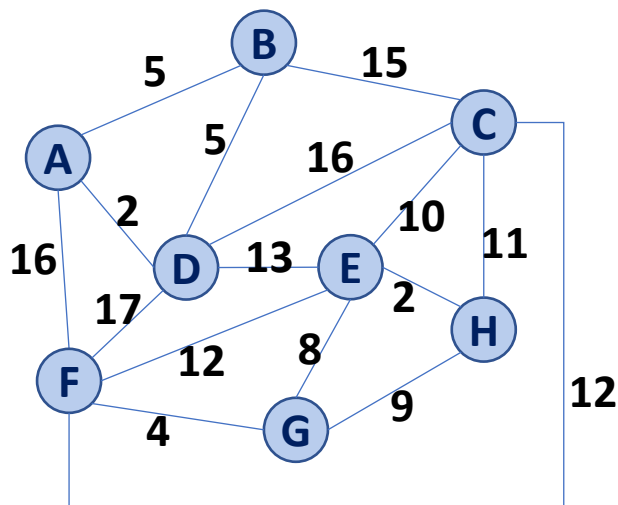
```

Handwritten annotations in blue ink:

- Next to line 3: $O(n)$
- Next to line 10: $O(n)$
- Next to line 12: $O(m)$
- A large closing curly brace $\}$ spans from line 3 to line 8.
- A large closing curly brace $\}$ spans from line 6 to line 17.
- A large closing curly brace $\}$ spans from line 12 to line 17.

Kruskal's Algorithm

(A, D)
(E, H)
(F, G)
(A, B)
(B, D)
(G, E)
(G, H)
(E, C)
(C, H)
(E, F)
(F, C)
(D, E)
(B, C)
(C, D)
(A, F)
(D, F)



```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union(forest.find(u),
17                         forest.find(v))
18
19     return T
    
```

Handwritten annotations in blue ink:

- Next to line 3: $O(n)$
- Next to line 12: $O(n)$ (with an arrow pointing to the condition $|T.edges()| < n-1$)
- Next to line 13: $O(m)$ (with an arrow pointing to $Q.removeMin()$)
- Next to line 16: $O(1)$ (with an arrow pointing to the `forest.union` call)

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8		
Each removeMin :13		

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u) ,
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(M)$	
Each removeMin :13		

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u) ,
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13		

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u) ,
17                  forest.find(v) )
18
19  return T
```


Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13		

$$m = O(n^2) \leq C * n^2$$

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13		

$$m = O(n^2) \leq C * n^2$$
$$\lg(m) \leq \lg(C * n^2)$$

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13		

$$m = O(n^2) \leq C * n^2$$
$$\lg(m) \leq \lg(C * n^2)$$
$$\lg(m) \leq \lg C + 2 \lg(n)$$
$$\lg(m) = O(\lg(n))$$

```
1 KruskalMST(G) :  
2     DisjointSets forest  
3     foreach (Vertex v : G) :  
4         forest.makeSet(v)  
5  
6     PriorityQueue Q      // min edge weight  
7     foreach (Edge e : G) :  
8         Q.insert(e)  
9  
10    Graph T = (V, {})  
11  
12    while |T.edges()| < n-1:  
13        Edge (u, v) = Q.removeMin()  
14        if forest.find(u) != forest.find(v) :  
15            T.addEdge(u, v)  
16            forest.union( forest.find(u),  
17                          forest.find(v) )  
18  
19    return T
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13	$O(\lg(n))$	

$$\begin{aligned}
 m &= O(n^2) \leq C * n^2 \\
 \lg(m) &\leq \lg(C * n^2) \\
 \lg(m) &\leq \lg C + 2 \lg(n) \\
 \lg(m) &= O(\lg(n))
 \end{aligned}$$

```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G) :
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G) :
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v) :
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T
    
```

Kruskal's Algorithm

Priority Queue:	Heap	Sorted Array
Building :6-8	$O(m)$	$O(m \lg(n))$
Each removeMin :13	$O(\lg(n))$	$O(1)$

$$m = o(n^2) \leq C * n^2$$
$$\lg(m) \leq \lg(C * n^2)$$
$$\lg(m) \leq \lg C + 2 \lg(n)$$
$$\lg(m) = o(\lg(n))$$

```
1 KruskalMST(G) :  
2     DisjointSets forest  
3     foreach (Vertex v : G) :  
4         forest.makeSet(v)  
5  
6     PriorityQueue Q      // min edge weight  
7     foreach (Edge e : G) :  
8         Q.insert(e)  
9  
10    Graph T = (V, {})  
11  
12    while |T.edges()| < n-1:  
13        Edge (u, v) = Q.removeMin()  
14        if forest.find(u) != forest.find(v) :  
15            T.addEdge(u, v)  
16            forest.union( forest.find(u),  
17                          forest.find(v) )  
18  
19    return T
```

Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	
Sorted Array	

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u) ,
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	
Sorted Array	

```
1 KruskalMST(G) :
2   DisjointSets forest
3   foreach (Vertex v : G) :
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G) :
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v) :
15      T.addEdge(u, v)
16      forest.union( forest.find(u) ,
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	
Sorted Array	

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

$O(n)$

$O(n)$

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n^2 \log n)$
Sorted Array	

```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T

```

Handwritten annotations in blue ink:

- A large closing bracket $)$ on the right side of lines 2-4, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 6-8, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 10-11, with $O(n)$ written next to it.

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n + m)$
Sorted Array	

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Handwritten annotations in blue ink:

- A large closing bracket $)$ on the right side of lines 2-4, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 6-8, with $O(m)$ written next to it.
- A bracket $\leftarrow$ on the right side of line 12, with $O(n)$ written next to it.
- A bracket $\leftarrow$ on the right side of line 13, with $O(m)$ written next to it.

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n+m) + O(m \lg n)$
Sorted Array	

```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T
    
```

Handwritten annotations in blue ink:

- A large closing bracket $)$ on the right side of lines 2-4, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 6-8, with $O(m)$ written next to it.
- A large closing bracket $)$ on the right side of lines 10-11, with $O(n)$ written next to it.
- A large closing bracket $)$ on the right side of lines 12-17, with $O(m)$ written next to it.
- Arrows pointing from the $O(n)$ and $O(m)$ annotations to the corresponding lines in the code.

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n+m) + O(m \lg n)$
Sorted Array	$O(n + m \lg n)$

```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T

```

Handwritten annotations in the code block:

- A large closing bracket $)$ spans lines 2 through 5, with $O(n)$ written next to it.
- A large closing bracket $)$ spans lines 6 through 8, with $O(n)$ written next to it.
- A large closing bracket $)$ spans lines 10 through 11, with $O(n)$ written next to it.
- A large closing bracket $)$ spans lines 12 through 17, with $O(m)$ written next to it.
- Arrows point from the $O(n)$ and $O(m)$ annotations to the corresponding lines in the code.

Kruskal's Algorithm

Setup + Iter

Priority Queue:	Total Running Time
Heap	$O(n+m) + O(m \lg n)$
Sorted Array	$O(n + m \lg n) + O(m)$

```

1  KruskalMST(G) :
2      DisjointSets forest
3      foreach (Vertex v : G):
4          forest.makeSet(v)
5
6      PriorityQueue Q      // min edge weight
7      foreach (Edge e : G):
8          Q.insert(e)
9
10     Graph T = (V, {})
11
12     while |T.edges()| < n-1:
13         Edge (u, v) = Q.removeMin()
14         if forest.find(u) != forest.find(v):
15             T.addEdge(u, v)
16             forest.union( forest.find(u),
17                           forest.find(v) )
18
19     return T

```

Handwritten annotations in the code block:

- A large closing bracket $)$ spans lines 2 through 4, with $O(n)$ written next to it.
- A large closing bracket $)$ spans lines 6 through 8, with $O(m)$ written next to it.
- A large closing bracket $)$ spans lines 10 through 11, with $O(n)$ written next to it.
- A large closing bracket $)$ spans lines 12 through 17, with $O(m)$ written next to it.
- Arrows point from the $O(n)$ and $O(m)$ annotations to the corresponding parts of the code.

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	$O(n + m) + O(m \lg(n))$
Sorted Array	$O(n + m \lg(n)) + O(m)$

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	$O(n + m) + O(m \lg(n))$
Sorted Array	$O(n + m \lg(n)) + O(m)$

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```


Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	$O(n + m) + O(m \lg(n))$
Sorted Array	$O(n + m \lg(n)) + O(m)$

Heap - allows changing edge weights in $O(\lg(n))$

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Priority Queue:	Total Running Time
Heap	$O(n + m) + O(m \lg(n))$
Sorted Array	$O(n + m \lg(n)) + O(m)$

Heap - allows changing
edge weights in $O(\lg(n))$
Sorted array -
General

```
1 KruskalMST(G):
2   DisjointSets forest
3   foreach (Vertex v : G):
4     forest.makeSet(v)
5
6   PriorityQueue Q    // min edge weight
7   foreach (Edge e : G):
8     Q.insert(e)
9
10  Graph T = (V, {})
11
12  while |T.edges()| < n-1:
13    Edge (u, v) = Q.removeMin()
14    if forest.find(u) != forest.find(v):
15      T.addEdge(u, v)
16      forest.union( forest.find(u),
17                  forest.find(v) )
18
19  return T
```

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

- Heap:
- Sorted Array:

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

- Heap: *Flexible (change edge weights)*
- Sorted Array:

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

- Heap: *Flexible (change edge weights)*
- Sorted Array: *General*

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

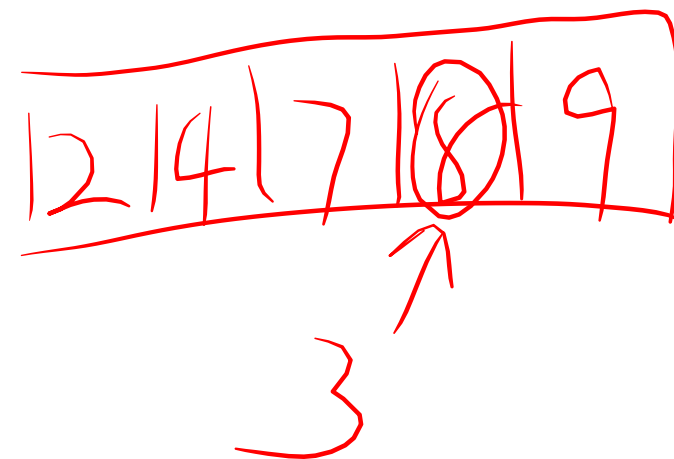
- Heap: *Flexible (change edge weights)*
 - Sorted Array: *General*
- 12 14 17 18 19

Kruskal's Algorithm

Which Priority Queue Implementation is better for running Kruskal's Algorithm?

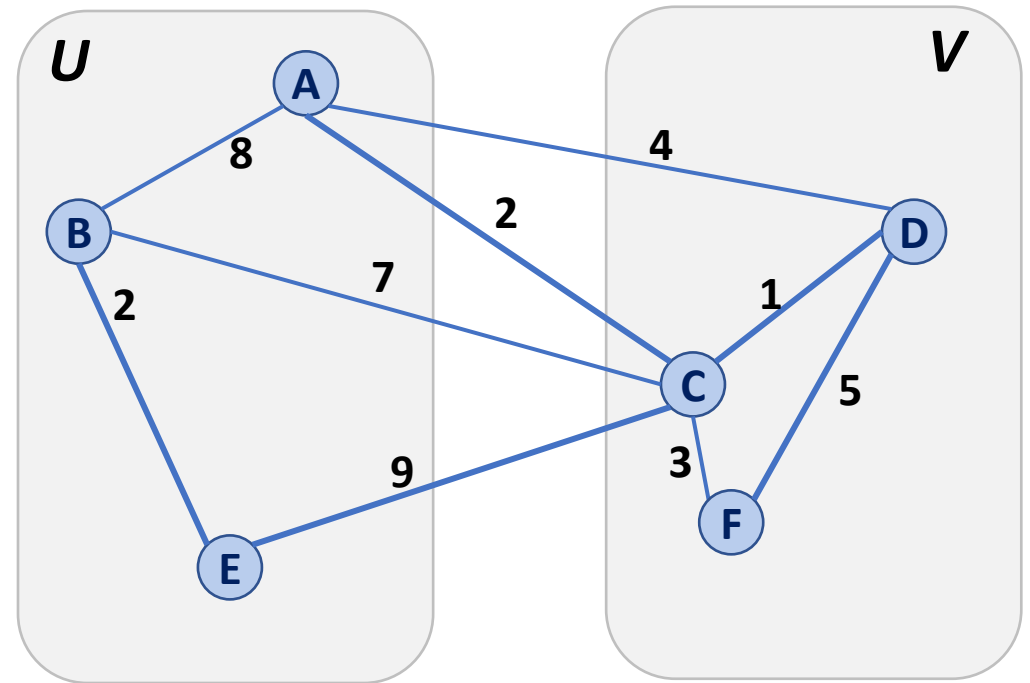
• Heap: *Flexible (change edge weights)*

• Sorted Array: *General*



Partition Property

Consider an arbitrary partition of the vertices on **G** into two subsets **U** and **V**.

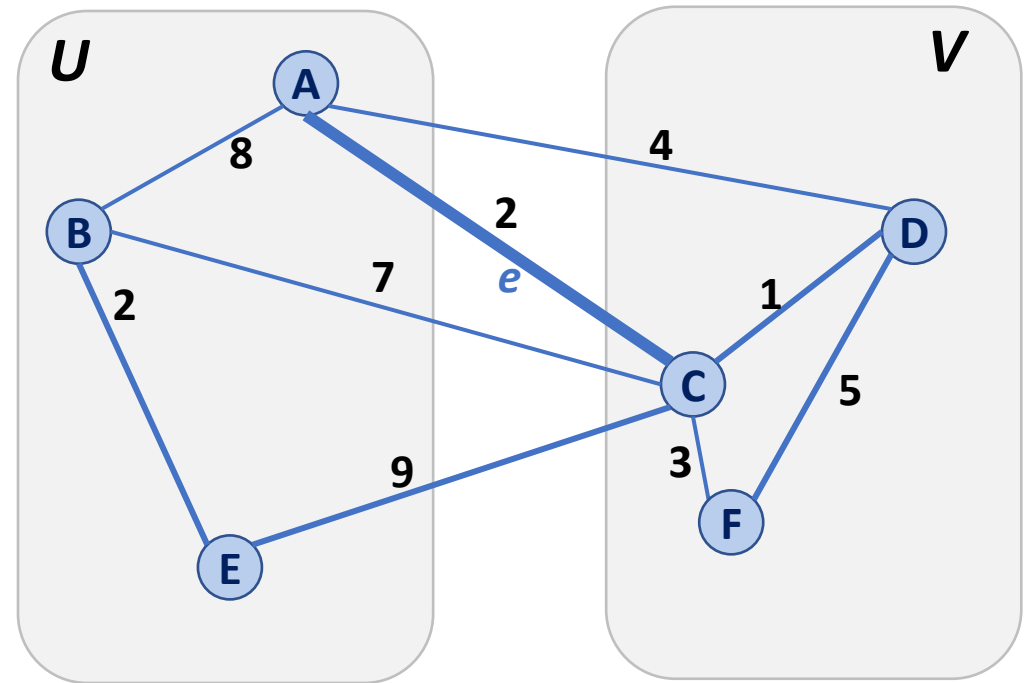


Partition Property

Consider an arbitrary partition of the vertices on G into two subsets U and V .

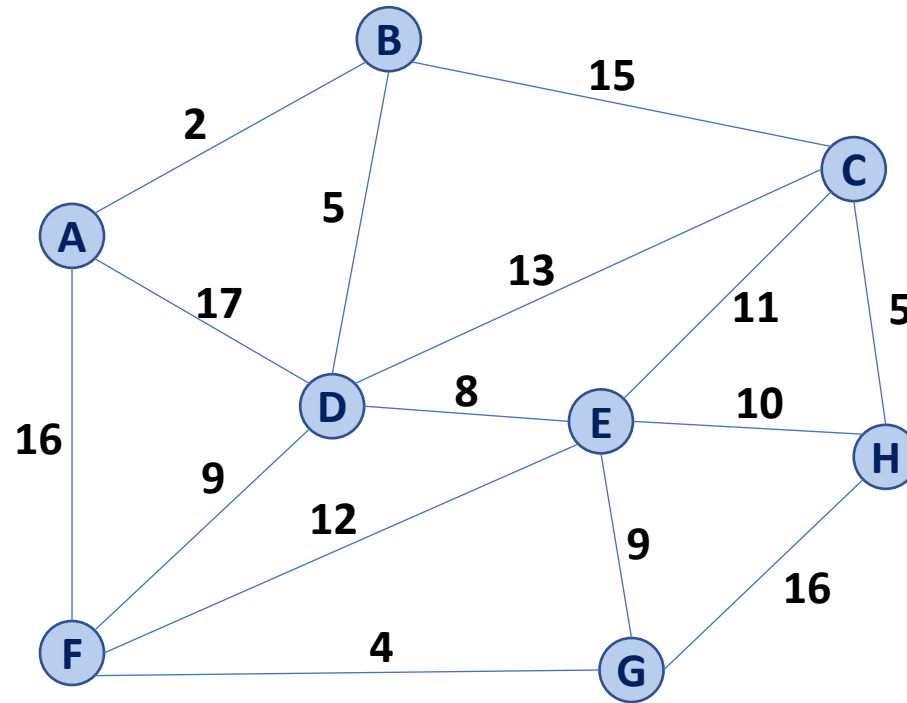
Let e be an edge of minimum weight across the partition.

Then e is part of some minimum spanning tree.



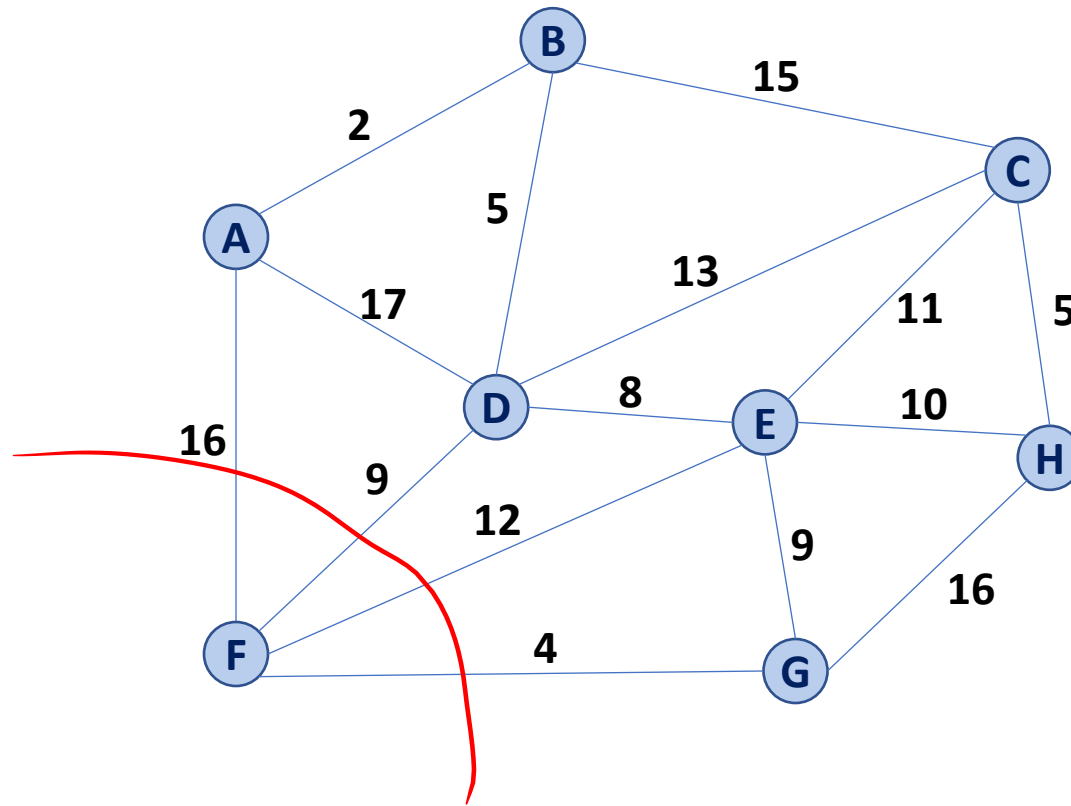
Partition Property

The partition property suggests an algorithm:



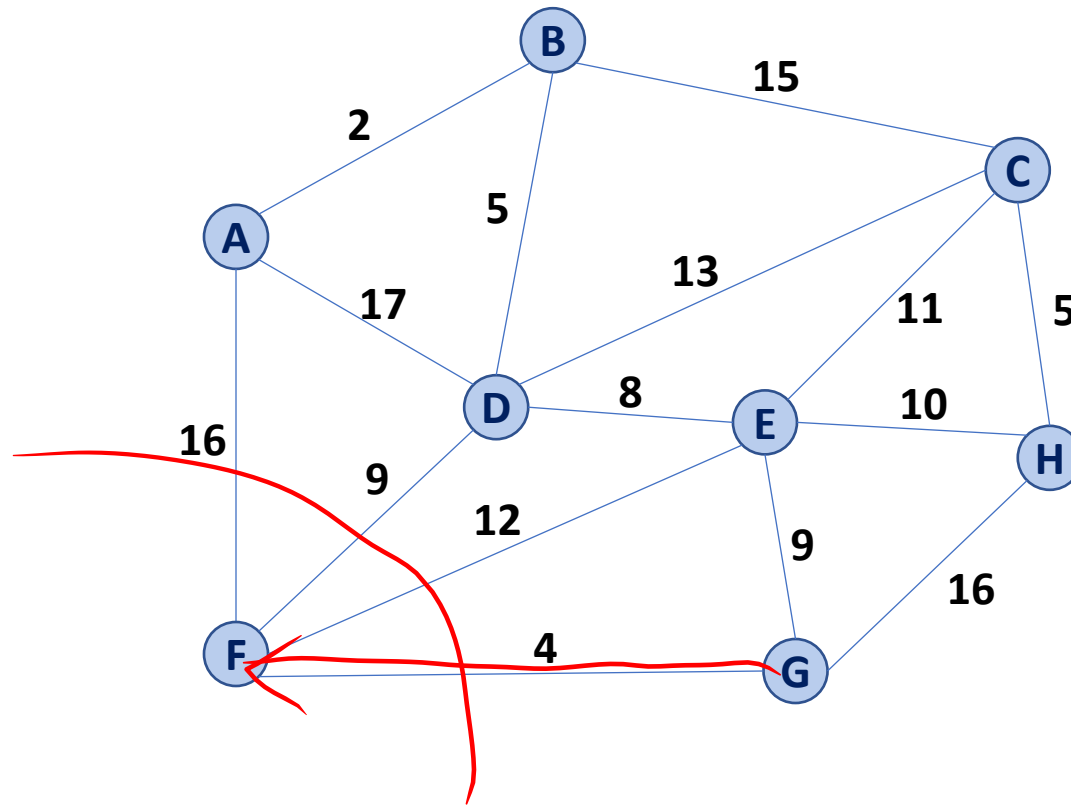
Partition Property

The partition property suggests an algorithm:



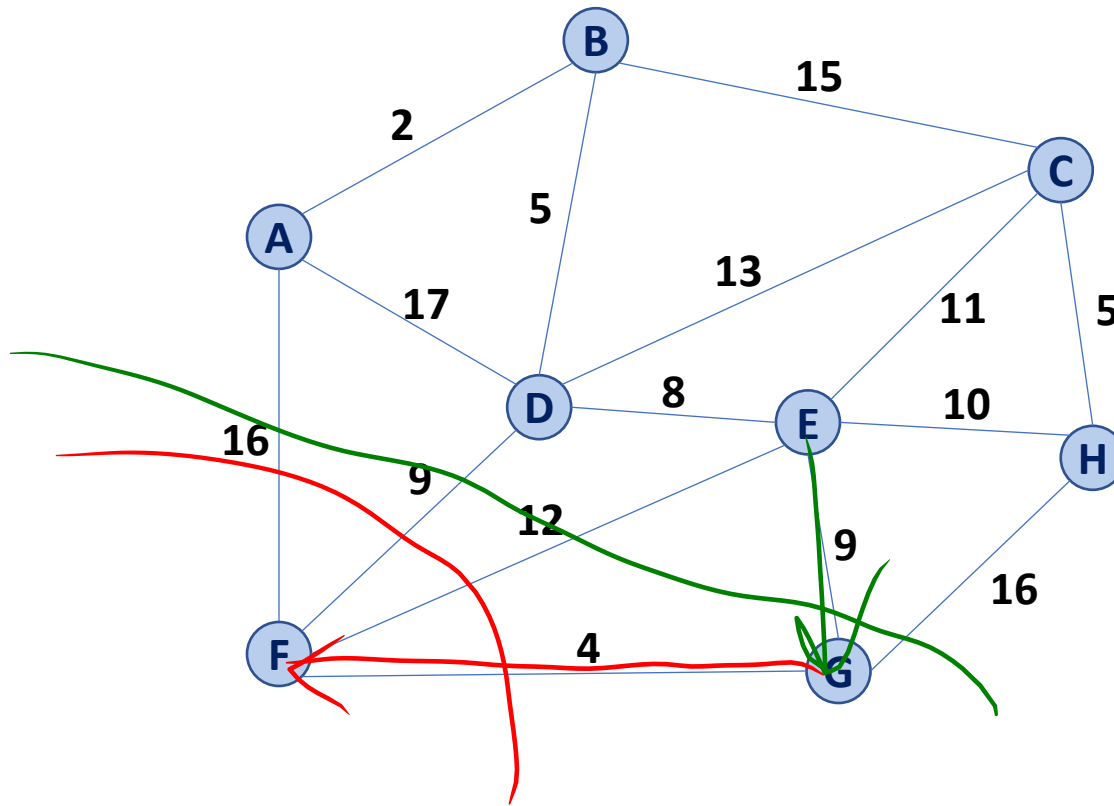
Partition Property

The partition property suggests an algorithm:



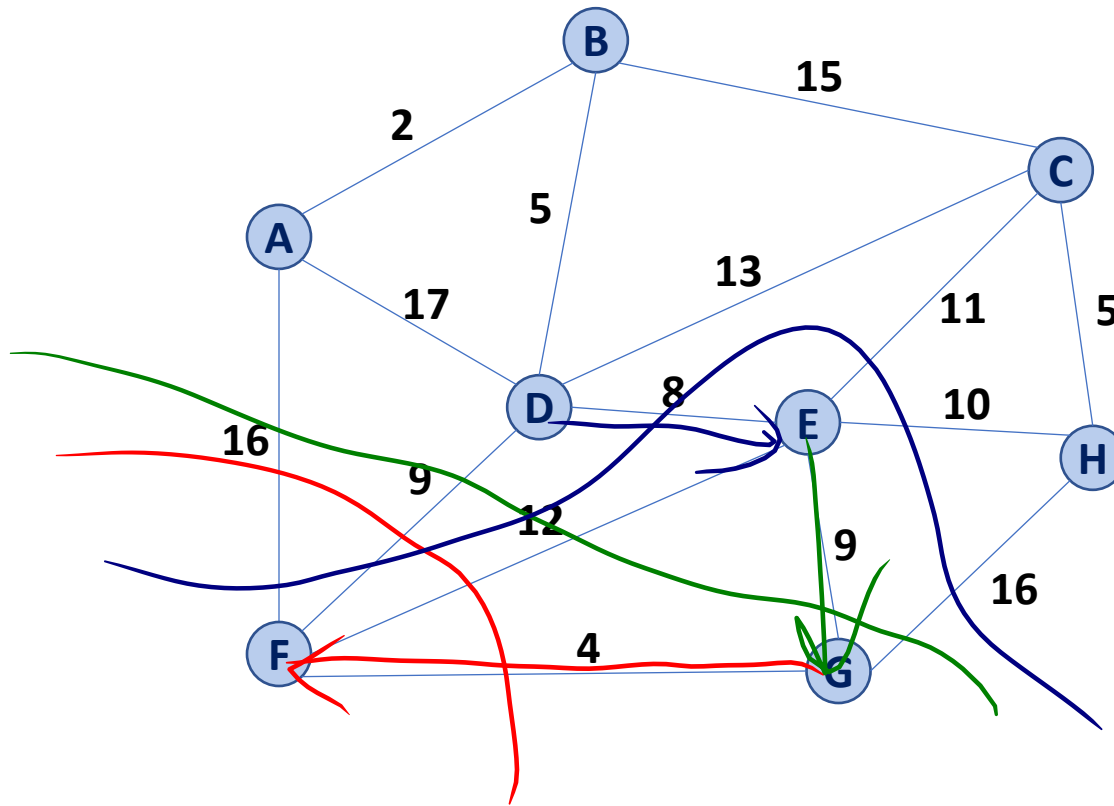
Partition Property

The partition property suggests an algorithm:



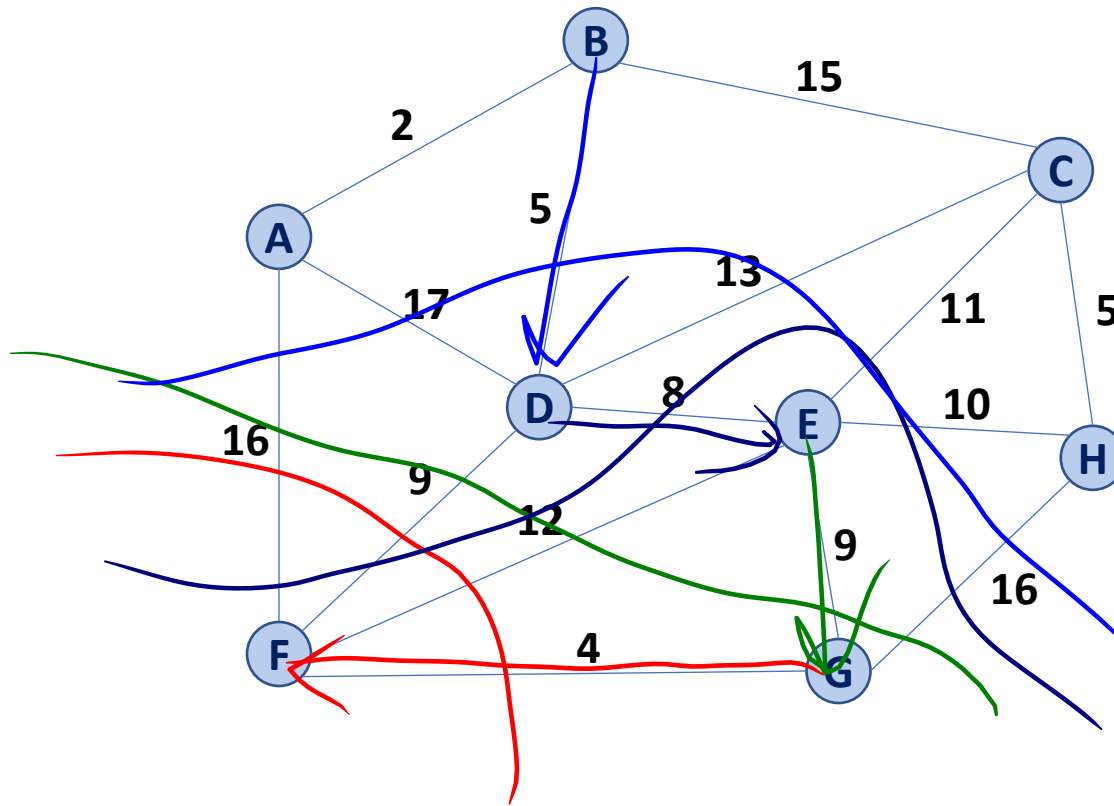
Partition Property

The partition property suggests an algorithm:



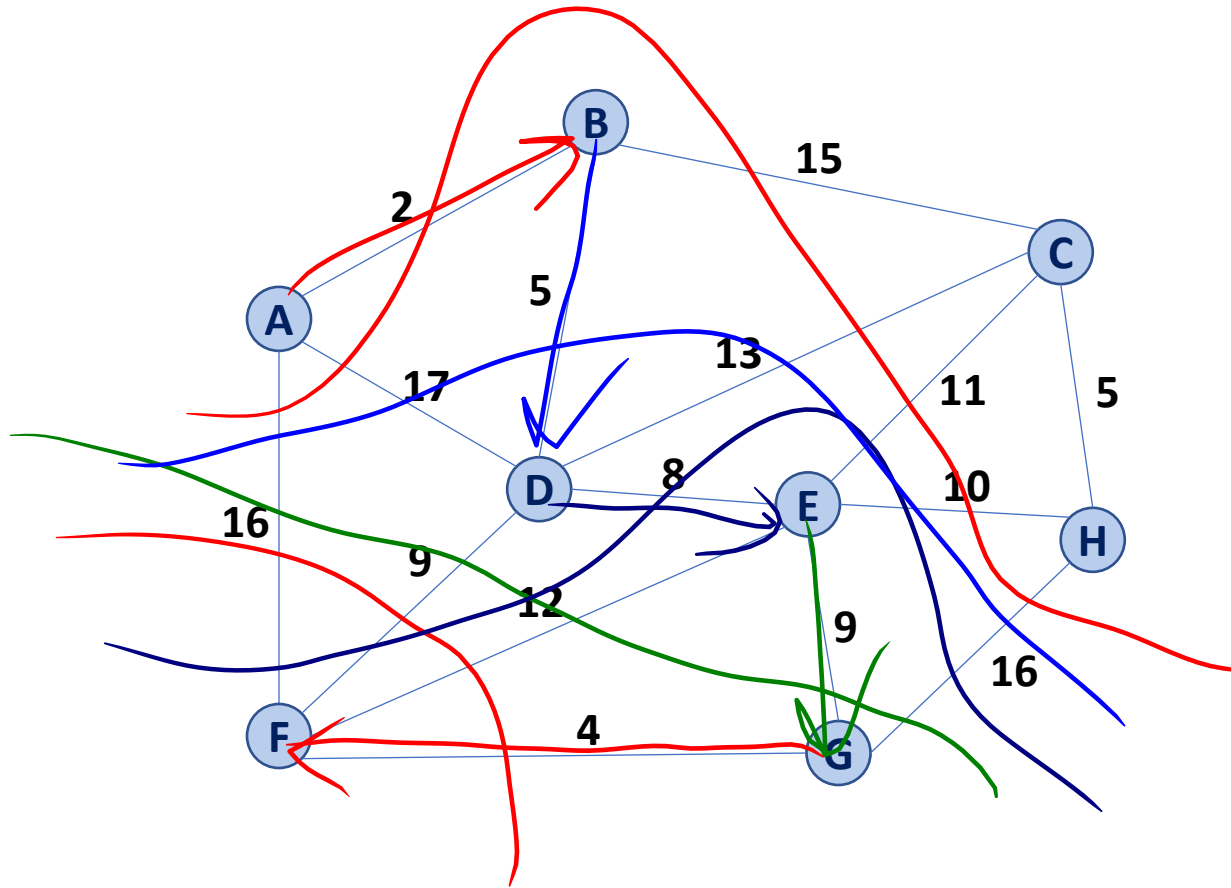
Partition Property

The partition property suggests an algorithm:



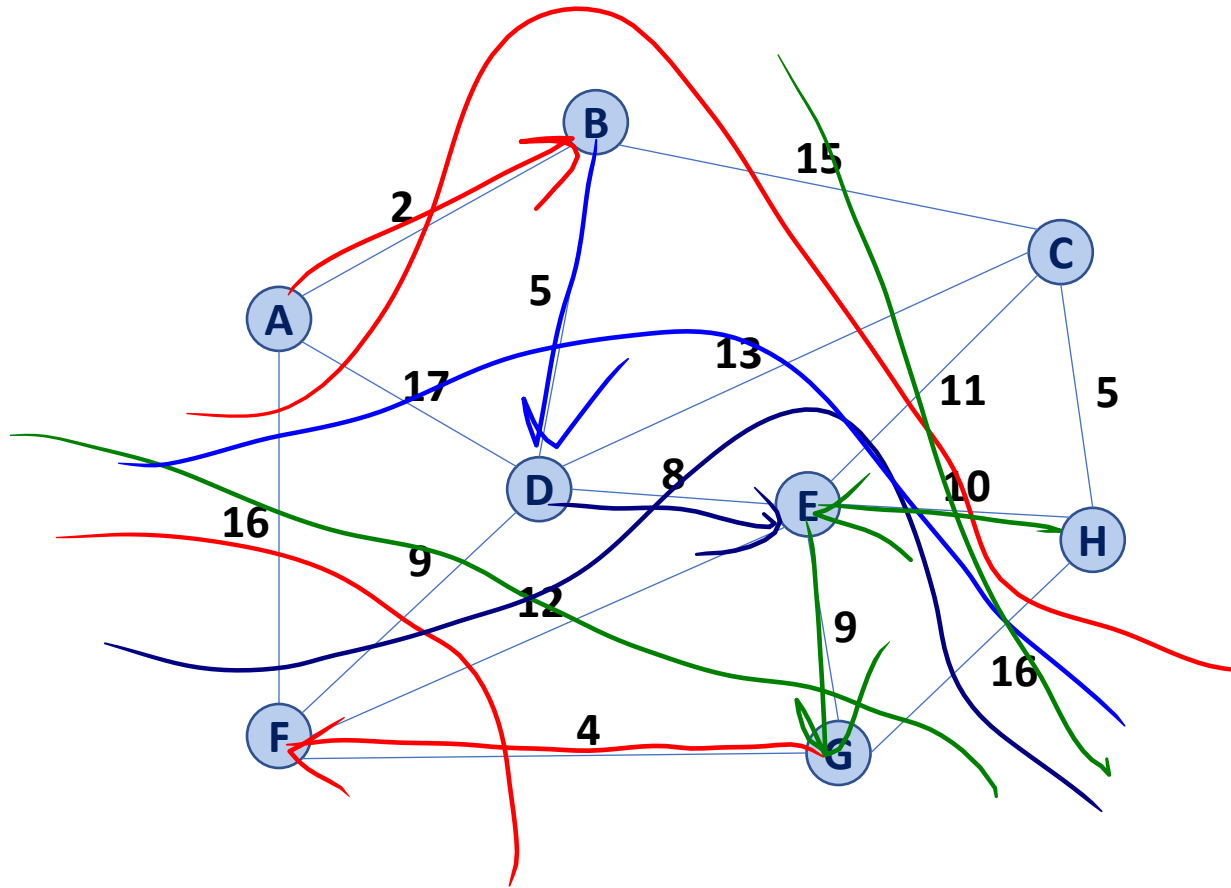
Partition Property

The partition property suggests an algorithm:



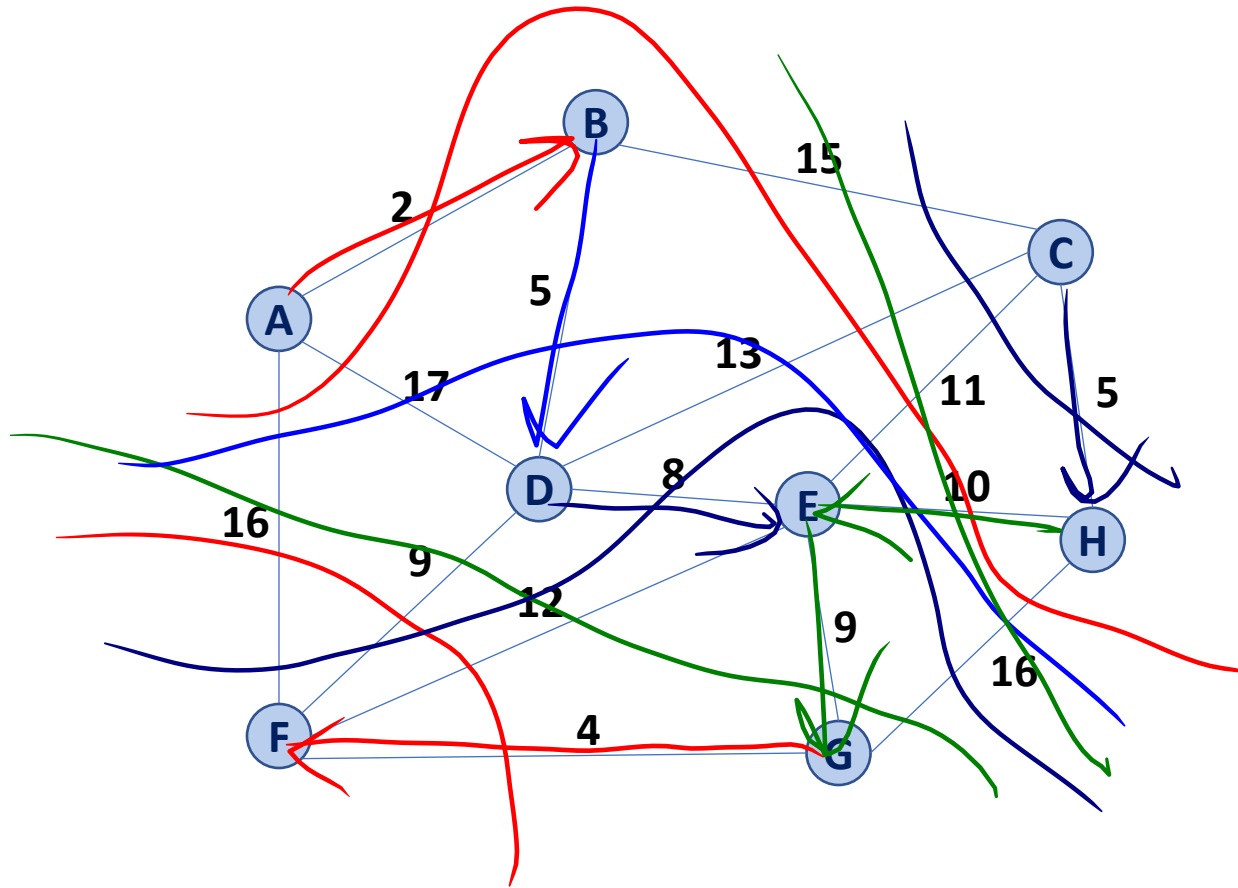
Partition Property

The partition property suggests an algorithm:

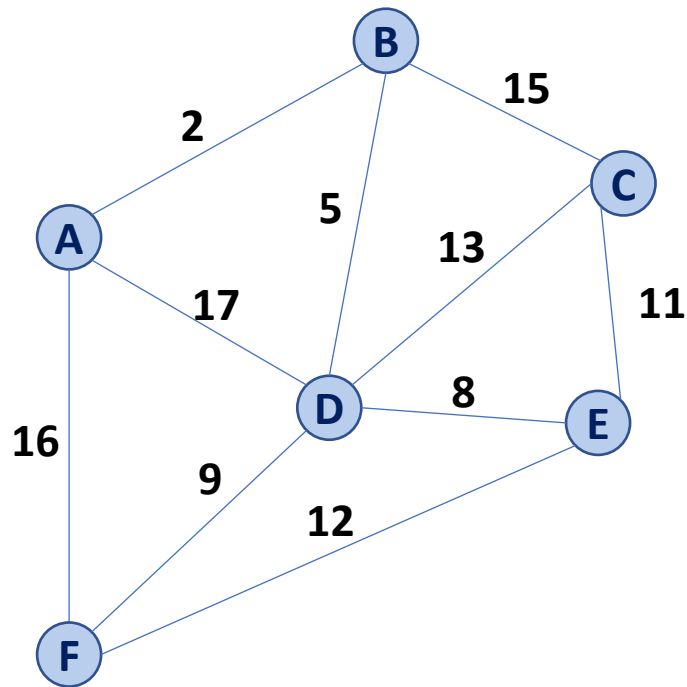


Partition Property

The partition property suggests an algorithm:

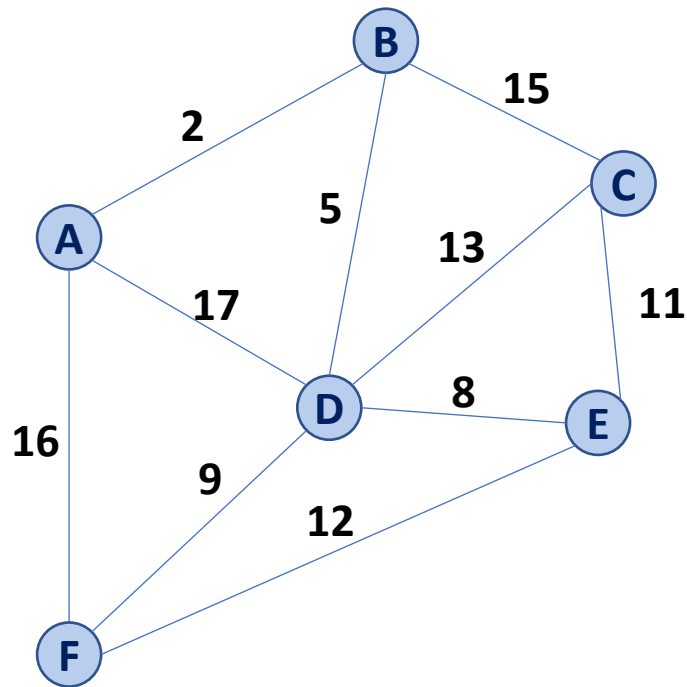


Prim's Algorithm



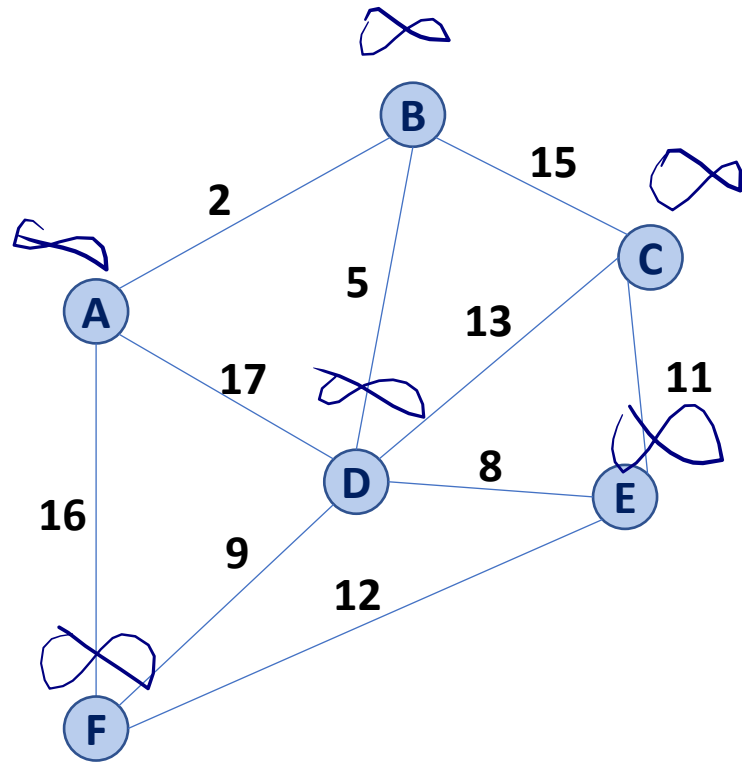
```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Prim's Algorithm



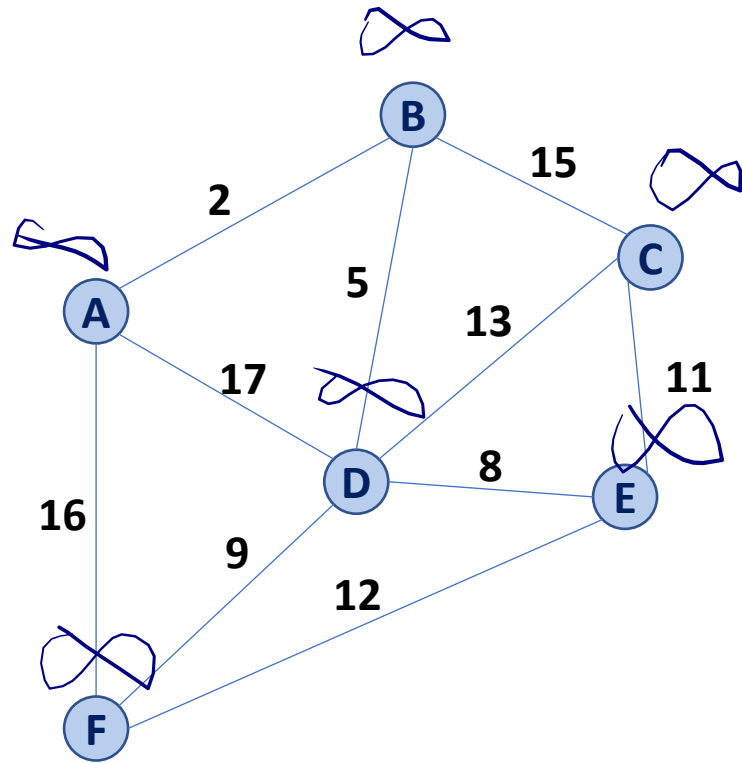
```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Prim's Algorithm



```
1 PrimMST(G, s):
2   Input: G, Graph;
3           s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16       Vertex m = Q.removeMin()
17       T.add(m)
18       foreach (Vertex v : neighbors of m not in T):
19           if cost(v, m) < d[v]:
20               d[v] = cost(v, m)
21               p[v] = m
22
23   return T
```

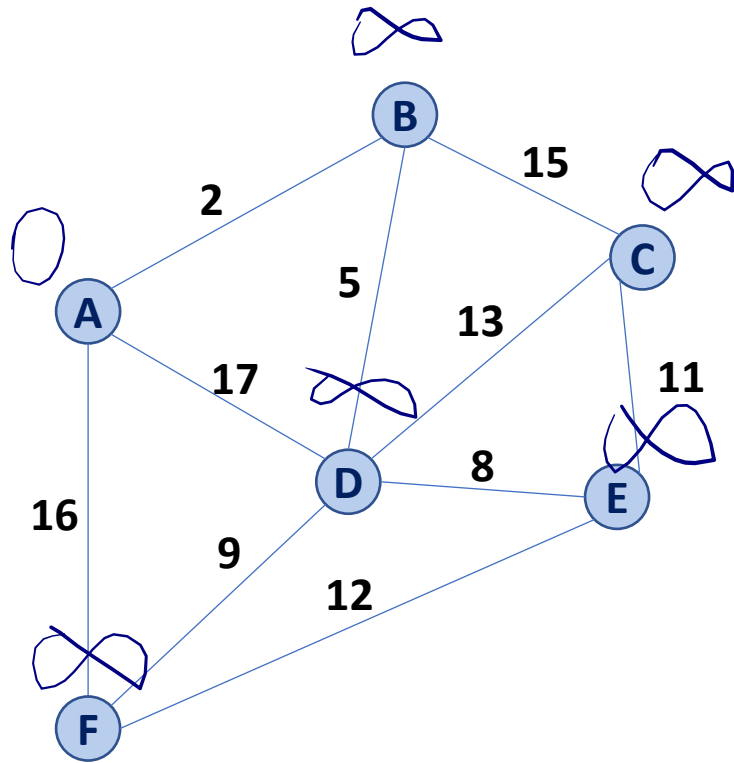
Prim's Algorithm



← vertex A

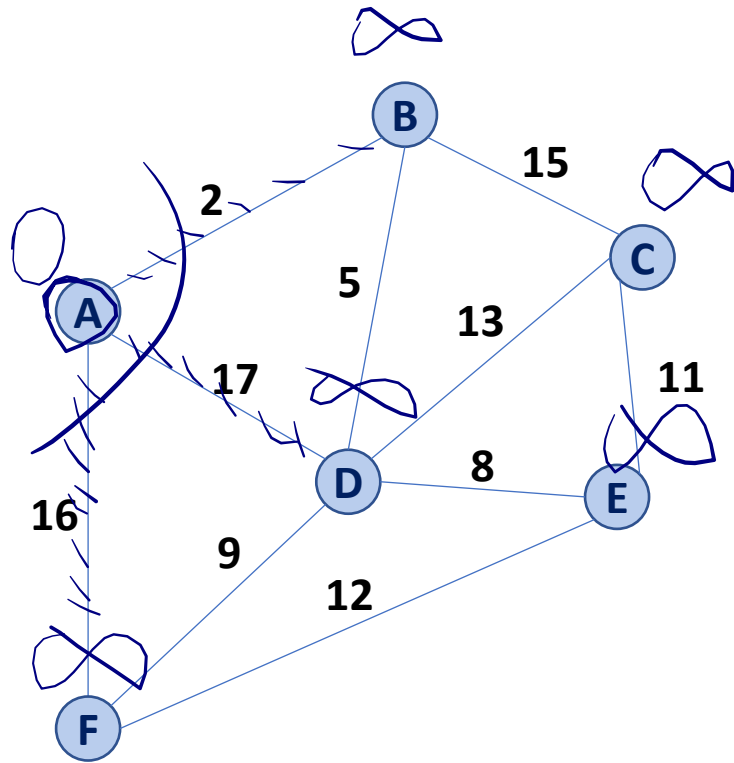
```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16       Vertex m = Q.removeMin()
17       T.add(m)
18       foreach (Vertex v : neighbors of m not in T):
19           if cost(v, m) < d[v]:
20               d[v] = cost(v, m)
21               p[v] = m
22
23   return T
```

Prim's Algorithm



1 PrimMST(G, s): ← vertex A
2 Input: G , Graph;
3 s , vertex in G , starting vertex
4 Output: T , a minimum spanning tree (MST) of G
5
6 foreach (Vertex $v : G$):
7 $d[v] = +\infty$
8 $p[v] = \text{NULL}$
9 $d[s] = 0$
10
11 PriorityQueue Q // min distance, defined by $d[v]$
12 $Q.\text{buildHeap}(G.\text{vertices}())$
13 Graph T // "labeled set"
14
15 repeat n times:
16 Vertex $m = Q.\text{removeMin}()$
17 $T.\text{add}(m)$
18 foreach (Vertex $v : \text{neighbors of } m \text{ not in } T$):
19 if $\text{cost}(v, m) < d[v]$:
20 $d[v] = \text{cost}(v, m)$
21 $p[v] = m$
22
23 return T

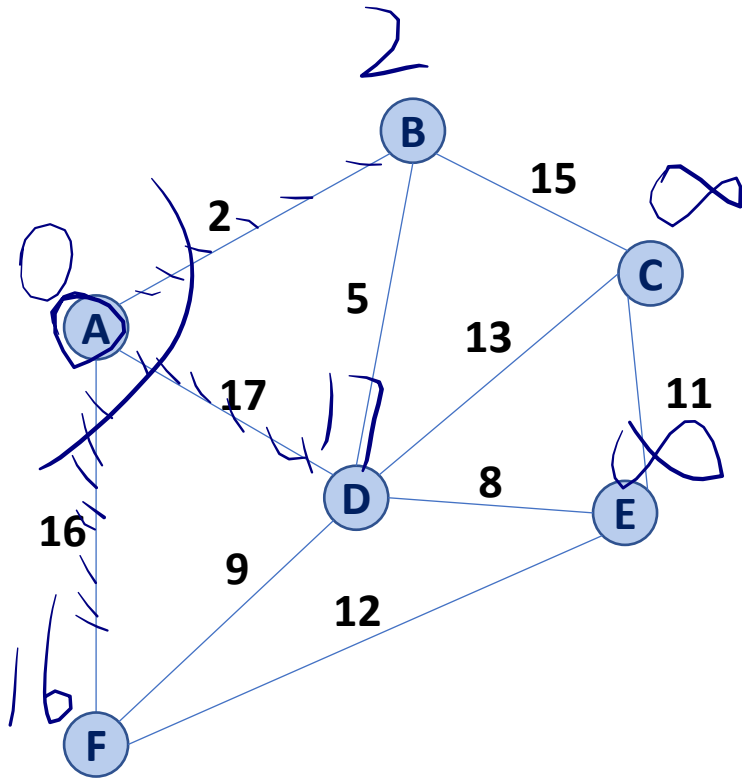
Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9     d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

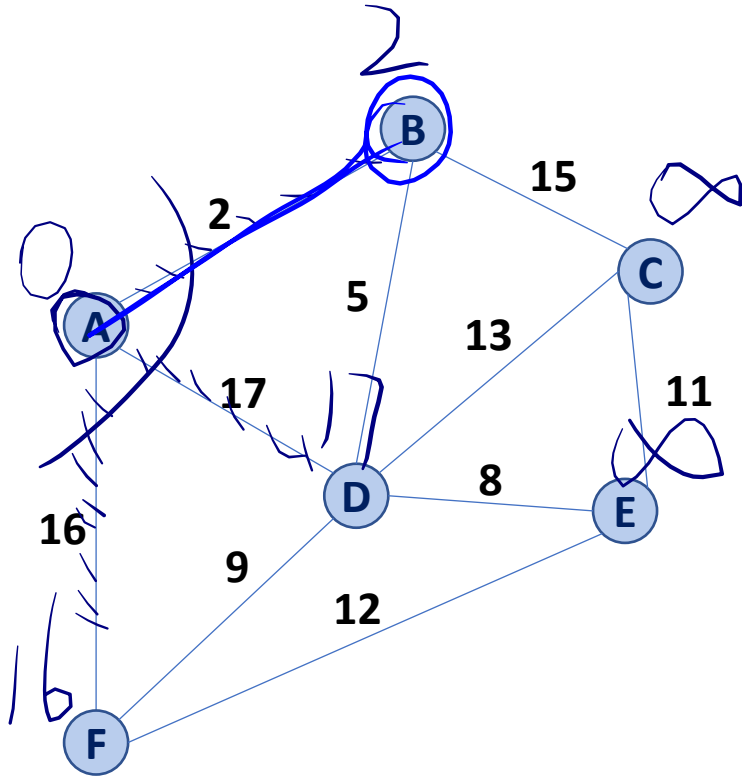

Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9     d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

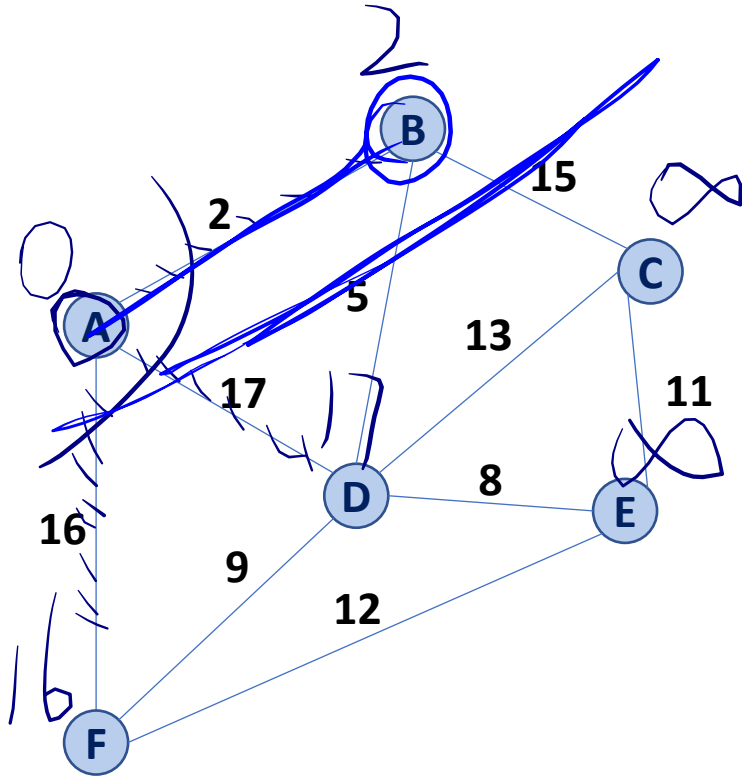
Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Prim's Algorithm



← vertex A

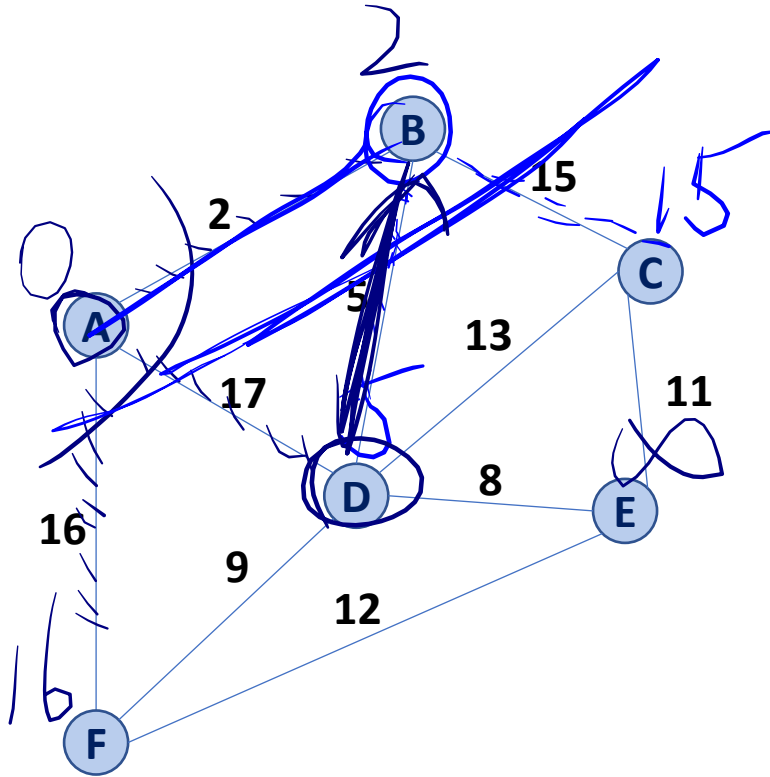
```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

```

1  PrimMST(G, s):
2      Input: G, Graph;
3             s, vertex in G, starting vertex
4      Output: T, a minimum spanning tree (MST) of G
5
6      foreach (Vertex v : G):
7          d[v] = +inf
8          p[v] = NULL
9          d[s] = 0
10
11     PriorityQueue Q    // min distance, defined by d[v]
12     Q.buildHeap(G.vertices())
13     Graph T            // "labeled set"
14
15     repeat n times:
16         Vertex m = Q.removeMin()
17         T.add(m)
18         foreach (Vertex v : neighbors of m not in T):
19             if cost(v, m) < d[v]:
20                 d[v] = cost(v, m)
21                 p[v] = m
22
23     return T

```

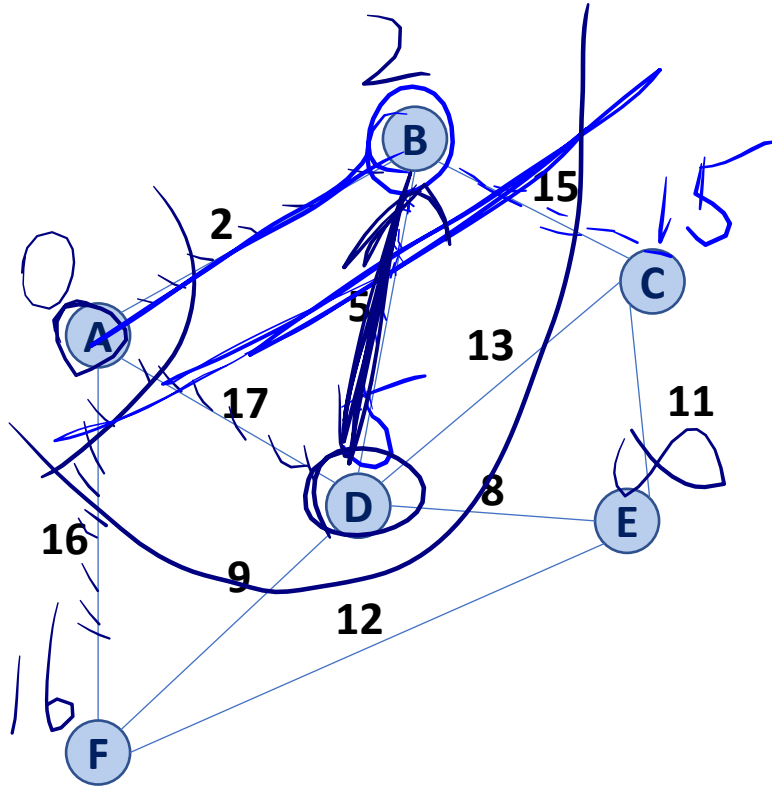
Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

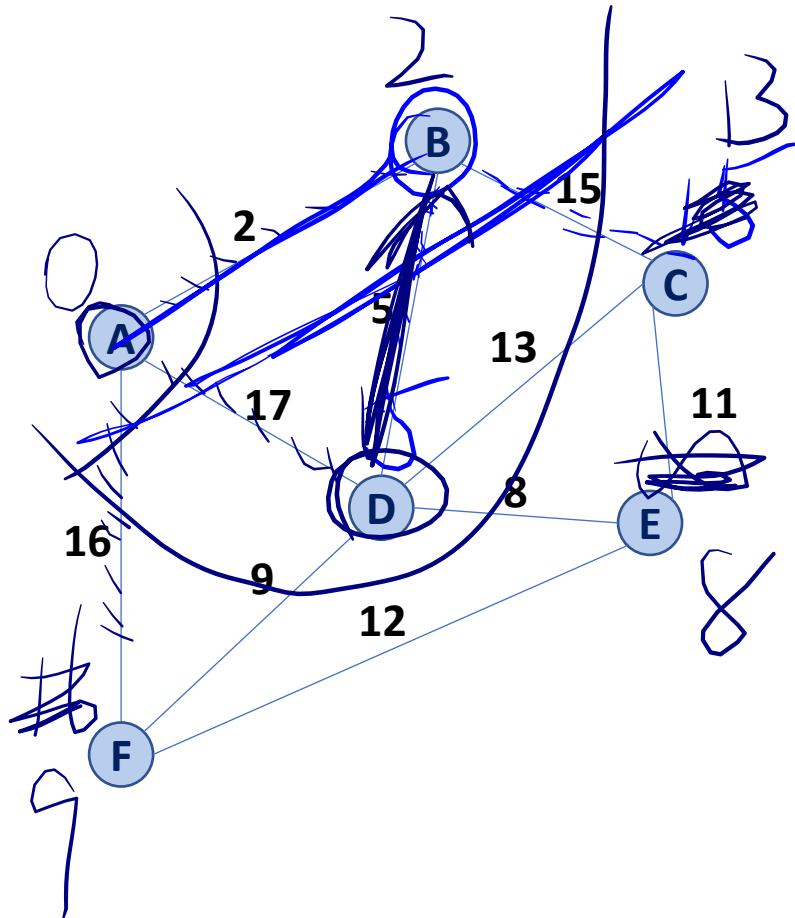
Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16       Vertex m = Q.removeMin()
17       T.add(m)
18       foreach (Vertex v : neighbors of m not in T):
19           if cost(v, m) < d[v]:
20               d[v] = cost(v, m)
21               p[v] = m
22
23   return T
```

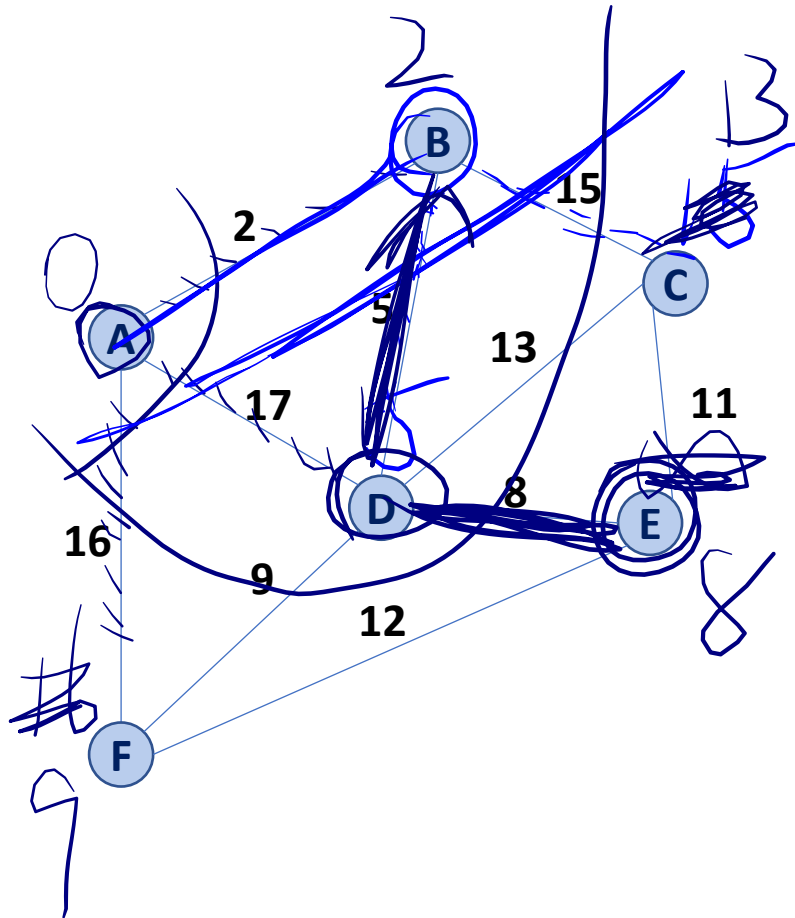
Prim's Algorithm



← vertex A

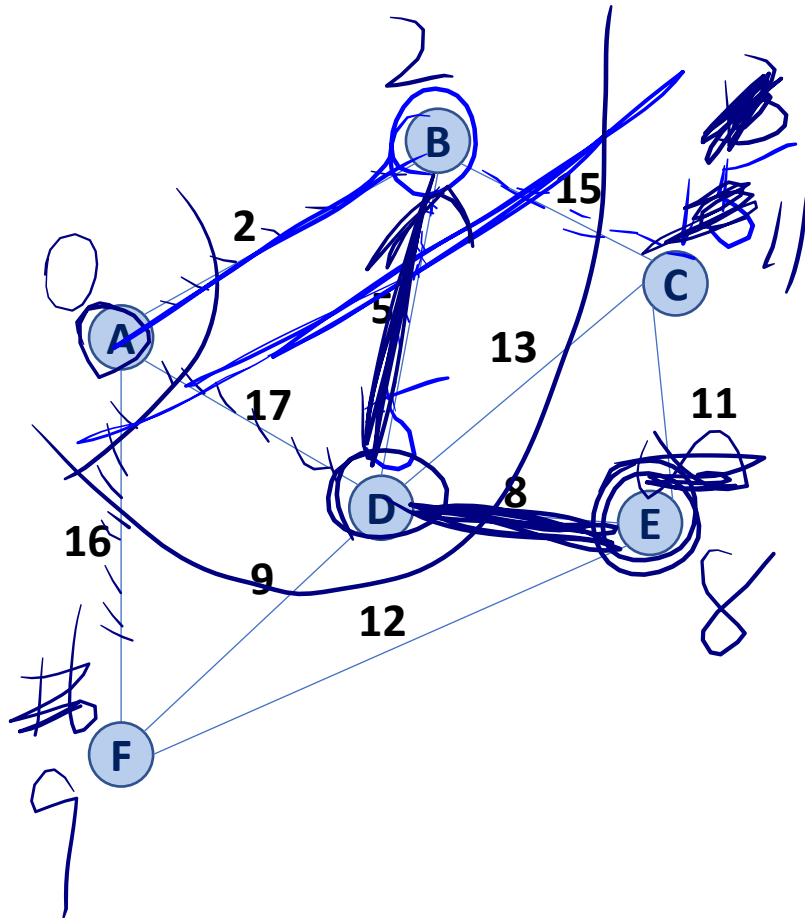
```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Prim's Algorithm



```
1  PrimMST(G, s): ← vertex A
2    Input: G, Graph;
3           s, vertex in G, starting vertex
4    Output: T, a minimum spanning tree (MST) of G
5
6    foreach (Vertex v : G):
7        d[v] = +inf
8        p[v] = NULL
9    d[s] = 0
10
11    PriorityQueue Q // min distance, defined by d[v]
12    Q.buildHeap(G.vertices())
13    Graph T // "labeled set"
14
15    repeat n times:
16        Vertex m = Q.removeMin()
17        T.add(m)
18        foreach (Vertex v : neighbors of m not in T):
19            if cost(v, m) < d[v]:
20                d[v] = cost(v, m)
21                p[v] = m
22
23    return T
```

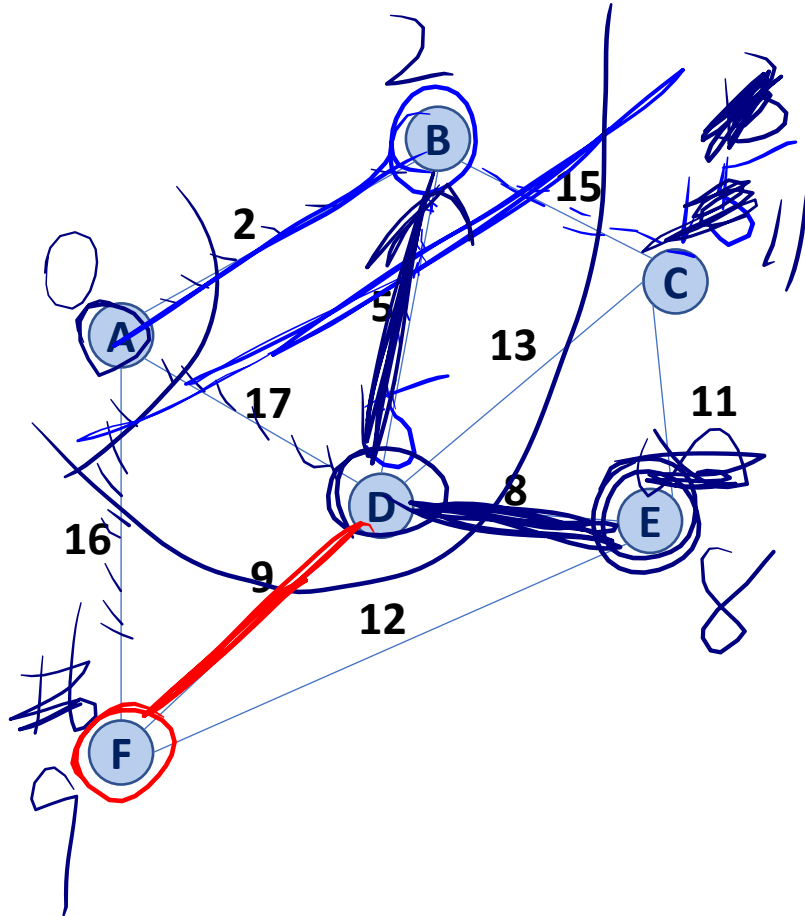

Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

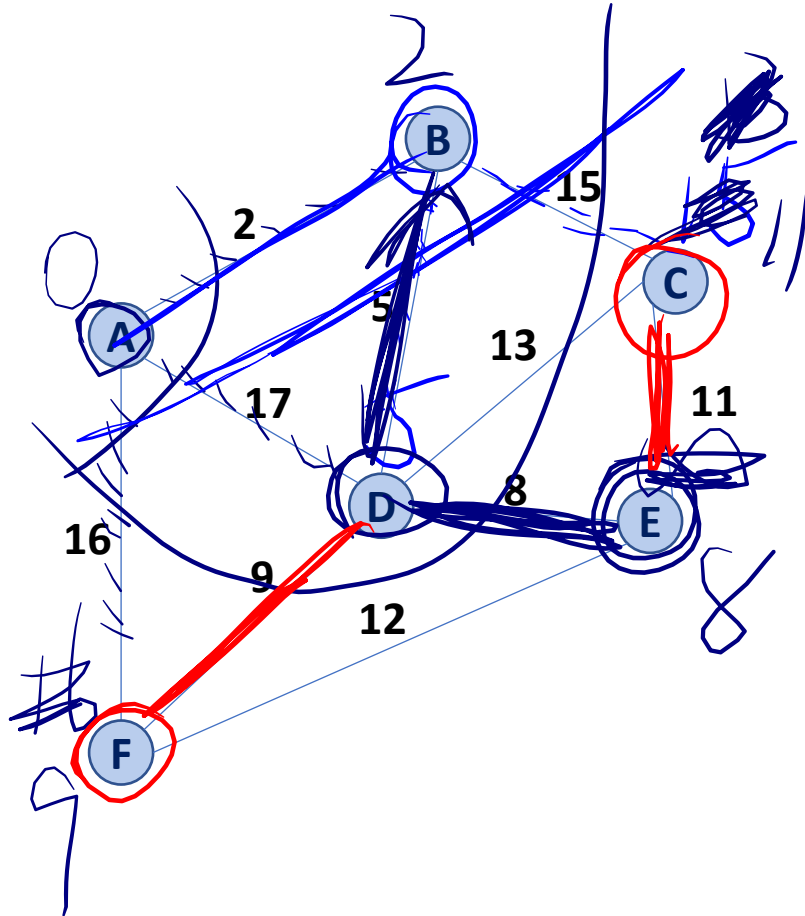
Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16       Vertex m = Q.removeMin()
17       T.add(m)
18       foreach (Vertex v : neighbors of m not in T):
19           if cost(v, m) < d[v]:
20               d[v] = cost(v, m)
21               p[v] = m
22
23   return T
```

Prim's Algorithm



← vertex A

```
1 PrimMST(G, s):
2   Input: G, Graph;
3         s, vertex in G, starting vertex
4   Output: T, a minimum spanning tree (MST) of G
5
6   foreach (Vertex v : G):
7     d[v] = +inf
8     p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T // "labeled set"
14
15   repeat n times:
16     Vertex m = Q.removeMin()
17     T.add(m)
18     foreach (Vertex v : neighbors of m not in T):
19       if cost(v, m) < d[v]:
20         d[v] = cost(v, m)
21         p[v] = m
22
23   return T
```

Prim's Algorithm

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

Handwritten red note: } O(n)

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7    foreach (Vertex v : G):
8      d[v] = +inf
9      p[v] = NULL
10   d[s] = 0
11
12   PriorityQueue Q // min distance, defined by d[v]
13   Q.buildHeap(G.vertices())
14   Graph T          // "labeled set"
15
16   repeat n times:
17     Vertex m = Q.removeMin()
18     T.add(m)
19     foreach (Vertex v : neighbors of m not in T):
20       if cost(v, m) < d[v]:
21         d[v] = cost(v, m)
22         p[v] = m

```

Handwritten red annotations:

- A red bracket on the right side of lines 7-10, labeled $O(n)$.
- A red bracket on the left side of lines 12-14, labeled $O(n)$.

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m

```

Handwritten red annotations: $O(n)$ is written next to line 12, and $O(n)$ is written next to line 16. A large red bracket groups lines 7 through 10.

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m

```

$O(n)$
 $O(n)$
 $O(\log n)$

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m

```

$O(n)$
 $O(n)$
 $O(\log n)$
 $O(\deg(v))$

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7    foreach (Vertex v : G):
8      d[v] = +inf
9      p[v] = NULL
10   d[s] = 0
11
12   PriorityQueue Q // min distance, defined by d[v]
13   Q.buildHeap(G.vertices())
14   Graph T          // "labeled set"
15
16   repeat n times:
17     Vertex m = Q.removeMin()
18     T.add(m)
19     foreach (Vertex v : neighbors of m not in T):
20       if cost(v, m) < d[v]:
21         d[v] = cost(v, m)
22         p[v] = m

```

$O(n)$
 $O(n)$
 $O(\log n)$
 $O(\deg(v))$
 $O(\log n)$

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

```

6  PrimMST(G, s):
7    foreach (Vertex v : G):
8      d[v] = +inf
9      p[v] = NULL
10   d[s] = 0
11
12   PriorityQueue Q // min distance, defined by d[v]
13   Q.buildHeap(G.vertices())
14   Graph T          // "labeled set"
15
16   repeat n times:
17     Vertex m = Q.removeMin()
18     T.add(m)
19     foreach (Vertex v : neighbors of m not in T):
20       if cost(v, m) < d[v]:
21         d[v] = cost(v, m)
22         p[v] = m

```

$\sum_{v \in V} \deg(v)$
 $O(m \lg n)$

$O(n)$
 $O(\lg n)$
 $O(\deg(v))$
 $O(\lg n)$

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) \leftarrow O(m \lg(n))$$

$$\begin{matrix} O(n) \\ O(\lg(n)) \\ O(\deg(v)) \\ O(\lg(n)) \end{matrix}$$

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m
    
```

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) \rightarrow O(m \lg(n))$$

$$O(n) + O(\lg(n)) + O(\deg(v)) + O(\lg(n))$$

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m
    
```

	Adj. Matrix	Adj. List
Heap		
Unsorted Array		

Prim's Algorithm

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) \rightarrow O(m \lg(n))$$

$$O(n) + O(\lg(n)) + O(\deg(v)) + O(\lg(n))$$

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m
    
```

	Adj. Matrix	Adj. List
Heap		$O(n \lg(n) + m \lg(n))$
Unsorted Array		

Prim's Algorithm

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) \rightarrow O(m \lg(n))$$

$$O(n) \\ O(\lg(n)) \\ O(\deg(v)) \\ O(\lg(n))$$

```

6  PrimMST(G, s):
7      foreach (Vertex v : G):
8          d[v] = +inf
9          p[v] = NULL
10     d[s] = 0
11
12     PriorityQueue Q // min distance, defined by d[v]
13     Q.buildHeap(G.vertices())
14     Graph T          // "labeled set"
15
16     repeat n times:
17         Vertex m = Q.removeMin()
18         T.add(m)
19         foreach (Vertex v : neighbors of m not in T):
20             if cost(v, m) < d[v]:
21                 d[v] = cost(v, m)
22                 p[v] = m
    
```

	Adj. Matrix	Adj. List
Heap		$O(n \lg(n) + m \lg(n))$
Unsorted Array		

Prim's Algorithm

```

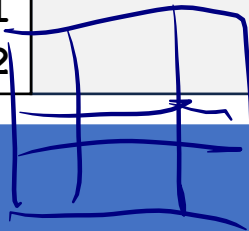
6  PrimMST(G, s):
7    foreach (Vertex v : G):
8      d[v] = +inf
9      p[v] = NULL
10   d[s] = 0
11
12   PriorityQueue Q // min distance, defined by d[v]
13   Q.buildHeap(G.vertices())
14   Graph T          // "labeled set"
15
16   repeat n times:
17     Vertex m = Q.removeMin()
18     T.add(m)
19     foreach (Vertex v : neighbors of m not in T):
20       if cost(v, m) < d[v]:
21         d[v] = cost(v, m)
22         p[v] = m

```

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) = O(m \lg(n))$$

$$O(n) + O(\lg(n)) + O(\deg(v)) + O(\lg(n))$$



	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \lg(n))$	$O(n \lg(n) + m \lg(n))$
Unsorted Array		

Prim's Algorithm

```

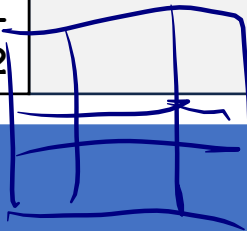
6  PrimMST(G, s):
7    foreach (Vertex v : G):
8      d[v] = +inf
9      p[v] = NULL
10   d[s] = 0
11
12   PriorityQueue Q // min distance, defined by d[v]
13   Q.buildHeap(G.vertices())
14   Graph T          // "labeled set"
15
16   repeat n times:
17     Vertex m = Q.removeMin()
18     T.add(m)
19     foreach (Vertex v : neighbors of m not in T):
20       if cost(v, m) < d[v]:
21         d[v] = cost(v, m)
22         p[v] = m

```

$$O(n) + O(n) + O(n \lg(n)) + O(m \lg(n))$$

$$\sum_{v \in V} \deg(v) = O(m \lg(n))$$

$$O(n) + O(\lg(n)) + O(\deg(v)) + O(\lg(n))$$



	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \lg(n))$	$O(n \lg(n) + m \lg(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

Dense Graph:

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \log(n))$	$O(n \log(n) + m \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

Dense Graph:

$n^2 \sim m$

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \log(n))$	$O(n \log(n) + m \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

Dense Graph:

$n^2 \sim m$

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + \cancel{m} \log(n))$	$O(n \log(n) + \cancel{m} \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

$n \sim m$

Dense Graph:

$n^2 \sim m$

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + \cancel{m} \log(n))$	$O(n \log(n) + \cancel{m} \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

Prim's Algorithm

Sparse Graph:

$n \sim m$

Dense Graph:

$n^2 \sim m$

```
6 PrimMST(G, s):
7   foreach (Vertex v : G):
8     d[v] = +inf
9     p[v] = NULL
10  d[s] = 0
11
12  PriorityQueue Q // min distance, defined by d[v]
13  Q.buildHeap(G.vertices())
14  Graph T          // "labeled set"
15
16  repeat n times:
17    Vertex m = Q.removeMin()
18    T.add(m)
19    foreach (Vertex v : neighbors of m not in T):
20      if cost(v, m) < d[v]:
21        d[v] = cost(v, m)
22        p[v] = m
```

	Adj. Matrix	Adj. List
Heap	$O(n^2 + m \log(n))$	$O(n \log(n) + m \log(n))$
Unsorted Array	$O(n^2)$	$O(n^2)$

MST Algorithm Runtime:

- Kruskal's Algorithm:

$O(n + m \log(n))$

- Prim's Algorithm:

$O(n \log(n) + m \log(n))$

- What must be true about the connectivity of a graph when running an MST algorithm?
- How does n and m relate?

MST Algorithm Runtime:

- Kruskal's Algorithm:

$O(n + m \log(n))$

- Prim's Algorithm:

$O(n \log(n) + m \log(n))$

- What must be true about the connectivity of a graph when running an MST algorithm?

Min edges $n-1$

- How does n and m relate?

MST Algorithm Runtime:

- Kruskal's Algorithm:

$O(n + m \log(n))$

- Prim's Algorithm:

$O(n \log(n) + m \log(n))$

- What must be true about the connectivity of a graph when running an MST algorithm?

min edges $n-1$

max edges n^2

- How does n and m relate?

MST Algorithm Runtime:

- Kruskal's Algorithm:

$$O(n + m \log(n))$$

- Prim's Algorithm:

$$O(n \log(n) + m \log(n))$$

- What must be true about the connectivity of a graph when running an MST algorithm?

min edges $n-1$

max edges n^2

$$O(n) \sim O(m) \quad m \geq n-1$$

- How does n and m relate?

MST Algorithm Runtime:

- Kruskal's Algorithm:

$$O(n + m \log(n))$$

- Prim's Algorithm:

$$O(\cancel{n \log(n)} + m \log(n))$$

- What must be true about the connectivity of a graph when running an MST algorithm?

min edges $n-1$

max edges n^2

$$O(n) \sim O(m) \quad m \geq n-1$$

- How does n and m relate?

MST Algorithm Runtime:

- Kruskal's Algorithm:

$$O(n + m \log(n))$$

- Prim's Algorithm:

$$O(\cancel{n \log(n)} + m \log(n))$$

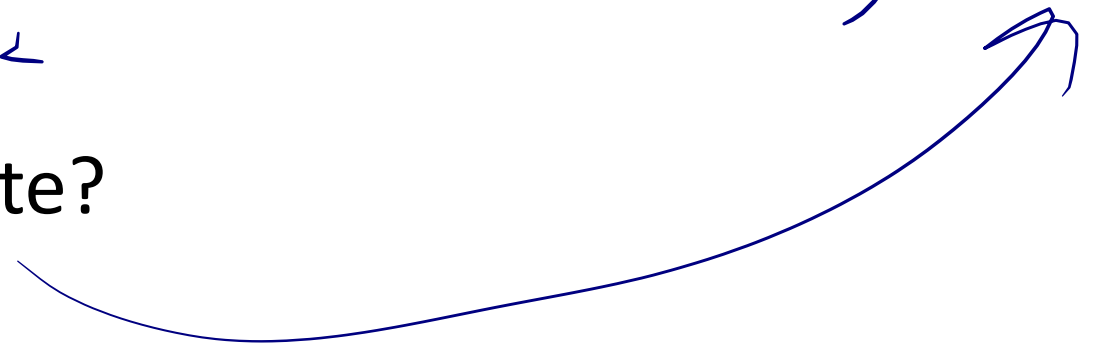
- What must be true about the connectivity of a graph when running an MST algorithm?

min edges $n-1$

max edges n^2

$$O(n) \sim O(m) \quad m \geq n-1$$

- How does n and m relate?



MST Algorithm Runtime:

- Upper bound on MST Algorithm Runtime:
 $O(m \log(n))$

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

```
PrimMST(G, s):
6   foreach (Vertex v : G):
7       d[v] = +inf
8       p[v] = NULL
9   d[s] = 0
10
11   PriorityQueue Q // min distance, defined by d[v]
12   Q.buildHeap(G.vertices())
13   Graph T          // "labeled set"
14
15   repeat n times:
16       Vertex m = Q.removeMin()
17       T.add(m)
18       foreach (Vertex v : neighbors of m not in T):
19           if cost(v, m) < d[v]:
20               d[v] = cost(v, m)
21               p[v] = m
```

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

```
PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
```

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$O(n)$

```
PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
```


Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$O(n)$ [

$O(n)$ [

```
PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
```

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

	<pre>PrimMST(G, s): 6 foreach (Vertex v : G): 7 d[v] = +inf 8 p[v] = NULL 9 d[s] = 0 10 11 PriorityQueue Q // min distance, defined by d[v] 12 Q.buildHeap(G.vertices()) 13 Graph T // "labeled set" 14 15 repeat n times: 16 Vertex m = Q.removeMin() 17 T.add(m) 18 foreach (Vertex v : neighbors of m not in T): 19 if cost(v, m) < d[v]: 20 d[v] = cost(v, m) 21 p[v] = m</pre>
--	---

$O(n)$

$O(n)$

$O(n)$

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

	<pre>PrimMST(G, s): 6 foreach (Vertex v : G): 7 d[v] = +inf 8 p[v] = NULL 9 d[s] = 0 10 11 PriorityQueue Q // min distance, defined by d[v] 12 Q.buildHeap(G.vertices()) 13 Graph T // "labeled set" 14 15 repeat n times: 16 Vertex m = Q.removeMin() 17 T.add(m) 18 foreach (Vertex v : neighbors of m not in T): 19 if cost(v, m) < d[v]: 20 d[v] = cost(v, m) 21 p[v] = m</pre>
--	---

$O(n)$

$O(n)$

$O(n)$
 $O(\log(n))$

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

	$O(n)$	6	PrimMST(G, s):
		7	foreach (Vertex v : G):
		8	d[v] = +inf
		9	p[v] = NULL
		10	d[s] = 0
		11	PriorityQueue Q // min distance, defined by d[v]
		12	Q.buildHeap(G.vertices())
		13	Graph T // "labeled set"
		14	
		15	repeat n times:
		16	Vertex m = Q.removeMin()
		17	T.add(m)
		18	foreach (Vertex v : neighbors of m not in T):
		19	if cost(v, m) < d[v]:
		20	d[v] = cost(v, m)
		21	p[v] = m

$O(n)$

$O(n)$

$O(n)$
 $O(\log n)$
 $O(\deg(v))$

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

		PrimMST(G, s):
	$O(n)$	6 foreach (Vertex v : G):
		7 d[v] = +inf
		8 p[v] = NULL
		9 d[s] = 0
		10
	$O(n)$	11 PriorityQueue Q // min distance, defined by d[v]
		12 Q.buildHeap(G.vertices())
		13 Graph T // "labeled set"
		14
	$O(n)$	15 repeat n times:
	$O(\lg n)$	16 Vertex m = Q.removeMin()
		17 T.add(m)
$O(2m)$	$O(\deg(v))$	18 foreach (Vertex v : neighbors of m not in T):
		19 if cost(v, m) < d[v]:
		20 d[v] = cost(v, m)
		21 p[v] = m

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

	<pre>PrimMST(G, s): 6 foreach (Vertex v : G): 7 d[v] = +inf 8 p[v] = NULL 9 d[s] = 0 10 11 PriorityQueue Q // min distance, defined by d[v] 12 Q.buildHeap(G.vertices()) 13 Graph T // "labeled set" 14 15 repeat n times: 16 Vertex m = Q.removeMin() 17 T.add(m) 18 foreach (Vertex v : neighbors of m not in T): 19 if cost(v, m) < d[v]: 20 d[v] = cost(v, m) 21 p[v] = m</pre>
$O(n)$	lines 6-9
$O(n)$	lines 11-13
$O(n)$	line 15
$O(\log n)$	line 16
$O(1)$	line 17
$O(\deg(v))$	line 18
$O(1)$	lines 19-21
$O(2n)$	total

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$n \lg(n)$

$O(2m)$

$O(1)$

$O(\lg n)$

$O(\deg(v))$

$O(n)$

$O(n)$

$O(n)$

$O(\lg n)$

$O(1)$

$O(\deg(v))$

```

PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m

```

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$n(\lg(n) + m)$

$O(2m)$ — $O(\lg(n))$ — $O(\deg(v))$ — $O(1)$

$O(n)$

$O(n)$

```

PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
        
```


Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$O(n)$

$O(n)$

$n(\lg(n) + m)$

$O(2m)$

$O(\lg(n))$

$O(\deg(v))$

$O(1)$

```

PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
        
```

Suppose I have a new heap:

	Binary Heap	Fibonacci Heap
Remove Min	$O(\log(n))$	$O(\log(n))$
Decrease Key	$O(\log(n))$	$O(1)^*$

What's the updated running time?

$m \lg(n)$

$\ln(\lg(n)) + m$

$O(2m)$

$O(1)$

$O(\lg(n))$

$O(\deg(v))$

$O(n)$

$O(n)$

```

PrimMST(G, s):
6  foreach (Vertex v : G):
7      d[v] = +inf
8      p[v] = NULL
9  d[s] = 0
10
11  PriorityQueue Q // min distance, defined by d[v]
12  Q.buildHeap(G.vertices())
13  Graph T          // "labeled set"
14
15  repeat n times:
16      Vertex m = Q.removeMin()
17      T.add(m)
18      foreach (Vertex v : neighbors of m not in T):
19          if cost(v, m) < d[v]:
20              d[v] = cost(v, m)
21              p[v] = m
        
```